

Fraus

Medição de satisfação sem LLM em runtime

Table of contents

1. O cliente <i>diz</i> uma coisa. O texto mede outra.	7
1.1 O que ler primeiro	8
1.2 Sem LLM em runtime	9
1.3 Baixar	9
2. A tese	10
2.1 O problema	10
2.2 A aposta	10
2.3 O que sai disso, e que nada mais consegue calcular	10
2.4 O que a tese NÃO promete	10
3. As dez invariantes	12
3.1 1. Sem LLM em runtime	12
3.2 2. Ausência de dado não é insatisfação	12
3.3 3. Score, nota e categoria são derivados no servidor	12
3.4 4. Faixas de NPS: 0-6 detrator, 7-8 neutro, 9-10 promotor	13
3.5 5. Latência nunca é persistida	13
3.6 6. Timestamps timezone-aware	13
3.7 7. Modelo ausente é falha alta e explícita	14
3.8 8. A ordem das classes é 0 insatisfeito, 1 neutro, 2 satisfeito	14
3.9 9. As 39 chaves de feature batem exatamente com NOMES_FEATURES	14
3.10 10. O corpus de treino não pode entregar o rótulo	15
4. Arquitetura	16
4.1 A decisão que explica o desenho	16
4.2 As sete famílias que pontuam	17
4.3 Onde roda, e com que motor	17
4.4 A ironia continua carregada, e não pontua	18
4.5 Régua: o que pontuou uma conversa não é fixo no tempo	18
4.6 Repontuar é um trabalho, não uma requisição	18
4.7 Onde cada coisa mora	20
5. Treinamento dos modelos do Fraus	21
5.1 Como abrir no Colab	22
5.2 Notebook 01 — classificador de texto	22
5.2.1 Exigência de GPU	22
5.2.2 Onde o checkpoint fica no Drive	22
5.2.3 Para onde copiar o artefato no repo local	22
5.2.4 Mapeamento de rotulo	23

5.3	Notebook 02 — fusor dos tres sinais	24
5.3.1	Rotulo do fusor: recommend_to_a_friend	24
5.3.2	Limitacao metodologica a declarar no relatorio	25
5.3.3	O vazamento do primeiro fusor — 13/08/2026	25
5.3.4	O fusor de 35 features — 24/08/2026	26
5.3.5	Consequencia no NPS: a classe neutra cai em detrator — HISTORICO (fusor de 16 features, ate 24/08/2026)	27
5.3.6	Retreino do fusor apos as features de incongruencia (03/09/2026)	28
5.3.7	A ironia sai do vetor (04/09/2026)	30
5.3.8	O fusor de 38 features — 04/09/2026	31
5.3.9	O contrato sobe para 39: incongruencia implicita (04/09/2026)	32
5.3.10	Conferencia do artefato	32
5.4	Contrato de features	33
5.5	Notebook 03 — classificador de emocao	34
5.5.1	Ordem canonica das classes	34
5.5.2	Corpus: go_emotions_ptbr	34
5.5.3	O desequilibrio, e por que NAO subamostrar	35
5.5.4	O teto do desprezo — a declarar no relatorio	35
5.5.5	Por que o desprezo NAO e treinado	35
5.6	Notebook 04 — classificador de ironia	36
5.6.1	Fontes publicas do treino e teste oficial fechado	36
5.6.2	A divisao por autor — conserto de 02/10/2026	36
5.6.3	Candidato Laya	37
5.6.4	Alternativas ao IDPT — levantamento de 14/08/2026	38
5.6.5	Onde existe corpus PT-BR de ironia (e como pedir)	39
5.6.6	O padrao: corpus sintetico	39
5.6.7	Portao de promocao do artefato	40
5.7	Sinal lexico — SentiLex-PT02	40
5.8	Sinal de estilo	41
5.9	Notebook 05 — calibração, e por que ele não retreina nada	42
5.10	Notebook 02 — a seção 10 é uma pendência, não uma etapa	42
5.10.1	As três saídas, medidas em 15/09/2026 — decidido: só log1p	42
5.10.2	Promovido em 15/09/2026	43
6.	Ironia: corpus e Laya	45
6.1	Resumo: consertar o corpus antes de treinar o Laya	45
6.2	O IDPT entrega o rótulo, e nenhum encoder escapa disso	45
6.3	Treinar só ironia exige mexer em sete células e num rótulo de produção	46
6.4	Consertar o negativo vale mais que trocar o encoder	49
6.5	Três treinos e uma régua de 300 pares separam as causas	49

6.6	O ganho esperado é de poucos pontos, e vem do dado	51
6.7	Régua de pares mínimos	52
6.8	Conclusão	52
7.	Integração com o Tars	54
7.1	Contrato	54
7.2	Idempotência e privacidade	54
7.3	Automação opcional	54
8.	Design System: Fraus — Instrumento	55
8.1	Overview	55
8.2	Colors	56
8.2.1	Primary	56
8.2.2	Secondary (as duas vozes)	56
8.2.3	Tertiary (categorias, famílias, grafo)	56
8.2.4	Neutral	57
8.2.5	Named Rules	57
8.3	Typography	57
8.3.1	Hierarchy	58
8.3.2	Named Rules	58
8.4	Layout	58
8.4.1	Named Rules	59
8.5	Elevation & Depth	59
8.5.1	Named Rules	59
8.6	Shapes	59
8.7	Components	59
8.7.1	SegmentoLED (peça-assinatura)	59
8.7.2	TelemetriaTopo	60
8.7.3	Navegação numerada	60
8.7.4	Painel (hairline)	60
8.7.5	Armadura de indicadores	60
8.7.6	Cabeça da categoria (EtiquetaCategoria, slot único)	60
8.7.7	Vitrine — a LP (/, ln-*)	61
8.7.8	Ícone de ação	62
8.7.9	Estados (coreografia)	62
8.7.10	Easter egg da mentira (lib/mentira.tsx)	62
8.7.11	Motion	62
8.8	Do's and Don'ts	63
8.8.1	Do:	63
8.8.2	Don't:	63

8.9	Temas e gates	63
8.10	Adições e desvios conscientes desta rodada	64
9.	Conformidade regulatória — o Fraus e o EU AI Act	65
9.1	1. Por que este documento existe	65
9.2	2. Escopo do sistema	65
9.3	3. As três obrigações que já valem, e a que vem	65
9.3.1	3.1 Transparência como compromisso do produto	65
9.3.2	3.2 Limite de finalidade: não pontuar trabalhadores	66
9.3.3	Regra: o Fraus não pontua atendentes.	66
9.3.4	3.3 Controles recomendados para uma implantação comercial	66
9.4	4. Limitações que importam para quem lê um número do Fraus	67
9.5	5. Fontes	67
10.	Encolher os modelos — o que foi medido, e por que a int8 foi recusada	69
10.1	O problema	69
10.2	O que foi testado	69
10.3	O resultado da melhor variante	70
10.4	E é aqui que ela é recusada	70
10.5	A variante mista, medida em 11/09/2026	71
10.5.1	De quem é a culpa, então	71
10.5.2	E o que isso faz com a máquina de 1 GB	72
10.5.3	O que sobra de ganho real	72
10.6	O backend ONNX em produção, medido em 15/09/2026	73
10.6.1	Emoção com int8 só nos pesos — aceita pelo portão, não promovida por padrão	74
10.7	O que sobra como caminho	74
10.8	Reproduzir	75
10.9	Variante experimental: um encoder, três cabeças	75
11.	Limitações conhecidas	76
11.1	Limitações operacionais do deploy gratuito	76
11.2	O NPS é inferido, não perguntado	76
11.3	O sinal de tempo é treinado em dados sintéticos	77
11.4	A cabeça de ironia é pouco confiável	77
11.5	Desprezo não é uma classe treinada	77
11.6	Nenhuma feature agregada reverte uma probabilidade saturada	78
11.7	O que não dá para consertar sem corpus de atendimento real	78
12.	Referência de código	79
12.1	Indicadores	79
12.1.1	fraus.indicadores	79

12.2	Modelo canônico	83
12.2.1	fraus.modelos	83
12.3	O fusor e as 39 features	84
12.3.1	fraus.fusor	84
12.4	Os sinais	88
12.4.1	Texto — o único que pontua sozinho	88
12.4.2	fraus.sinais.texto	88
12.4.3	Emoção	89
12.4.4	fraus.sinais.emocao	89
12.4.5	Tempo	90
12.4.6	fraus.sinais.tempo	90
12.4.7	Léxico	90
12.4.8	fraus.sinais.lexico	90
12.5	Evidência e deriva	92
12.5.1	Evidência fraca — a cabeça tracejada	92
12.5.2	fraus.evidencia	92
12.5.3	Deriva de distribuição — a invariante 10 em runtime	93
12.5.4	fraus.deriva	93

FRAUS — DOCUMENTAÇÃO

1. O cliente diz
uma coisa. O
texto mede
outra.

Ferramenta que lê atendimento por chatbot e estima satisfação **sem perguntar nada ao cliente**. O caso que uma pesquisa de NPS declarada não captura: quem escreve "*ok, obrigado 😊*" e sai insatisfeito não responde pesquisa — e é ele que o sistema precisa enxergar.

LER AS INVARIANTES	A TESE	ARQUITETURA
--------------------	--------	-------------

FAMÍLIAS DE SINAL 00	FEATURES NO FUSOR 39	BERTIMBAU FINE-TUNADOS 03	LLMS NA INFERÊNCIA 00
-------------------------	-------------------------	------------------------------	--------------------------

 TODO NÚMERO AQUI É INFERIDO

O NPS do Fraus **não é perguntado**: ele é derivado do texto do atendimento. Toda exibição carrega a etiqueta de estimativa, e apresentá-lo como NPS declarado seria falso. Ver [Limitações](#) e [Conformidade](#).

1.1 O QUE LER PRIMEIRO

A TESE

A aposta do projeto e o caso que ela cobre.

AS INVARIANTES

Para mexer no código: comece por aqui, sem exceção.

ARQUITETURA

Como as peças se ligam, do adaptador de ingestão ao dashboard.

TREINAMENTO

Como os modelos foram treinados e o que foi medido.

DESIGN DA INTERFACE

O sistema visual: LED, régua dito/medido e as regras.

LIMITAÇÕES

O que o sistema **não** sabe, dito sem rodeio.

CONFORMIDADE

O que o produto lê e o que ele se recusa a fazer.

ENCOLHER OS MODELOS

Como o runtime ficou pequeno sem trocar o modelo, medido.

CARTÃO DO MODELO

Desempenho por fatia, sem esconder a fatia ruim.

1.2 SEM LLM EM RUNTIME

Três BERTimbau fine-tunados (satisfação, emoção, ironia) e um fusor `LogisticRegression` sobre 39 features de sete famílias de sinal. Inferência local, em CPU, sem nenhuma chamada de rede no caminho de predição.

Isso é **requisito**, não otimização: o trabalho existe para demonstrar que dá para medir isso com modelos pequenos e interpretáveis — coeficiente de regressão logística responde *por que* um atendimento recebeu a nota, e floresta densa não.

1.3 BAIXAR

O PDF com o site inteiro está em [pdf/fraus.pdf](#), gerado no mesmo build que publica estas páginas — então ele nunca está mais velho que o site.

2. A tese

■ 2.1 O PROBLEMA

Quem opera um chatbot de atendimento mede satisfação de dois jeitos, e os dois falham no mesmo ponto.

Pesquisa declarada (NPS, CSAT). Depende de resposta, e a taxa de resposta é baixa e enviesada: responde quem ficou muito irritado ou muito satisfeito. O cliente do meio — o que resolveu o problema com esforço, ou desistiu no meio — simplesmente não aparece.

Taxa de contenção. Mede se o bot segurou o atendimento sem escalar. É um número de **volume**, não de qualidade: sobe igual quando o bot resolve e quando o cliente desiste de tentar.

Cruzados, os dois produzem o pior cenário possível: contenção alta com satisfação não medida. O painel fica verde e ninguém sabe se a operação está boa.

■ 2.2 A APOSTA

O texto do atendimento **já contém** a evidência da satisfação. Ela não precisa ser perguntada — precisa ser lida.

O caso canônico do projeto é este:

"que atendimento maravilhoso, só esperei 3 horas"

Uma pesquisa nunca vai receber essa frase. Um classificador de sentimento ingênuo lê "maravilhoso" e marca promotor. O sistema precisa das duas metades ao mesmo tempo: o que foi **dito** e o que foi **medido**.

■ 2.3 O QUE SAI DISSO, E QUE NADA MAIS CONSEGUE CALCULAR

Falso containment. O cruzamento de "não escalou" com "saiu detrator". É a tese virando número, e só quem infere satisfação do texto consegue calcular: quem depende de pesquisa respondida perde justamente o cliente contido, que é o que não responde.

■ 2.4 O QUE A TESE NÃO PROMETE

- **Não é NPS declarado.** É estimativa, e a interface diz isso em todo lugar.
- **Não decide nada sozinha.** Nenhuma saída aciona ação sobre ninguém.
- **Não pontua atendentes** — ver [Conformidade](#).

- **Não é infalível, e diz onde falha.** O sinal de tempo é treinado em dados sintéticos; a cabeça de ironia é pouco confiável e por isso não pontua; fora da faixa de latência do treino o relógio domina a nota. Tudo isso está em [Limitações](#), com número.

3. As dez invariantes

Não são preferências de estilo. **Cada uma nasceu de um defeito real**, e violá-la é reintroduzir o defeito — por isso cada regra aqui vem com a cicatriz que a produziu. Documentação que explica cicatriz é a que não dá para copiar de tutorial.

O formato é sempre o mesmo: **a regra**, *por que ela existe*, **o que quebra se alguém a violar**, e onde ela mora no código.

■ 3.1 1. SEM LLM EM RUNTIME

A regra. Inferência local em CPU. Nenhuma chamada de rede no caminho de predição.

Por que existe. É requisito do trabalho, não otimização de custo. A tese é que dá para medir satisfação com modelos pequenos, fine-tunados e interpretáveis — chamar uma API de LLM não deixaria o sistema mais rápido, e invalidaria o que ele se propõe a demonstrar.

O que quebra. O trabalho inteiro. Nenhum teste pega isso: o código funcionaria melhor e a defesa cairia.

Onde mora. Três BERTimbau em `fraus/sinais/` e um `LogisticRegression` em `fraus/fusor.py`.

■ 3.2 2. AUSÊNCIA DE DADO NÃO É INSATISFAÇÃO

A regra. Conversa sem fala do cliente tem `score: None` e aparece como "sem sinal" — nunca `0`, em lugar nenhum: célula, gráfico, ordenação, export, agregado.

Por que existe. `None` e `0` significam coisas diferentes e não podem se confundir. Zero numa coluna de tempo de resposta se lê como "respondeu na hora", e a conversa que nunca teve atendente humano apareceria como **a mais ágil da operação**. Zero num NPS se lê como um número que alguém mediu.

O que quebra. Silenciosamente, tudo: o pior valor da escala aparece onde a verdade era "não medi". Em 10/09/2026 essa regra ainda estava furada dentro do próprio arquivo que mais a defende — `containment_rate` devolvia `0.0` no conjunto vazio, e o front tinha uma guarda para não acreditar nele.

Onde mora. Em toda parte. Procure `?? 0` e `|| 0` antes de commitar — e **também `return 0` no Python**, que é o mesmo defeito disfarçado de inicialização.

■ 3.3 3. SCORE, NOTA E CATEGORIA SÃO DERIVADOS NO SERVIDOR

A regra. Nunca aceitos do corpo da requisição, nunca recalculados no cliente.

Por que existe. O `round` do Python é bancário (`round(6.5) == 6`) e o `Math.round` do JavaScript arredonda meio para cima (`Math.round(6.5) == 7`).

O que quebra. Já quebrou: a tabela exibia **nota 7 ao lado da etiqueta "Detrator"**, nas fronteiras 6/7 e 8/9. Duas fontes da verdade para o mesmo número, discordando na tela do usuário.

Onde mora. `fraus/indicadores.py::nota 0 10`. O front consome; não recalcula.

■ 3.4 4. FAIXAS DE NPS: 0-6 DETRATOR, 7-8 NEUTRO, 9-10 PROMOTOR

A regra. Padrão de fábrica em `FAIXAS NPS`, configurável por `PUT /configuracoes` — e só se as faixas cobrirem 0..10 de forma contígua. A faixa vigente é passada **por parâmetro**, nunca lida de estado global.

Por que existe. Estado global mutável muda sob os pés de quem já calculou. E faixa com buraco ou sobreposição não é preferência de gosto: é nota sem categoria, ou com duas.

O que quebra. Já quebrou: `categoriaDaNota` no front tinha `<= 6` e `<= 8` digitados no corpo, e isso rodava no caminho feliz. Um operador que mexesse em Configurações via a **mesma tela dar dois vereditos** para o mesmo atendimento — a etiqueta da linha com as faixas novas, a cor da barra com as antigas.

Onde mora. `fraus/indicadores.py::FAIXAS NPS`, `categoria nps`, `validar faixas nps`.

■ 3.5 5. LATÊNCIA NUNCA É PERSISTIDA

A regra. Sempre derivada dos timestamps, na leitura.

Por que existe. Latência gravada é latência que envelhece: qualquer correção na definição de "espera" deixaria o banco cheio de números calculados pela regra antiga, indistinguíveis dos novos.

O que quebra. Duas definições de latência convivendo — uma para a lista, outra para o detalhe — e as duas telas discordando sobre o mesmo atendimento.

Onde mora. `fraus/sinais/tempo.py::latencias da conversa`.

■ 3.6 6. TIMESTAMPS TIMEZONE-AWARE

A regra. `datetime` naive é erro de validação, não conversão silenciosa.

Por que existe. Data sem fuso é data que muda de significado conforme quem lê.

O que quebra. A armadilha já paga: **a data da Totalk é `MM/DD/YYYY`**, não `DD/MM`. Lida como brasileira, espalha as mensagens por cinco meses e a latência sai absurda **em silêncio**.

Onde mora. `fraus/modelos.py`.

■ 3.7.7. MODELO AUSENTE É FALHA ALTA E EXPLÍCITA

A regra. Sem fallback, sem motor dublê silencioso. A API não sobe sem os três modelos treinados.

Por que existe. Servir predição sem modelo carregado é **pior que estar fora do ar**: a interface não distingue número real de número sintético.

O que quebra. Já quebrou, e custou um dia: o `scripts/api_demo.py` estava no ar por causa do diretório de onde foi lançado, e a dashboard escreveu "API no ar" durante um dia inteiro enquanto **todo número da tela era sintético** — os pesos por feature saíam numa progressão 0,2 / 0,25 / 0,3 e passavam por coeficiente de regressão logística.

Onde mora. `Fusor.carregar` (valida `n features in`), `GET /saude` (`"motor": "real" | "duble"`), e `GET /saude/deriva`, que recusa diagnosticar com dublê pela mesma lógica.

■ 3.8.8. A ORDEM DAS CLASSES É 0 INSATISFEITO, 1 NEUTRO, 2 SATISFEITO

A regra. No notebook, no sinal de texto, no fusor e nos indicadores.

Por que existe. É a única convenção que amarra quatro lugares que não se importam por código.

O que quebra. Nada estoura. O sistema pontua ao contrário, em silêncio, com a mesma confiança de sempre. É a invariante mais barata de violar e a mais cara de descobrir.

Onde mora. `fraus/sinais/texto.py` (`INSATISFEITO`, `NEUTRO`, `SATISFEITO`).

■ 3.9.9. AS 39 CHAVES DE FEATURE BATEM EXATAMENTE COM `NOMES FEATURES`

A regra. As sete famílias do vetor produzem exatamente 39 chaves. `vetorizar` levanta `KeyError` na falta — nunca zero silencioso.

Por que existe. Zero numa feature ausente não é neutro: o modelo aprendeu o que zero significa naquela dimensão, e a conversa entraria como se tivesse sido medida.

O que quebra. Já quebrou de outro jeito: o `CLAUDE.md` chegou a declarar "38" na invariante e "(35)" no mapa — dois números de duas revisões, nenhum o vigente. A documentação que governa toda sessão de agente envelhecendo em silêncio. Hoje há teste amarrando o número do `CLAUDE.md` a `len(NOMES FEATURES)`.

Onde mora. `fraus/fusor.py::NOMES FEATURES` — e o comentário dele conta a história inteira: 40 – 2 (a ironia saiu) + 1 (`incongruencia situacao negativa` entrou) = 39.

■ 3.10 10. O CORPUS DE TREINO NÃO PODE ENTREGAR O RÓTULO

A regra. Distribuição por rótulo se sobrepõe; feature constante no treino nasce com peso zero. **Acurácia alta demais é sintoma, não vitória.**

Por que existe. Faixa de latência disjunta por classe fez o primeiro fusor marcar **99,3% lendo só o relógio**, com o BERTimbau apagado.

O que quebra. Um número excelente que não mede nada. E ela não para no treino: em 08/09/2026 a mesma invariante apareceu **em runtime** — numa conversa com 3 h de espera, a latência contribuiu $-85,4$ contra $+4,6$ do texto inteiro, e o score saiu em $4,4e-24$. O corpus nunca passou de minutos, então a guarda de vazamento, que olha o corpus, não tinha o que dizer.

Onde mora. [docs/treinamento.md](#), e agora também [GET/saude/deriva](#), que transforma a regra de disciplina de notebook em alarme de runtime.

4. Arquitetura

```

flowchart TD
    A["CSV / Discord / WhatsApp / webhook"] -->|"adapter de ingestão<br/>fraus/ingest/"| B["Conversa<br/><small>modelo canônico único — fraus/modelos.py</small>"]

    B --> texto["texto<br/><small>BERTimbau por mensagem</small>"]
    B --> emoji["emoji<br/><small>lexicon + posição</small>"]
    B --> tempo["tempo<br/><small>latência, escalação</small>"]
    B --> emocao["emoção<br/><small>7 classes + desprezo</small>"]
    B --> lexico["léxico<br/><small>SentiLex-PT02 + negação</small>"]
    B --> estilo["estilo<br/><small>caixa alta, palavrão</small>"]
    B --> incong["incongruência<br/><small>emoji x texto</small>"]
    B --> ironia["ironia*<br/><small>cabeça binária — só tela</small>"]

    texto --> V["39 features"]
    emoji --> V
    tempo --> V
    emocao --> V
    lexico --> V
    estilo --> V
    incong --> V

    V --> F["Fusor<br/><small>LogisticRegression + StandardScaler</small>"]
    F -->|"score 0-100"| N["nota 0-10 → categoria NPS → indicadores agregados"]
    N --> API["FastAPI"]
    API --> D["dashboard Next.js"]
    ironia -->|"por mensagem, direto pra tela"| D

    style ironia stroke-dasharray: 4 4

```

* a cabeça de ironia é lida por mensagem e mostrada na tela, mas não entra nas 39 features do Fusor — ver a seção abaixo.

■ 4.1 A DECISÃO QUE EXPLICA O DESENHO

Um modelo canônico, muitos adaptadores. CSV, Totalk, transcrição de `.docx`/`.pdf` e webhook desembocam todos em `Conversa` — e o resto do sistema não sabe de onde o dado veio. Adicionar uma fonte é escrever um adaptador, não tocar em sinal, fusor ou indicador.

■ 4.2 AS SETE FAMÍLIAS QUE PONTUAM

FAMÍLIA	O QUE MEDE	POR QUE EXISTE
texto	P(insatisfeito/neutro/satisfeito) por mensagem	o sinal principal — o único fine-tunado para a tarefa
emoji	polaridade e posição relativa	emoji no fim da conversa não vale o mesmo que no começo
tempo	latências, escalção, abandono	espera longa degrada satisfação (literatura de live chat)
emoção	7 de Ekman + desprezo derivado	raiva e frustração distinguem <i>tipos</i> de insatisfação
léxico	SentiLex-PT02 com escopo de negação	âncora determinística, auditável, imune a retreino
estilo	caixa alta, alongamento, palavrão, censura	intensidade que o classificador de sentença perde
incongruência	elogio convivendo com situação negativa	é o que alcança a frase canônica do projeto

■ 4.3 ONDE RODA, E COM QUE MOTOR

Backend dos modelos	<code>FRAUS_BACKEND=torch</code> (padrão) ou <code>onnx</code> — os mesmos checkpoints, sem torch em runtime no segundo
Cabeça de ironia	<code>FRAUS_IRONIA_BACKEND</code> : <code>padrao</code> (BERTimbau), <code>laya</code> ou <code>laya-onnx</code> — troca só a leitura por mensagem; a ironia não entra no score
Motor	carregado por um provedor preguiçoso (<code>ProvedorDeMotor</code>): <code>frio</code> → <code>carregando</code> → <code>pronto</code> ou <code>erro</code> , sem fallback para dublê
Aquecimento	a carga começa em segundo plano no boot e a cada <code>GET /saude</code> ; <code>GET /saude/prontidao</code> só dá 200 com o motor pronto
Produção	dashboard e API em dois projetos Vercel, estado no Supabase Postgres; a API é serverless e esfria

O aquecimento reduz o cold start e não o elimina. Para eliminá-lo, a API precisa de um container sempre ligado (ver [Limitações](#)).

■ 4.4 A IRONIA CONTINUA CARREGADA, E NÃO PONTUA

A oitava cabeça existe, é obrigatória para a API subir e aparece na tela **por mensagem** — mas **saiu do vetor** em 04/09/2026.

Medida no próprio corpus de treino do fusor (B2W-Reviews01), ela marca **74% das resenhas satisfeitas** como irônicas contra **9% das insatisfeitas**: naquele corpus ela é um detector de sentimento **positivo**, não de ironia. O fusor aprendeu peso **+0,77** para `ironia_prob_media`, empurrando ironia para *satisfeito* — o inverso do que o nome promete.

Feature que mede outra coisa que não o nome dela é pior que feature ausente. Ela some do vetor sem que ninguém perceba a inversão, porque a acurácia global continua ótima.

Ver [Treinamento](#) para a medição completa.

■ 4.5 RÉGUA: O QUE PONTUOU UMA CONVERSA NÃO É FIXO NO TEMPO

Cada conversa é gravada com a **régua** que a pontuou: a assinatura do motor servindo (`Motor.regua`, `None` no motor dublê) mais a versão da curadoria (`lexico-versao`). Trocar de checkpoint, trocar de backend (torch/onnx) ou o analista ensinar um termo novo ao léxico muda a régua — e o banco passa a ter conversas antigas pontuadas por uma régua que não é mais a vigente.

O sistema não recalcula sozinho: ele **avisa**. `GET /indicadores` expõe `pontuadas com regua antiga`, contando contra a régua vigente lida a cada requisição (nunca guardada em memória, pelo mesmo motivo de `faixas vigentes` e `curadoria vigente` — sem cópia de estado envelhecendo). A tela mostra o aviso com o motivo (checkpoint novo, léxico curado, ou os dois), nunca um número solto sem explicação.

■ 4.6 REPONTUAR É UM TRABALHO, NÃO UMA REQUISIÇÃO

Zerar o aviso de régua misturada significa rodar os três BERTimbau de novo em cada conversa do banco — em milhares de atendimentos isso estoura qualquer timeout HTTP. `POST /conversas/repontuar` (`fraus/api/repontuacao.py`) só dispara uma thread em segundo plano; `GET /conversas/repontuar` lê o progresso (`total`, `feitas`, `estado`, `erro`) que a tela transforma em barra. Não é fila — é **um trabalho por vez**: uma segunda chamada enquanto a primeira roda recebe 409, porque duas repontuações em paralelo gravariam o mesmo banco com duas curadorias diferentes. O estado mora na memória do processo (reiniciar a API no meio perde o progresso, não o que já foi gravado — cada conversa salva já carrega sua própria `lexico-versao` e régua).

```
sequenceDiagram
    actor T as Tela
    participant API as FastAPI
    participant R as Repontuacao (thread)
    participant M as Motor (3× BERTimbau)
```

participant DB as banco

T->>API: POST /conversas/repontuar

API->>R: iniciar(ctx)

Note over R: le curadoria, faixas e régua
UMA vez — janela fechada

R-->>API: estado=rodando, total=N

API-->>T: 202 (ou 409 se já rodando)

R->>M: dispara thread em segundo plano

loop cada conversa do lote

M->>M: pontuar_conversa

M->>DB: salvar(score, lexico_versao, régua)

R->>R: feitas += 1

end

loop enquanto roda

T->>API: GET /conversas/repontuar

API-->>T: {total, feitas, estado}

Note over T: barra de progresso

end

R-->>API: estado=concluido (ou falhou)

T->>API: GET /conversas/repontuar

API-->>T: estado final

■ 4.7 ONDE CADA COISA MORA

CAMINHO	RESPONSABILIDADE
<code>fraus/modelos.py</code>	<code>Conversa</code> / <code>Mensagem</code> — o modelo canônico
<code>fraus/ingest/</code>	adaptadores de entrada, um por formato
<code>fraus/sinais/</code>	as oito famílias de sinal
<code>fraus/sinais/curadoria.py</code>	o que o analista ensinou ao léxico — vence o SentiLex
<code>fraus/fusor.py</code>	<code>NOMES FEATURES</code> (39) e o <code>Fusor</code>
<code>fraus/indicadores.py</code>	NPS, CSAT, contenção, falso containment, série
<code>fraus/evidencia.py</code>	evidência fraca — a cabeça tracejada
<code>fraus/deriva.py</code>	deriva de distribuição — a invariante 10 em runtime
<code>fraus/cortesia.py</code>	"ok, obrigado" sozinho é sem sinal, não promotor
<code>fraus/api/</code>	FastAPI: <code>criar app</code> (fábrica) e as rotas por domínio
<code>fraus/api/contexto.py</code>	faixas, curadoria e régua vigentes, lidas por requisição
<code>fraus/api/repontuacao.py</code>	repontuar em segundo plano, com progresso e 409 se já rodando
<code>fraus/api/vazao.py</code>	teto de <code>/auth/*</code> por IP e de <code>/ingestao</code> por fonte
<code>dashboard/</code>	Next.js — ver Design

5. Treinamento dos modelos do Fraus

 COMO LER ESTA PÁGINA (30/09/2026)

Ela é o **diário técnico do treino** e preserva a ordem em que as coisas aconteceram, inclusive números que já foram superados. O estado **atual** é: fusor de **39 features** (a ironia saiu do vetor em 04/09/2026 e `incongruencia_situacao_negativa` entrou: $40 - 2 + 1$), espera em `log1p` e acurácia de 0,947 no holdout do notebook 02 — ver o [cartão do modelo](#). As menções a 35, 38 e 40 features abaixo são história, não o contrato vigente.

#	NOTEBOOK	O QUE PRODUZ	GPU?	ESTADO
1	<code>notebooks/</code> <code>01_treino_bertimbau.ipynb</code>	<code>modelos/bertimbau-</code> <code>satisfacao/</code> — classificador de texto (3 classes), fine-tune do <code>neuralmind/bert-base-</code> <code>portuguese-cased</code> sobre o B2W-Reviews01	sim	pronto
2	<code>notebooks/</code> <code>02_treino_fusor.ipynb</code>	<code>modelos/fusor.joblib</code> e <code>modelos/importancias.json</code> — a regressao logistica que funde os sinais	nao	pronto
3	<code>notebooks/</code> <code>03_treino_emocao.ipynb</code>	<code>modelos/bertimbau-emocao/</code> e <code>modelos/</code> <code>metricas_emocao.json</code> — classificador das sete classes de emocao	sim	pronto
4	<code>notebooks/</code> <code>04_treino_ironia.ipynb</code>	<code>modelos/bertimbau-ironia/</code> e <code>modelos/metricas_ironia.json</code> — classificador binario de ironia	sim	pronto, mas o corpus precisa ser solicitado

O notebook 02 **depende do artefato do 01**: ele carrega o BERTimbau treinado para extrair o sinal de texto de cada conversa. A primeira célula do 02 falha com erro explícito se `modelos/bertimbau-satisfacao` não estiver no Drive.

Os notebooks 03 e 04 são **independentes entre si e do 01** — treinam cabeças separadas, em corpora próprios. O 02 é que passa a depender dos três, porque é ele que monta o vetor de features completo. Ordem de execução: 01, 03 e 04 em qualquer ordem, e o 02 por último.

O sinal lexico (`fraus/sinais/lexico.py`) **nao tem notebook**: o SentiLex-PT não é treinado, e recurso pronto, versionado em `fraus/dados/sentilex_pt02.csv`. Ver [Sinal lexico](#) no fim.

Os dois artefatos são obrigatórios para o app real subir. `fraus/api/main.py` carrega o classificador e o fusor no boot e propaga o erro sem fallback — servir predição sem modelo carregado é pior do que estar fora do ar. Com só um dos dois, o único servidor que sobe é `scripts/api_demo.py`, que usa motor duble e não serve para produção.

O treino roda inteiramente no Google Colab, fora deste repositório — não há teste automatizado local para essas etapas.

■ 5.1 COMO ABRIR NO COLAB

1. Va em colab.research.google.com.
2. `Arquivo > Fazer upload de notebook` e selecione o notebook (ou abra direto do GitHub se o repositório estiver acessível a partir da sua conta).

■ 5.2 NOTEBOOK 01 — CLASSIFICADOR DE TEXTO

5.2.1 Exigência de GPU

O notebook só funciona com acelerador de GPU habilitado. Antes de rodar qualquer célula:

`Ambiente de execução > Alterar tipo de ambiente de execução > GPU`

A primeira célula do notebook faz `assert torch.cuda.is_available()` e falha com uma mensagem explícita se o ambiente de execução estiver sem GPU.

5.2.2 Onde o checkpoint fica no Drive

O notebook monta o Google Drive (`drive.mount('/content/drive')`) e usa dois diretórios dentro dele, para sobreviver a uma queda de sessão do Colab — o mesmo padrão já usado no notebook de RVC do projeto Neuro-ai:

- `/content/drive/MyDrive/fraus/checkpoints` — checkpoints intermediários salvos pelo `Trainer` a cada época (`save_strategy="epoch"`, `save_total_limit=2`). Se a sessão cair, o próximo `Trainer.train()` pode retomar a partir do último checkpoint salvo aqui.
- `/content/drive/MyDrive/fraus/modelos/bertimbau-satisfacao` — destino final do modelo treinado, do tokenizador e do `metrics.json`, escritos na última célula do notebook.

5.2.3 Para onde copiar o artefato no repo local

Depois que o notebook terminar (célula de avaliação e export), baixe a pasta `fraus/modelos/bertimbau-satisfacao` do Google Drive e copie o conteúdo para:

modelos/bertimbau-satisfacao/

na raiz deste repositório. Os arquivos esperados são:

- `config.json`
- `model.safetensors`
- `tokenizer.json`
- `tokenizer_config.json`
- `special_tokens_map.json`
- `metricas.json` — `{"acuracia": float, "f1_macro": float, "classes": ["insatisfeito", "neutro", "satisfeito"]}`

`fraus/sinais/texto.py` carrega esse diretório para servir o classificador — e o notebook 02 também, para extrair o sinal de texto das conversas de treino do fusor.

Importante: `modelos/` está no `.gitignore` deste repositório. O artefato do modelo treinado (pesos, tokenizer, métricas) **nao vai para o git** — ele fica local, versionado apenas via o checkpoint no Drive.

5.2.4 Mapeamento de rotulo

O rótulo de satisfação vem do campo `overall_rating` (nota de 1 a 5) do B2W-Reviews01:

<code>OVERALL_RATING</code>	ROTULO	INDICE
1-2	insatisfeito	0
3	neutro	1
4-5	satisfeito	2

O corpus é balanceado por subamostragem da classe majoritária antes do split de treino/teste, porque as notas 4-5 dominam o corpus e enviesariam o classificador.

■ 5.3 NOTEBOOK 02 — FUSOR DOS TRES SINAIS

Roda DEPOIS do 01, sem GPU. O que ele faz, em sequencia:

1. monta o Drive e confere `modelos/bertimbau-satisfacao` (falha alto apontando o notebook 01 se faltar);
2. clona este repositorio e instala o pacote `fraus` — a extracao de features usa o MESMO codigo da API (`fraus.fusor.montar_features`), nunca uma reimplementacao;
3. carrega o B2W-Reviews01 e rotula por `recommend to a friend` (ver abaixo);
4. costura as frases em conversas sinteticas com `fraus.ingest.simulador.gerar_lote`, deterministico por semente, com latencia log-normal e emoji calibrados por rotulo;
5. extrai as features de cada conversa com o BERTimbau do notebook 01 carregado — hoje sao **39** (ver [Contrato de features](#));
6. treina o `Fusor` (`treinar(exemplos, rotulos)`);
7. avalia num conjunto de teste separado — conversas geradas com outra semente e a partir de frases disjuntas — imprimindo acuracia e F1-macro;
8. exporta `fusor.joblib` (via `Fusor.salvar`) e `importancias.json` (o retorno de `Fusor.importancias()`), que vira o grafico "qual sinal pesou mais" da apresentacao).

Os dois arquivos saem em `/content/drive/MyDrive/fraus/modelos/`. Baixe para `modelos/` na raiz do repositorio local, ao lado de `bertimbau-satisfacao/`.

5.3.1 Rotulo do fusor: `recommend to a friend`

Esta e a etapa em que o campo `recommend to a friend` do B2W-Reviews01 entra. A spec (secao 5.1) o declara como a **ancora do NPS inferido** — "sem ele, o NPS da dashboard seria metrica inventada" — e o alvo do fusor e justamente a categoria de NPS que a dashboard exhibe. O notebook 01 usou `overall_rating` porque treinava sentimento de texto; o 02 usa a recomendacao porque treina NPS.

CAMPO	VALOR	CLASSE	INDICE
<code>recommend to a friend</code>	<code>Yes</code>	satisfeito	2
<code>recommend to a friend</code>	<code>No</code>	insatisfeito	0
<code>overall_rating</code>	<code>3</code>	neutro	1

Por que a linha do neutro existe. `recommend to a friend` e binario, e as classes do projeto sao tres. Um fusor de duas classes zeraria o termo do neutro no score ($100 * (P(\text{satisfeito}) + 0.5 * P(\text{neutro}))$) e empurraria todo atendimento morno para uma das pontas, inflando o NPS nos dois sentidos. A alternativa seria treinar em duas classes e recortar o neutro por uma faixa de score — mas isso trocava um rotulo observado por um limiar arbitrado a mao depois do treino. Aqui o neutro e `overall_rating == 3`, um rotulo do corpus, e a nota 3 tem precedencia sobre a recomendacao para que a mesma resenha nao caia em duas classes. As duas pontas — as unicas que contam no NPS, ja que neutros nao entram na formula — seguem ancoradas em `recommend to a friend`.

As tres classes sao subamostradas ao tamanho da menor (a nota 3 e minoria no corpus), o mesmo balanceamento do notebook 01.

5.3.2 Limitacao metodologica a declarar no relatorio

O fusor e treinado sobre conversas SINTETICAS: nenhum corpus publico de resenha PT-BR tem timestamps de dialogo, entao a latencia, a escalacao e o abandono saem de distribuicoes calibradas pela literatura de live chat, nao de atendimento observado. O texto e real (B2W), o tempo nao e.

As metricas do notebook 02 continuam OTIMISTAS mesmo depois da correcao descrita abaixo: elas medem que o fusor aprendeu a combinar os sinais no dominio sintetico, nao que ele acertaria essa taxa em atendimento real.

5.3.3 O vazamento do primeiro fusor — 13/08/2026

O primeiro fusor treinado marcou **99,3% de acuracia e F1-macro 0,993**, e era inutil. Vale registrar porque o modo de falha e silencioso e pode voltar.

O sintoma. Carregado no repositorio local e alimentado com conversas escritas a mao, o fusor devolvia praticamente a mesma nota para tudo:

	LATENCIA 5S	30S	200S
texto otimo (BERTimbau: 0,96 satisfeito)	52,48	51,32	50,02
texto pessimo (BERTimbau: 0,98 insatisfeito)	49,94	49,92	49,84

Dois textos que o classificador de texto separa com mais de 95% de confianca saiam com **2,5 pontos** de diferenca, ambos colados em 50 e ambos classificados como neutro.

A causa. O `PERFIL POR ROTULO` do simulador amostrava latencia de faixas **disjuntas** — insatisfeito 60-400s, neutro 15-60s, satisfeito 3-15s. Sem sobreposicao nenhuma, a latencia *era* o rotulo: dava para acertar a classe olhando so o relógio, sem ler uma letra do texto. A regressao logistica fez exatamente isso. Os quatro maiores pesos aprendidos eram todos de tempo (`latencia_p90_s` 2,06 · `latencia_mediana_s` 2,03 · `latencia_primeira_resposta_s` 1,91 · `duracao_total_s` 1,52), contra 1,39 e 1,12 das features de texto. Fora da variedade que ele decorou, o modelo respondia "neutro" para tudo.

Os 99,3% nao mediam fusao. Mediam que o simulador vazava o gabarito.

Um segundo achado, do mesmo diagnostico. As features `emoji_score_medio`, `emoji_frac_positivos` e `emoji_frac_negativos` tinham coeficiente **0,0000**: o simulador nunca emitia emoji, entao elas eram constantes no treino e um dos tres sinais da arquitetura nao contribuia nada.

A correcao ([fraus/ingest/simulador.py], coberta por [tests/test_simulador.py]):

- a latencia passou a ser **log-normal** com caudas que se cruzam. As medianas continuam ordenadas, como manda a literatura de live chat, mas ~11% das conversas satisfeitas sao mais lentas que a mediana das insatisfeitas, e vice-versa. Existe atendimento rapido que termina mal;
- as falas do cliente passaram a levar **emoji do lexicon**, com cruzamento deliberado — cliente irritado manda 🙄 (+0,418), cliente satisfeito manda 😊 (-0,368) — para o emoji nao virar o proximo gabarito.

Como saber se voltou. Acuracia alta e sintoma que pede investigacao, nao veredito. Depois da correcao o notebook 02 marcou **0,96**, e desta vez o numero se sustentou: os pesos aprendidos mudaram de lugar.

FEATURE	FUSOR VAZADO	FUSOR CORRIGIDO
[emoji_score_medio]	0,0000 (morto)	1,6662 (1º)
[texto_prob_satisfeito_media]	1,3858	1,6026 (2º)
[texto_prob_insatisfeito_media]	1,1212	1,4722 (3º)
[latencia_p90_s]	2,0575 (1º)	0,0674
[latencia_mediana_s]	2,0329	0,1621
[latencia_primeira_resposta_s]	1,9068	0,0138

O mesmo teste de sanidade que expôs o vazamento — mesmo texto, so mudando o relógio — passou de 2,5 pontos de separacao para **99,6**: texto otimo pontua 99,98 e texto pessimo 0,35, com a latencia mexendo fracoas de ponto. O sinal de tempo virou o modificador fraco que ele deve ser.

O criterio, entao, nao e o numero sozinho. Diante de acuracia alta, olhe os PESOS: se as features que lideram forem as que carregam o conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais (tempo, contagem de turnos), procure o vazamento.

5.3.4 O fusor de 35 features — 24/08/2026

O contrato subiu de 16 para 35 features em 21/08 (emocao, lexico, ironia e estilo entraram no vetor) e o **artefato ficou tres dias para tras**. Nesse intervalo a API real nao subia: o [StandardScaler] de 16 rejeitava o vetor de 35 na carga. Falha alta e explicita, que e o comportamento que as invariantes 7 e 9 pedem — mas ainda assim tres dias de API parada por artefato desatualizado.

O notebook 02 rodou de novo e o artefato daquele momento passou a ter [n_features_in = 35], na ordem exata de [NOMES_FEATURES]. (Historico: o artefato vigente hoje tem 38 — ver as secoes de 03/09 e 04/09 abaixo.)

A acuracia caiu de 0,96 para 0,93, e a queda e o resultado saudavel. O fusor de 16 features media um problema mais facil. Dezenove features novas entraram, tres delas vindas de uma cabeca de ironia com

vazamento de corpus conhecido e nao consertado — um numero MENOR e o esperado. Pela regra desta secao, o que decide nao e o numero e sim onde os pesos foram parar:

FEATURE	FUSOR DE 16	FUSOR DE 35
<code>texto_prob_satisfeito_media</code>	1,6026 (2º)	1,6572 (1º)
<code>texto_prob_insatisfeito_media</code>	1,4722 (3º)	1,3867 (2º)
<code>emoji_score_medio</code>	1,6662 (1º)	1,3354 (3º)
<code>emoji_frac_positivos</code>	0,8430	0,8098
<code>emocao_surpresa_media</code>	—	0,6655
<code>emocao_desprezo_derivado</code>	—	0,4757
<code>latencia_mediana_s</code>	0,1621	0,1200
<code>latencia_p90_s</code>	0,0674	0,1419
<code>duracao_total_s</code>	0,4964	0,3397

O topo continua sendo **texto e emoji**, e as features de tempo continuam no rodape, onde devem estar. As de emocao entram no meio da tabela com peso real sem deslocar o conteudo — `desprezo_derivado`, que e a diade calculada e nao uma classe do modelo, puxa 0,48. Nao ha sinal de vazamento novo.

A divida que este fusor assume. Ele foi treinado com a cabeca de ironia vazando (ver a pendencia 1 do README: 6 em 10 frases sinceras saem marcadas como ironicas). `ironia_prob_media` e `ironia_prob_max` pesam 0,30 e 0,31 — nao e desprezivel. **Retreinar a ironia obriga a retreinar o fusor de novo**, e essa ordem esta registrada aqui para nao se perder.

Teste de sanidade ponta a ponta, com os tres BERTimbau carregados, sobre conversas do simulador: rotulo insatisfeito pontua ~0, neutro ~73–75, satisfeito ~99, e as tres categorias de NPS saem certas em 9 de 9. **Isso nao e evidencia de qualidade** — sao conversas do proprio gerador sintetico, o dominio em que o fusor foi treinado. O numero honesto continua sendo 0,93 no conjunto de teste separado, e a ressalva do topo desta secao continua valendo: as metricas do notebook 02 medem fusao no dominio sintetico, nao acerto em atendimento real.

5.3.5 Consequencia no NPS: a classe neutra cai em detrator — HISTORICO (fusor de 16 features, ate 24/08/2026)

Esta secao mede o fusor de 16 features, NAO o vigente. Ela ficou aqui sem data enquanto o artefato foi trocado, e passou a ler como se descrevesse o fusor atual — que e o defeito mais caro deste projeto. Fica registrada porque o raciocinio sobre o score continua valendo, e porque a medicao dela e o contraponto do que o fusor de 35 features entrega.

A consequencia DEIXOU de valer com o fusor de 35 features. Base: o teste de sanidade ponta a ponta da secao anterior, com os tres BERTimbau carregados — rotulo neutro pontua **~73–75**, cai na faixa 7–8 e as tres categorias de NPS saem certas em 9 de 9. Ou seja, o neutro nao cai mais em detrator. Ressalva do mesmo

tamanho: aquele teste corre sobre conversas do proprio simulador, o dominio em que o fusor foi treinado, entao ele desmente a medicao abaixo no dominio sintetico e nao promete nada sobre atendimento real.

Medido em 90 conversas do simulador, 30 por classe, o fusor corrigido separa bem — medianas **0,11 / 50,99 / 99,03** por rotulo verdadeiro. Mas a categoria de NPS derivada sai **67% detrator · 29% promotor · 4% neutro**, com **NPS -38** num lote equilibrado por construcao.

Nao e falha do treino. O score e $100 * (P(\text{satisfeito}) + 0.5 * P(\text{neutro}))$, entao conversa classificada com certeza como neutra pontua 50, vira nota 5 e cai na faixa 0-6. A faixa neutra do NPS (7-8) so seria alcancada com $P(\text{satisfeito})$ entre 0,4 e 0,8 — um empate entre classes, nao uma neutralidade confiante. As tres classes do modelo e as tres categorias do NPS foram decididas em lugares separados e nunca se encontraram.

Decisao de projeto: manter e declarar. A alternativa avaliada era subir o peso do neutro de 0,5 para 0,75 (o centro da faixa passiva do NPS), o que faria neutro puro pontuar 75, virar nota 8 e cair em neutro — alinhando as tres classes as tres categorias. Ficou registrada aqui e na docstring de `Fusor.pontuar` para nao ser "corrigida" por engano.

5.3.6 Retreino do fusor apos as features de incongruencia (03/09/2026)

O contrato subiu de 35 para 40 features: a familia `incongruencia *` (`incongruencia polaridade`, `incongruencia emoji texto`, `incongruencia marcador contraste`, `incongruencia hiperbole`, `incongruencia aspas ironicas`) entrou em `NOMES FEATURES`, anexada ao FIM da lista para preservar a ordem das 35 antigas. O fusor salvo em `modelos/` foi treinado com 35 e `vetorizar` agora produz 40 — ele NAO serve mais. `Fusor.carregar` (`joblib.load` puro, sem validacao de forma) carrega esse artefato sem erro nenhum -- a API real SOBE com ele. O erro de dimensao no `StandardScaler` so aparece na primeira pontuacao de verdade (`/ingestao` ou `/conversas/importar`), como HTTP 500, nao na carga do servidor. Isso e risco de demonstracao ao vivo, nao uma trava de subida -- ver a tabela do Fusor no README para o detalhe medido. A intencao da invariante 7 (falha alta e explicita) so se cumpre na metade "explicita"; a metade "na carga" fica para o retreino resolver o descompasso, nao para uma validacao que este codigo ainda nao tem.

O corpus sintetico tambem mudou, e isso importa mais do que a contagem de features.

`FRASES POR ROTULO`, em `fraus/ingest/simulador.py`, ganhou frases novas em duas levadas para curar dois vazamentos de rotulo — o mesmo tipo de falha silenciosa que ja custou o primeiro fusor (ver acima), so que desta vez em features que ainda nao estavam no vetor:

- **Intensificadores.** Antes, `incongruencia hiperbole` era nao-zero na maior parte das conversas do rotulo satisfeito e ZERO nos rotulos insatisfeito e neutro — um previsor unilateral quase perfeito de "satisfeito". A literatura inclui hiperbole na deteccao de ironia justamente porque elogio hiperbolico e a forma classica da ironia, entao o fusor teria aprendido a feature ao contrario do que ela significa. Depois da correcao, medido com

`gerar_lote(FRASES POR ROTULO, 180, semente=7)` — o mesmo lote e semente da guarda

`test_nenhuma_feature_e_previsor_unilateral` em `tests/test_simulador.py`: `insatisfeito 34/60 | neutro 0/60 | satisfeito 39/60`.

- **Negacao.** `lexico_frac_negados` era nao-zero so no rotulo insatisfeito e ZERO nos outros dois — previsor unilateral de "insatisfeito", pre-existente desde antes da familia `incongruencia *` e nunca notado por falta de guarda na familia `lexico *`. Depois, medido com o mesmo lote e semente acima: `insatisfeito 15/60 | neutro 17/60 | satisfeito 21/60`.

Consequencia pratica: o notebook 02 precisa REGERAR o corpus sintetico, nao reusar um lote cacheado de uma execucao anterior. O notebook 02 ja gera o lote do zero a cada execucao — `gerar_lote` roda

na propria celula, com semente fixa, e nao le nenhum artefato salvo de rodada passada — entao rodar o notebook de novo, sem alterar nada, ja traz o corpus corrigido. O risco e so para quem tiver guardado um `.csv`/`.pkl` de conversas de uma sessao antiga do Colab e tentar reaproveitar: um lote assim traz os dois vazamentos de volta.

O notebook ja importa o contrato do pacote, nao repete a lista. A celula marcada com o comentario

```
# 5. Os SETE sinais de cada conversa, pelo MESMO
```

`codigo que a API usa` (indice 8 do `.ipynb`, contando a partir de 0) faz `from fraus.fusor import NOMES_FEATURES, montar_features` e chama `montar_features` com os tres classificadores — as cinco features novas entram sozinhas, sem editar o notebook. Isso ja era assim antes desta rodada (o vazamento do ramo obsoleto registrado na celula 2 foi outra causa, a de importar de um ramo desatualizado do repositorio, nao de reimplementar a lista) — nao ha lista duplicada para divergir aqui.

Passos, no Colab:

1. Conferir que a celula do clone do repositorio (`RAMO`) aponta para `main` com a Task 5 mergeada, e rodar o notebook do inicio — sem tentar reaproveitar nenhum lote de conversas salvo de uma execucao anterior.
2. Antes de subir ao Colab (ou depois, localmente), rodar as guardas de vazamento: `uv run pytest tests/test_simulador.py -k unilateral`. Elas cobrem as familias `lexico *`, `emoji *`, `estilo *` e `incongruencia *` contra previsor unilateral e precisam estar verdes antes do treino valer a pena.
3. Conferir a acuracia contra a do fusor de 35 features (0,93). **Acuracia alta demais e sintoma, nao vitoria** — se saltar muito acima do valor anterior, suspeite de vazamento antes de comemorar, releia a secao "O vazamento do primeiro fusor" acima e confira os PESOS, nao so o numero: se as features que lideram forem as de conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais, procure o vazamento.
4. Conferir os coeficientes das cinco features novas em `Fusor.importancias()` / `Fusor.eixo_global()`.

CORRECAO DE 04/09/2026, e a licao vale mais que o numero: este passo dizia para esperar peso proximo de zero em `incongruencia marcador contraste` e `incongruencia aspas ironicas`, porque as duas sao constantes em zero no corpus do SIMULADOR. A previsao estava errada, e o erro foi confundir dois corpora diferentes. O simulador fornece a ESTRUTURA temporal (latencia, turnos, escalacao) a partir de seis frases por rotulo; o TEXTO do treino e o B2W-Reviews01, resenha real de e-commerce, onde marcador de contraste e aspas ironicas ocorrem normalmente. Medir uma feature de texto no gerador de estrutura e olhar para o corpus errado.

Os pesos reais do primeiro retreino de 40 features, no eixo satisfeito-menos-insatisfeito:

FEATURE	PESO
<code>incongruencia emoji texto</code>	−0,95
<code>incongruencia hiperbole</code>	−0,65
<code>incongruencia marcador contraste</code>	−0,38
<code>incongruencia polaridade</code>	−0,37
<code>incongruencia aspas ironicas</code>	−0,15

As cinco saíram NEGATIVAS, que é a direção que a literatura prevê: mais incongruência empurra a nota para insatisfeito. A hipérbole em $-0,65$ e a evidência de que o cruzamento do corpus (seção acima) funcionou — sem ele ela teria aprendido o sinal contrário.

O que continua valendo da versão anterior deste passo: as guardas de vazamento medem o SIMULADOR e são sobre a estrutura, não sobre o texto do B2W. `incongruencia_hiperbole` melhorou mas não chegou aos três: depois da correção ela dispara em DOIS dos três rótulos — `insatisfeito 34/60` e `satisfeito 39/60` (números medidos acima, na seção do retreino) —, e `neutro` continua em `0/60`. Isso é limitação declarada, não vazamento remanescente: fala neutra raramente intensifica termo polar com hipérbole, e a decisão foi não forçar uma frase artificial no gerador só para a classe deixar de ficar muda. A guarda `test_nenhuma_feature_e_previsor_unilateral` (`tests/test_simulador.py`) tolera isso por construção — ela só falha quando a feature dispara em EXATAMENTE UM rótulo, então 2-de-3 passa verde. Isso é posição aceita sobre o corpus sintético atual, não descuido: fica registrado aqui para o próximo leitor não achar que a guarda promete "sinal nos três rótulos" quando ela só promete "não previsor unilateral de UM rótulo só". A guarda não foi alterada. 5. Copiar `fusor.joblib` e `importancias.json` para `modelos/` na raiz do repositório local (guardando o artefato anterior como `.bak-<N>`, onde N é o número de features dele) e então, nesta ordem: `uv run python scripts/conferir_fusor.py` e `uv run pytest -q` inteiro. O laudo faz automaticamente os passos 3 e 4 desta lista — ver "Conferência do artefato" abaixo.

5.3.7 A ironia sai do vetor (04/09/2026)

Medição sobre o próprio corpus de treino do fusor (B2W-Reviews01, amostra equilibrada de 500 resenhas por rótulo, semente 20260904), rodando só a cabeça de ironia:

resenha NEGATIVA (nao): media $P(\text{ironia}) = 0,1012$ mediana $0,0013$ $\text{frac}(P > 0,5) = 0,092$ resenha POSITIVA (sim): media $P(\text{ironia}) = 0,7146$ mediana $0,9687$ $\text{frac}(P > 0,5) = 0,740$ diferença (sim-nao) = $+0,6134$

A cabeça, treinada no IDPT 2021 (tweet e notícia), aplicada a resenha de e-commerce funciona na prática como **detector de sentimento positivo**: marca 74% das resenhas satisfeitas como ironicas contra 9% das insatisfeitas. O fusor de 40 features aprendeu **+0,77** de peso para `ironia_prob_media` no eixo satisfeito-menos-insatisfeito — mais ironia empurrando para SATISFEITO, o inverso do que o nome da feature promete. O fusor de 35 já tinha o mesmo sinal ($+0,48$), então não é regressão desta rodada: é o mesmo vazamento medido de novo, agora com número.

Consequência observável: a frase canônica de ironia deste projeto — "que atendimento maravilhoso, só esperei 3 horas" — pontuava **99,98 / nota 10 / promotor** no fusor de 40, porque as duas features de ironia empurravam para cima em vez de para baixo. (A cabeça isolada acerta essa frase: 0,998 nela, 0,002 num elogio sincero. Quem lia ao contrário era o fusor, e só por causa do corpus.)

Decisão: `ironia_prob_media` e `ironia_prob_max` saem de `NOMES_FEATURES`. Feature que mede outra coisa que não o nome dela e pior que feature ausente — ela não adiciona sinal, duplica `texto_prob_satisfeito *` com ruído, e inverte o caso que o projeto usa como exemplo de manual. A cabeça de ironia CONTINUA carregada e obrigatória (invariante 7) e continua sendo lida por mensagem e exibida na dashboard, onde é honesta e já vem com a ressalva de confiabilidade que a tela mostra — o que muda é que ela para de pontuar. O retreino com 38 features foi executado no mesmo dia; ver a seção seguinte.

Nota sobre a janela entre a mudança do contrato e o artefato novo. Enquanto `modelos/fusor.joblib` era o artefato de 40 e o contrato já estava em 38, a API **nao subia** — `Fusor.carregar` passou a comparar `n features in` com `len(NOMES_FEATURES)` e levantar `FusorIncompativelError` (PR #29, 04/09/2026). Antes dessa validacao, o mesmo desencontro deixava a API subir normalmente e estourar HTTP 500 so na primeira pontuacao real — falha tardia, no pior momento possivel. A validacao troca isso por falha alta e explicita na subida, que e o que a invariante 7 pede.

5.3.8 O fusor de 38 features — 04/09/2026

Notebook 02 rodado de novo com a ironia fora do vetor. **Acuracia 0,9433** (f1_macro 0,9436), contra 0,9467 do fusor de 40. A queda de ~0,003 e o resultado saudavel e esperado: as duas features removidas *estavam* ajudando o classificador — so que como detectores de sentimento positivo duplicando `texto_prob_satisfeito *`, nao como medida de ironia. Perder um pouco de acuracia trocando uma feature que mente por nenhuma feature e o negocio certo.

Conferido com `uv run python scripts/conferir_fusor.py` (ver secao "Conferencia do artefato" abaixo): contrato 38/38, `classes = [0, 1, 2]` (invariante 8), nenhuma feature com peso perto de zero, e as cinco de incongruencia mantiveram a direcao negativa que a literatura preve:

FEATURE	FUSOR DE 40	FUSOR DE 38
<code>incongruencia_emoji_texto</code>	-0,95	-1,03
<code>incongruencia_hiperbole</code>	-0,65	-0,64
<code>incongruencia_polaridade</code>	-0,37	-0,37
<code>incongruencia_marcaador_contraste</code>	-0,38	-0,27
<code>incongruencia_aspas_ironicas</code>	-0,15	-0,19

Tres pesos com sinal aparentemente invertido, e por que NAO sao vazamento. O laudo sinaliza `texto_prob_satisfeito_ultima` (-0,50), `emoji_frac_positivos` (-0,18) e `emoji_frac_negativos` (+0,60) como suspeitos, porque o nome promete o sinal contrario. Conferidos contra os artefatos de 40 e de 35, os tres ja estavam assim nos dois — nao e efeito da remocao da ironia. A explicacao e **colinearidade**: `texto_prob_satisfeito_media` pesa +2,81 e `emoji_score_medio` +1,56, e features fortemente correlacionadas com elas recebem coeficiente negativo de correcao na regressao logistica. E o comportamento normal do modelo com features redundantes, nao inversao de semantica como a de `ironia_prob_media`. A diferenca entre os dois casos e verificavel: a ironia foi medida diretamente no corpus e provou medir outra coisa; estas tres continuam medindo o que o nome diz, so nao carregam informacao independente das dominantes. Fica registrado aqui porque o laudo vai sinalizar as tres em todo retreino futuro, e o proximo leitor precisa saber que ja foram investigadas.

O que este retreino NAO resolveu. A frase canonica do projeto continua pontuando alto:

"que atendimento maravilhoso, so esperei 3 horas"	score 99,95	nota 10
"otimo, mais uma vez ninguem resolveu nada"	score 99,74	nota 10
"resolveu rapido, muito obrigado! adorei"	score 97,82	nota 10

As cinco features de incongruência dão **0,0 nas duas frases ironicas** e disparam na satisfeita de verdade ([polaridade] 0,5, [hiperbole] 0,5) — ou seja, hoje elas pegam entusiasmo genuíno e não pegam ironia. Não é regressão: é o limite estrutural das cinco, todas função de [anotar texto]/emoji, e "so esperei 3 horas" não tem nenhuma palavra polar no SentiLex — e negativo por pragmática. O diagnóstico completo e a rota proposta ([incongruência_situacao_negativa], que levaria o vetor a 39 e exige mais um retreino) estão em [docs/superpowers/specs/2026-09-04-incongruência-implícita-design.md].

Corrigindo a previsão registrada na seção anterior: esperava-se que a frase deixasse de ser empurrada para cima com a saída da ironia. Não foi o que aconteceu — o [texto_prob_satisfeito_media] da própria cabeça de texto é 0,858 nela, e domina o eixo sozinho. A remoção consertou o vetor; não tocou nesta frase.

5.3.9 O contrato sobe para 39: incongruência implícita (04/09/2026)

[incongruência_situacao_negativa] entrou em [NOMES_FEATURES] -- elogio convivendo com situação negativa de atendimento na mesma fala. É a única das seis que alcança a frase canônica do projeto, porque não depende de um segundo termo polar no lexicon.

O retreino do notebook 02 é OBRIGATORIO antes de a API voltar a subir. O artefato de 38 não carrega mais: [Fusor.carregar] valida a dimensão e levanta [FusorIncompativeError]. Isso é o comportamento desejado, não um bug.

Medido ANTES de aceitar, no B2W-Reviews01 (2000 resenhas por rótulo, semente 20260904) -- que é o corpus que dá o TEXTO do treino:

insatisfeito	82/2000 = 4,10%
neutro	53/2000 = 2,65%
satisfeito	35/2000 = 1,75%

Dispara nos três rótulos, gradiente suave, razão 2,3:1 entre os extremos. Compare com a cabeça de ironia que saiu do vetor no mesmo dia (74% contra 9%, razão 8:1): aquilo era disjunção prática, isto é sinal.

Uma armadilha de método que quase repeti. Medir esta feature no corpus do SIMULADOR dava zero nos três rótulos, e eu quase concluí que ela nasceria com peso zero. Errado, e pelo mesmo motivo registrado na correção de 04/09 mais acima: [FRASES POR RÓTULO] **nao treina o fusor**. O notebook 02 usa 400 frases do B2W por classe e pega do simulador só a ESTRUTURA da conversa. O simulador importa para a guarda [test_nenhuma_feature_e_previsor_unilateral], não para o peso aprendido. As frases dos três rótulos foram cruzadas mesmo assim -- sem isso a feature ficava constante em zero lá e a guarda passaria por vacuidade, que é o modo de falha silencioso dela.

5.3.10 Conferência do artefato

uv run python scripts/conferir_fusor.py	# laudo completo
uv run python scripts/conferir_fusor.py --sem-sonda	# pula as frases-sonda

Laudo do artefato vigente em `modelos/`, em seis itens: contrato, ordem das classes, sinal dos pesos por familia, pesos perto de zero, tres frases-sonda e as metricas do treino. Instrumento, nao botao — nao escreve nada, no modelo de `scripts/medir_faixas.py`.

Ele existe porque **acuracia nao denuncia** os dois defeitos que esta semana custaram dois retreinos: o peso invertido de `ironia_prob_media` e o vazamento de `incongruencia_hiperbole`. Os dois so apareceram porque alguem olhou o sinal dos coeficientes e a distribuicao por rotulo — exatamente o que uma pessoa cansada depois de um treino longo pula. Por isso a acuracia nunca e impressa sozinha no laudo: vem sempre com o lembrete da invariante 10, e com alerta explicito acima de 0,97.

■ 5.4 CONTRATO DE FEATURES

`NOMES_FEATURES`, em `fraus/fusor.py`, e a lista canonica. `vetorizar` levanta `KeyError` se faltar chave — nunca zero silencioso (invariante 9).

Hoje sao **39 features**, de SETE familias no vetor (OITO sinais existem no sistema — a ironia continua existindo e sendo lida por mensagem, so nao entra mais aqui, ver a secao acima). O contrato subiu de 16 para 35 em 21/08/2026, quando os notebooks 03 e 04 passaram a existir e a condicao que justificava a espera acabou; de 35 para 40 em 03/09/2026, quando a familia `incongruencia *` entrou (ver [Retreino do fusor apos as features de incongruencia](#) acima); e caiu de 40 para 38 em 04/09/2026, quando `ironia_prob_media` e `ironia_prob_max` sairam (ver [A ironia sai do vetor](#) acima).

SINAL	MODULO	FEATURES	NO VETOR?
texto	<code>fraus/sinais/texto.py</code>	4	sim
emoji	<code>fraus/sinais/emoji.py</code>	5	sim
tempo	<code>fraus/sinais/tempo.py</code>	7	sim
emocao	<code>fraus/sinais/emocao.py</code>	8	sim
lexico	<code>fraus/sinais/lexico.py</code>	3	sim
ironia	<code>fraus/sinais/ironia.py</code>	2	nao (lida por mensagem, nao pontua)
estilo	<code>fraus/sinais/estilo.py</code>	6	sim
incongruencia	<code>fraus/sinais/incongruencia.py</code>	5	sim

Consequencia: `montar_features` exige DOIS classificadores (texto e emocao); `Motor` continua exigindo os TRES (a ironia entra na atribuicao por mensagem, nao no vetor), e a API nao sobe sem os tres artefatos em `modelos/`. E o comportamento correto da invariante 7.

■ 5.5 NOTEBOOK 03 — CLASSIFICADOR DE EMOCAO

Requisito de banca: distinguir as **sete emocoes humanas**. A cabeca treinada tem sete classes, e a setima emocao e derivada — ver abaixo.

A ordem das classes **nao e redigitada no notebook**: ele importa `NOMES_EMOCOES` de `fraus.sinais.emocao`, a mesma lista que o sinal usa para ler as probabilidades. Redigitar seria o jeito mais facil de treinar `raiva` no indice 2 e le-la no indice 3.

5.5.1 Ordem canonica das classes

`fraus/sinais/emocao.py` fixa a ordem, e ela vale no notebook, no sinal e no fusor. Inverter nao gera erro: atribui a emocao errada em silencio, a mesma armadilha da invariante 8.

INDICE	CLASSE
0	alegria
1	tristeza
2	raiva
3	medo
4	nojo
5	surpresa
6	neutro

5.5.2 Corpus: `go_emotions_ptbr`

Traducao PT-BR do GoEmotions (Demszky et al., 2020 — 58k comentarios do Reddit, 27 emocoes + neutro), com 43.410 exemplos de treino, 5.426 de validacao e 5.427 de teste. Licenca Apache 2.0. As 27 categorias sao reduzidas as seis de Ekman pelo **mapeamento oficial que o proprio GoEmotions publica** — nao por agrupamento inventado aqui.

O arquivo vem no formato CRU do GoEmotions — **uma linha por anotador**, nao por exemplo. Um rotulo conta quando **pelo menos 2 anotadores** o marcaram (a maioria dos exemplos tem 3 anotadores, entao 2 e maioria simples), e exemplos que caem em mais de uma classe de Ekman sao descartados: a tarefa e multiclasse de rotulo unico, e forcar vencedor entre duas emocoes empatadas inventaria rotulo.

O limiar mais alto rende MAIS exemplos, o que so parece contra-intuitivo ate ver o motivo — com 1 anotador quase todo exemplo recebe varios rotulos Ekman e cai fora do filtro de rotulo unico:

LIMIAR	EXEMPLOS COM ROTULO UNICO
1 anotador	17.367
2 anotadores	46.014

5.5.3 O desequilíbrio, e por que NAO subamostrar

Distribuicao medida com limiar 2:

CLASSE	EXEMPLOS
alegria	19.284
neutro	13.603
raiva	5.155
surpresa	4.339
tristeza	2.601
medo	588
nojo	444

Sao **43:1** entre a maior e a menor. Os notebooks 01 e 02 balanceiam por subamostragem, mas aqui isso seria destrutivo: cortar tudo ao tamanho de `nojo` deixaria **3.108 exemplos** para sete classes, jogando fora 93% do corpus.

Escolha: pesos de classe na perda, pela formula $n / (k * n_c)$ — a mesma do `class_weight='balanced'` do scikit-learn. Os 46 mil exemplos ficam. Por isso a metrica de selecao do melhor checkpoint e o **F1-macro, nunca a acuracia**: com 43:1, um modelo que chutasse `alegria` sempre marcaria 42% sem ter aprendido nada.

5.5.4 O teto do desprezo — a declarar no relatorio

`nojo` e a classe mais rara do corpus, e o **desprezo e derivado de raiva x nojo**. A qualidade do desprezo fica limitada de cima pela qualidade da pior classe do modelo: se o F1 de `nojo` sair baixo, o desprezo herda isso. O notebook imprime esse teto explicitamente e grava o F1 por classe em `metricas_emocao.json`, para o numero aparecer no relatorio em vez de ser descoberto pela banca.

Limitacao a declarar: a traducao foi feita por maquina (Google Tradutor), sem revisao humana. Por isso a avaliacao reporta tambem o **XED-pt** (7.220 linhas, CC-BY, Plutchik 8 + neutro) como conjunto de teste independente, que nao passou por traducao automatica. O mapeamento Plutchik -> Ekman descarta *anticipation* e *trust*; o resto casa 1:1.

Duas ressalvas sobre o XED-pt: ele e **portugues europeu** ("pa", "artola"), enquanto o treino e PT-BR — diferenca de variedade, nao so de dominio; e o numero dele vai sair **mais baixo** que o interno, o que e esperado e e justamente o ponto, porque e ele que responde "mas nao e so traducao automatica?".

5.5.5 Por que o desprezo NAO e treinado

A setima emocao e o **desprezo**, e nenhum corpus rotulado em portugues o anota. Havia um corpus multilingue sintetico com `contempt` entre os rotulos, e ele foi **recusado de proposito**: treinar seis classes com dado humano e so o desprezo com dado de outra procedencia ensinaria o modelo a reconhecer o *estilo do texto sintetico* em vez do desprezo. E o mesmo vazamento da latencia disjunta, com outra roupa.

O desprezo é derivado da **diade primaria raiva + nojo**, que é como Plutchik (1980) o define — referência citável, não rótulo inventado. A composição usa média **geométrica**, não aritmética: desprezo exige as duas emoções juntas, e a aritmética devolveria 0,5 para raiva pura sem nojo nenhum, o que é raiva e não desprezo.

■ 5.6 NOTEBOOK 04 — CLASSIFICADOR DE IRONIA

Classificação binária: 0 não-irônico, 1 irônico.

Corpus: IDPT 2021, a tarefa de *Irony Detection in Portuguese* do IberLEF — 15,2k tweets e 18,4k notícias anotados.

08/10/2026: o treino público do IDPT entrega o rótulo pela fonte. Os tweets não-irônicos são 100% `#economia` de portais de notícia; as notícias são rotuladas pelo site. Retreinado com a divisão certa, o BERTimbau deu F1 0,994 no teste e $P(\text{irônico}) = 1,0$ em "ok, obrigado" -- e não foi exportado. Analise, plano e o treino do Laya só em ironia: [ironia: corpus e Laya](#).

5.6.1 Fontes públicas do treino e teste oficial fechado

O **teste oficial**, anotado para a competição, continua protegido e deve ser solicitado aos organizadores em <https://sites.google.com/inf.ufpel.edu.br/idpt2021/>. Ele nunca entra no treino. As fontes públicas usadas para formar o treino do IDPT, porém, estão abertas: os tweets em <https://github.com/fabio-ricardo/deteccao-ironia> e as notícias em <https://github.com/schuberty/PLNCrawler>. O notebook baixa essas fontes no runtime por commits imutáveis e valida cada Git blob antes de usar.

O notebook aceita **três origens** para o corpus, escolhidas na variável `ORIGEM` da célula 3. Todas alimentam a mesma pasta e a mesma detecção de esquema, então trocar de fonte não muda o resto do notebook:

<code>ORIGEM</code>	QUANDO USAR
<code>idpt_publico</code> (padrão)	baixa as fontes públicas originais, remove hashtags que vazam o rótulo, duplicatas e conflitos
<code>sintetico</code>	gera o corpus com <code>fraus.ingest.gerador.ironia</code> — fica como abstração, não como fonte principal
<code>drive</code>	os arquivos já estão em <code>DIR_CORPUS</code> — foi assim que o IDPT entrou, se liberado
<code>kaggle</code>	baixa da conta Kaggle; exige <code>KAGGLE_USERNAME</code> / <code>KAGGLE_KEY</code> nos Secrets do Colab
<code>upload</code>	seletor de arquivos do navegador, para corpus que chegou por e-mail

5.6.2 A divisão por autor — conserto de 02/10/2026

A regra da divisão mora em `fraus/divisao.py` e é a mesma nos notebooks 04 e 07: um grupo por autor, e **cada linha sem autor e um grupo próprio**.

Ate 02/10/2026 a regra estava escrita nos notebooks com `autores.astype(str)`, que transforma o campo vazio no autor "nan". As 18.373 noticias nao tem autor: viraram uma pessoa so e caíram inteiras no teste.

	ANTES	DEPOIS
Treino	12.271 linhas, 86% ironicas, so tweets	28.365 linhas, 59% ironicas
Teste	20.872 linhas (63% do corpus), 43% ironicas	4.778 linhas (14%), 59% ironicas

Os numeros de "depois" foram medidos aplicando a regra nova ao corpus real, fora do Colab; os do treino saem quando o notebook rodar.

Consequencias:

- A cabeça de ironia treinada antes desta data **nunca viu uma noticia** no treino e foi avaliada num teste que era 88% noticias. O F1-macro de 0,511 que o notebook 07 recalculou para ela nesse teste mede mudanca de dominio, nao a tarefa.
- `conferir divisao` agora recusa, com erro, divisao cujo teste foge do tamanho pedido ou cuja proporcao de classe difere entre os dois lados.
- O notebook 04 grava em `metricas ironia.json` a impressao (SHA-256) dos textos do teste. O notebook 07 para antes do treino se o teste dele nao tiver a mesma impressao: sem isso, frases do treino do BERTimbau cairiam no teste da comparacao.
- Nas noticias a fonte entrega o rotulo (Estadao e sempre nao-ironico; os dois sites de satira, sempre ironicos). Com as noticias dos dois lados, o numero agregado tende a subir por reconhecer o jornal. Os dois notebooks imprimem o resultado **separado por meio**, e o que se le como desempenho em ironia e o dos tweets e o da regua de dominio. Numero alto no agregado e sintoma (invariante 10), nao vitoria.

5.6.3 Candidato Laya

O Laya multilíngue pode substituir apenas a cabeça de ironia, sem alterar satisfação, emoção ou o fusor. Ele é opt-in porque baixa aproximadamente 647 MB de pesos. O backend `laya` exige torch; o `laya-onnx` (30/09/2026) roda a mesma cabeça pelo ONNX Runtime, **sem PyTorch**, e é o que a API serverless usa quando o ZIP da Laya está configurado (ver `docs/deploy-vercel.md`, interno):

```
uv sync --extra laya
FRAUS_IRONIA_BACKEND=laya uv run uvicorn fraus.api.main:app
```

O checkpoint fica preso à revisão declarada em `FRAUS_LAYA_REVISAO`. Antes de promovê-lo, execute a mesma régua de transferência de domínio:

```
uv run python scripts/validar_candidato_ironia.py --backend laya
```

A aba **Modelo → Ironia · Laya** chama somente essa cabeça e não persiste nem altera notas. O tipo de pergunta usado é `choice` com chaves neutras, não `noul`, por causa do viés de rótulo documentado pelos próprios autores.

FINE-TUNING DO LAYA — NOTEBOOK 07 (02/10/2026, AINDA NÃO EXECUTADO)

O candidato acima responde a frio. `notebooks/07_treino_laya.ipynb` treina o mesmo checkpoint nas duas perguntas do Fraus — emoção (`PERGUNTA EMOCAO`), em `fraus/treino_laya.py` e ironia (`PERGUNTA IRONIA`) — com o script oficial de GPU única do Laya, e o compara com os BERTimbau dos notebooks 03 e 04 nos mesmos exemplos de teste: McNemar exato e IC por bootstrap da diferença de F1-macro (`fraus/comparacao_modelos.py`), mais o XED-pt e a régua de domínio.

Ele não promete 100% de acerto, e número perto disso seria sintoma (invariante 10). No fim, poda o vocabulário de 256 mil tokens para o que o português usa, exporta para ONNX e só adota INT8 se no máximo 1% das decisões mudar.

A última célula grava `comparacao_modelos.json` (acurácia, F1-macro, matriz de confusão, VP/FP/FN/VN por classe, McNemar, IC e latência em CPU) e o mesmo laudo em Markdown. Copiado para `modelos/`, ele aparece na aba **Modelo → Comparação** da dashboard (`GET /modelo/comparacao`), que também lê uma fala com as cabeças carregadas no servidor, lado a lado. Cada leitura do Laya diz se o artefato do servidor é o treinado pelo notebook 07 ou o checkpoint base ("Laya sem treino"): em produção é o base, que lê ironia e, a frio, emoção.

Para o laudo chegar à produção ele é versionado com o código, em `fraus/dados/comparacao_modelos.json`: `modelos/` fica fora do git e, na Vercel, é trocada inteira pelo ZIP dos pesos. A rota serve o laudo de `modelos/` quando ele existe e, na falta, o versionado. O campo opcional `ressalvas` (lista de textos) registra o que se soube sobre a medição depois de feita; ele aparece no topo da aba e do relatório. Os números do laudo nunca são editados à mão — laudo novo é notebook rodado de novo.

Rode primeiro com `MODO_FUMACA = True`. O que foi pesquisado, o que foi medido localmente e o que ainda falta está na nota interna `docs/notas/2026-10-02-treino-laya.md`.

5.6.4 Alternativas ao IDPT — levantamento de 14/08/2026

Registrado para ninguém repetir a busca. O que **nao serve**, e por que:

FONTE	VEREDITO
<code>arbm/multilingual_irony</code> (HF)	so <code>tweet_id</code> + <code>label</code> , sem texto ; reidratar exige API paga do Twitter
<code>ramondomingos/ptbr-irony-idioms-regionalism</code> (HF)	< 1.000 linhas, e e <i>benchmark de LLM</i> com gabarito em prosa — nao e corpus de classificacao
<code>rafaelanchieta/PILN</code> (GitHub)	sao os modelos treinados do IDPT, nao o corpus; embeddings dependem de servidor antigo do NILC
IroSvA / SemEval-2018 traduzidos	desaconselhado : traducao automatica reintroduz o vazamento de procedencia que ja custou o primeiro fusor

O candidato mais citado e o corpus de **Goncalves et al., BraSNAM 2015** ("Bazinga! Caracterizando e Detectando Sarcasmo e Ironia no Twitter", UFMG), que circula no Kaggle. Lendo o artigo: sao ~2.628 tweets por classe coletados pelas hashtags `#sarcasm` e `#irony` e caracterizados com LIWC — ou seja, e um trabalho brasileiro sobre o Twitter **anglofono**. Corpus em ingles. Usa-lo aqui seria o vazamento de procedencia com selo academico.

Verificado também e descartado: [OpenNeuro ds004533](#) ("Ironia VEV") e **neuroimagem** — ressonância de 40 sujeitos ouvindo frases ironicas, sem nenhuma frase rotulada para treino.

5.6.5 Onde existe corpus PT-BR de ironia (e como pedir)

Dois, os dois **sem download publico**:

1. **Vieira e Silva, A. (2025)** — *Detecção automática de ironia por meio de representações contextuais*, tese de doutorado em Linguística, USP, orientação de Marcos Lopes. Produziu um corpus de ironia em português com conversas da rede X: **1.186 exemplos** finais (de 1.200), anotados por **tres voluntarios humanos**, com o contexto da conversa. Achado relevante para o relatório: os anotadores precisaram do **contexto de produção em ~30% dos casos** — evidencia direta de que ironia em mensagem isolada tem teto baixo, e ela vale citada mesmo sem o corpus em mãos.
2. **Projeto IDPT / UFPel** — <https://institucional.ufpel.edu.br/projetos/id/u3345>, coordenação de **Larissa Astrogildo de Freitas**. É o grupo por trás do IDPT 2021. Enderece o pedido a coordenação.

Tamanho importa aqui: 1.186 exemplos anotados à mão não treinam um BERT do zero, mas são excelentes como **conjunto de teste independente** — exatamente o papel que o XED-pt cumpre no notebook 03. É a arquitetura de honestidade que o projeto já usa: treinar no que há em volume, medir no que há em qualidade.

5.6.6 O padrão: corpus sintético

Enquanto nenhum dos dois chega, o notebook gera o próprio corpus com [fraus.ingest.gerador_ironia](#). É o mesmo impasse do sinal de tempo e a mesma saída já aceita: o fusor treina em conversas sintéticas porque nenhum corpus de review tem timestamps.

O gerador foi escrito **contra** o vazamento que matou o primeiro fusor. A armadilha óbvia seria "ironico = elogio + fato ruim", que ensinaria o modelo a procurar fato ruim em vez da incongruência. Três cruzamentos impedem isso, cada um travado por teste em [tests/test_gerador_ironia.py](#):

- os mesmos **fatos ruins** aparecem nas reclamações diretas (não-ironicas);
- as mesmas **palavras elogiosas** aparecem em elogios sinceros e em reclamações que começam agradecendo;
- **numeros e unidades de tempo** aparecem nas duas classes;
- há uma família de ironia **por atenuação**, sem palavra elogiosa nenhuma, para que "tem elogio" não seja condição necessária.

A métrica interna será otimista por construção — ela mede o quanto o modelo aprendeu estes padrões, não o quanto lê ironia humana. Isso precisa estar no relatório.

Por isso o [metricas_ironia.json](#) grava a **procedência derivada de** [ORIGEM](#), nunca [IDPT 2021](#) fixo: artefato que mente sobre a própria fonte e pior que artefato ausente.

Como o notebook **nunca viu os arquivos**, a célula de carga tem esquema **configurável**: ela lê todo [.csv](#)/[.tsv](#) da pasta, imprime as colunas encontradas, tenta achar a de texto e a de rótulo pelos nomes mais prováveis e **para nomeando as colunas disponíveis** se não achar. Rótulo fora do mapeamento também para com erro, em vez de virar 0 silenciosamente.

Se o corpus nao for liberado a tempo, a saida honesta e declarar a ironia como trabalho futuro no relatorio. Trocar por um corpus de sarcasmo em ingles traduzido repetiria, com outro nome, o vazamento de procedencia que ja custou o primeiro fusor.

5.6.7 Portao de promocao do artefato

A acuracia/F1 do holdout sintetico **nao autoriza** substituir o modelo em producao. Depois do notebook 04, copie primeiro o checkpoint para uma pasta de candidato e rode:

```
uv run python scripts/validar_candidato_ironia.py /caminho/do/candidato \
--saida-json metricas_regua_ironia.json
```

O comando exige no maximo 30% de falso positivo nas dez falas sinceras com marcador e 30% de falso negativo nas dez ironias sem marcador. Ele devolve codigo 1 se qualquer fatia falhar, para que o artefato nao seja promovido por uma unica media boa. As falas e o calculo canonicos moram em [fraus/avaliacao_ironia.py](#); o laudo por fatias usa a mesma fonte.

Essa e uma **regua autoral de regressao**, deliberadamente desenhada contra os atalhos ja encontrados. Nao e corpus independente, nao estima acuracia em atendimento real e nao torna honesto reativar a ironia no fusor. Um corpus humano PT-BR separado do treino continua sendo o bloqueio cientifico.

A metrica de selecao do melhor checkpoint e o **F1 da classe ironica isolada**, nao a macro nem a acuracia: e essa classe que o sinal consome, e corpus de ironia costuma ser desequilibrado o bastante para um modelo que responde "nao e ironia" sempre marcar boa acuracia. O treino tambem usa pesos de classe, pelo mesmo motivo.

Por que uma cabeca separada e nao mais uma classe do sinal de texto: ironia nao e um sentimento, e uma relacao entre o que o texto DIZ e o que ele SIGNIFICA. "Que atendimento maravilhoso, so esperei 3 horas" e lexicamente positivo e pragmaticamente negativo ao mesmo tempo. Como quarta classe de satisfacao, o modelo seria obrigado a escolher uma das duas leituras — e a informacao de que ha conflito, que e justamente o sinal de ironia, se perderia.

Limitacao a declarar: o IDPT e anotado em tweets e comentarios de noticia, nao em atendimento de chatbot. A transferencia de dominio nao esta verificada — ironia em reclamacao de suporte pode ter forma diferente de ironia em comentario de politica.

■ 5.7 SINAL LEXICO — SENTILEX-PT02

Nao tem notebook: e recurso pronto, nao treinado.

Fonte: SentiLex-PT02 — Silva, Carvalho e Sarmento, "*Building a Sentiment Lexicon for Social Judgement Mining*" (PROPOR 2012), CC-BY. 82.347 formas flexionadas na origem, **79.189** depois da conversao.

Preparo: [scripts/preparar_sentilex.py](#) converte o arquivo original em [fraus/dados/sentilex_pt02.csv](#) e imprime o relatorio do que mexeu — 315 entradas cuja polaridade veio do complemento ([POL:N1]), 9 com sinal normalizado (marcadas [REV:POL] na fonte, com o contador vazado no campo de polaridade) e 2.032 formas homografias com polaridades conflitantes, que foram **zeradas** em vez de decididas pela ordem do arquivo.

Por que um lexicon se o BERTimbau ja le o texto: porque ele nao foi treinado nos mesmos dados nem carrega os mesmos vieses. Dois sinais de procedencia independente que concordam sustentam a nota melhor que um sozinho — e a **discordancia** entre eles e o formato tipico da ironia.

Negacao por escopo: "nao foi otimo" inverte a polaridade de "otimo", com alcance de tres tokens, e o escopo **morre na pontuacao forte** — em "nao chegou. otimo atendimento" o *otimo* continua positivo.

Acento: o SentiLex e acentuado e cliente de chat nem sempre. Ha indice de reserva sem acento; das 74.443 chaves resultantes, apenas **24** colidem ([*incomodo*]/[*incomodo*], [*ingenua*]/[*ingenua*]) e essas ficam zeradas — chute de polaridade errado e pior que termo ausente.

Limitacao do recurso, a declarar: o SentiLex e lexicon de **julgamento social** — anota polaridade dirigida a entidades humanas. E forte no adjetivo que julga ([*pessimo*], [*otimo*], [*incompetente*]) e **neutro em verbo de afeto do proprio falante:** [*gostar*], [*adorar*] e [*odiar*] valem 0 nele. Por isso o sinal lexico complementa o BERTimbau e nao o substitui — quem le "adorei o produto" e o transformer.

■ 5.8 SINAL DE ESTILO

Nao tem notebook: e **deterministico**, nao treinado — [*fraus/sinais/estilo.py*] mede a FORMA da escrita (caixa alta, pontuacao enfatica, alongamento, palavrão com gradacao, censura), nao o conteudo.

Por que deterministico e nao um classificador. O BERTimbau nao aprende enfase porque o B2W-Reviews01 — resenha moderada de e-commerce — praticamente nao contem gritaria nem xingamento. Modelo nao aprende fenomeno que o corpus de treino nao tem, e mais epocas sobre o mesmo texto so reproduziriam o mesmo artefato. Regra escrita a mao cobre exatamente o que falta: forma, nao semantica.

Fonte do lexicon de palavra: [*fraus/dados/palavroes_ptbr.csv*], curadoria propria, ~172 termos, colunas [*termo,intensidade,alvo*]. A intensidade e graduada ([*leve*]/[*medio*]/[*pesado*], mapeada para 0,33/0,66/1,0) porque "que droga" e "vai tomar no cu" nao sao o mesmo evento, e o alvo ([*pessoa*]/[*desabafo*]) existe porque xingar o PRODUTO e reclamacao enquanto xingar o ATENDENTE e ruptura da conversa. Nenhuma dependencia externa entrou para isso, e o SentiLex-PT02 tambem nao serve — ele anota polaridade de julgamento, nao gradacao de baixo calao nem alvo do xingamento.

Ordem censura-antes-de-normalizacao. [*tem_censura*] roda sobre a palavra CRUA, com a mascara ([**@#%&*]) ainda intacta, e so depois [*normalizar*] reverte homoglifo e apaga acento. Inverter a ordem apagaria a marca de censura antes de ela ser vista — "p*rra" perderia o asterisco e viraria uma palavra qualquer, nunca reconhecida como autocensura.

Casamento por curinga. [*casar_censurado*] trata cada simbolo de mascara como uma letra qualquer (regex com [*.*] no lugar do simbolo), nao como uma letra fixa: [*p*rra*] casa [*porra*], [*c*ralho*] casa [*caralho*]. Mapear o simbolo para uma vogal fixa acertaria um caso e erraria o outro em silencio. Quando mais de um termo do lexicon casa o mesmo padrao com curinga, o desempate e pela MAIOR intensidade — deterministico, e nao pela ordem em que a linha aparece no CSV, o que deixaria o resultado dependente de onde alguem inseriu a linha.

A execucao do [*kkkk*]. Alongamento de caractere ([*MINIMO_ALONGAMENTO = 3*] repeticoes seguidas) e tratado como enfase, exceto para as letras de riso ([*k*], [*h*]): "kkkk" e o marcador POSITIVO mais comum do chat brasileiro, e contar risada junto de "naooooo" inverteria o sentido da feature em boa parte das conversas reais.

Limitação a declarar: o lexicon de palavras e curadoria própria, não recurso acadêmico publicado. Diferente do SentiLex-PT02 (Silva, Carvalho e Sarmiento, PROPOR 2012) e do Emoji Sentiment Ranking usados pelos outros sinais, `palavroes_ptbr.csv` não tem paper citável nem revisão por pares — a cobertura e a graduação vieram de julgamento próprio, e isso precisa aparecer no relatório como o que é.

■ 5.9 NOTEBOOK 05 — CALIBRAÇÃO, E POR QUE ELE NÃO RETREINA NADA

Acrescentado em 10/09/2026. Ele ajusta **um escalar por cabeça** (a temperatura) sobre os checkpoints que já existem, e mede o quanto isso aproxima a confiança declarada da frequência empírica — o **ECE**, mais o diagrama de confiabilidade.

Temperature scaling não muda a decisão do modelo. Dividir todos os logits pelo mesmo `T > 0` preserva a ordem, então acurácia e F1 ficam idênticos. É a única forma de calibração que não pode piorar o classificador — e o notebook tem uma asserção que falha se o `argmax` mudar, porque isso seria defeito de código, não propriedade do método.

Ele produz `modelos/calibracao.json`. **Nada no runtime consome esse arquivo ainda, e isso é deliberado:** aplicar a temperatura muda o score de toda conversa do banco, e a régua deste projeto para trocar o que gera número é medir o efeito em `score/nota/categoria` antes de promover — a mesma disciplina que recusou a quantização int8 (ver `docs/encolhimento.md`).

A ironia é o caso a não confundir. Calibrar vai melhorar o ECE dela e ela vai continuar errando 6 em 10 falas sinceras de atendimento. Temperatura ajusta *o quanto* o modelo se diz confiante, não *sobre o que* ele está confiante. Apresentar a melhora do ECE como conserto do domínio seria falso.

■ 5.10 NOTEBOOK 02 — A SEÇÃO 10 É UMA PENDÊNCIA, NÃO UMA ETAPA

A saturação das features de tempo está descrita lá com a medição, o conserto (`winsorizar no p99` + `log1p` antes do scaler) e — o que mais importa — **o acoplamento**: a transformação precisa existir no notebook e no runtime, ou o score sai errado em silêncio. Um `fusor.joblib` treinado sem a compressão contra um runtime que comprime continua tendo `n_features_in_ = 39`, então `Fusor.carregar` **não pega** essa incompatibilidade.

5.10.1 As três saídas, medidas em 15/09/2026 — decidido: só `log1p`

O instrumento (hoje `scripts/retreinar_fusor_local.py`; a versão de medição está no commit `e719218`) reproduz este notebook localmente (mesmo B2W, mesmas sementes, mesmo `montar_features`, extração por ONNX) e **confere a reprodução antes de medir**: acurácia 0,9500 nos dois, score diferindo no máximo 0,0001 do `fusor.joblib` vigente. Nada foi promovido.

A coluna de espera é a frase canônica (texto 0,858 satisfeito, todo o resto na média do treino), com score, nota e categoria:

VARIANTE	ACURÁCIA	F1	CATEGORIAS ≠ VIGENTE (DE 300)	60 S	300 S	600 S
vigente	0,9500	0,9501	—	99,9 · 10 P	98,1 · 10 P	44,5
só <code>log1p</code>	0,9467	0,9469	6	99,7 · 10 P	96,6 · 10 P	90,1
só teto p99	0,9433	0,9434	1	99,9 · 10 P	96,1 · 10 P	18,4
p99 + <code>log1p</code>	0,9467	0,9469	5	99,7 · 10 P	96,6 · 10 P	90,7

O que a tabela muda na recomendação escrita acima:

- **p99 + `log1p` conserta o relógio mandando sozinho apagando a espera longa.** O teto do treino é ~7 min (`latencia_mediana s` p99 = 424 s), então três horas pesam o mesmo que dez minutos: a frase canônica, que é **irônica**, sai promotor. Consertar um defeito criando o oposto não é conserto.
- **Só teto** não resolve nada: o penhasco só muda de lugar (10 min → nota 2).
- **Só `log1p`** é a única que mantém a espera **monótona e sem penhasco**: 10 min ainda promotor, 30 min detrator 6, 3 h detrator 1 — o texto modula, o relógio pesa cada vez mais devagar, e nenhum dos dois apaga o outro. Custa 0,33 ponto de acurácia no teste sintético.
- **Ablação da invariante 10:** só as quatro features de tempo acertam 0,513 (cru) e 0,527 (p99+log1p), contra 0,333 do acaso. O relógio carrega sinal do rótulo no simulador por construção, e nenhuma das variantes muda isso.

5.10.2 Promovido em 15/09/2026

As três partes indivisíveis, feitas juntas:

1. **Função única.** `log1p` mora em `fraus.fusor.vetorizar`, sobre `FEATURES EM LOG`. As chaves continuam em segundos (atribuição, tela e deriva leem segundos); só o vetor que o scaler recebe é comprimido. Treino, pontuação, contribuição e deriva passam pela mesma função.
2. **Artefato regerado** por `scripts/retreinar_fusor_local.py --promover`: acurácia 0,9467, F1-macro 0,9469. O anterior ficou em `modelos/fusor.joblib.bak-segundos`. O artefato agora carrega `fraus_escalas_do_tempo = "log1p"`, e `Fusor.carregar` **recusa** artefato sem essa marca: é o caso que `n features in` não pega.
3. **Medição de categoria.** No teste do B2W, 27 notas e 6 de 300 categorias mudam. Nas 180 conversas do simulador, 23 notas e 7 categorias, com a distribuição indo de 60/63/57 para 60/60/60 (detrator/neutro/promotor).

`scripts/medir_dominio_do_tempo.py` contra o artefato novo: o relógio empata com o texto em **613 s** (antes 411 s), e com 3 h pesa **2,1x** o texto (antes 20x). Uma conversa sincera ("resolveu, muito obrigado") com 30 min de espera saía 0,30; agora sai 55,3, ainda detrator. Com 3 h, 21,95 em vez de 0,00.

Quem tem o artefato em outro lugar precisa regerar (a VM, o Drive): um `fusor.joblib` de antes de 15/09 não sobe mais, e isso é o comportamento correto da invariante 7.

6. Ironia: corpus e Laya

Estado em 08/10/2026. O BERTimbau de ironia foi retreinado no notebook 04 com a divisão por autor corrigida (treino 28.365 linhas, 58,8% irônicas; teste 4.778, 59,2%). Deu F1-macro de **0,994** no teste interno (0,991 só nos tweets) e $P(\text{irônico}) = 1,000$ em frases neutras de atendimento, como "qual o prazo de entrega do meu pedido" e "ok, obrigado". O modelo **não foi exportado** e o Laya não foi treinado. Esta página explica por quê e o que fazer antes do próximo treino. Medições e passos da sessão: nota interna [docs/notas/2026-10-08-ironia-corpus.md](#).

Produção não muda: a ironia em produção é lida pelo Laya sem treino (`laya-onnx`) e não entra no vetor do Fusor.

6.1 RESUMO: CONSERTAR O CORPUS ANTES DE TREINAR O LAYA

Treinar o Laya (mmBERT-base, `laya==0.3.21`) só em ironia é fácil de fazer e não resolve o problema que motivou a ideia. O vício do IDPT 2021 está nos dados, não no modelo nem na mistura de tarefas: a classe não irônica dos tweets é **100% de tweets com #economia**, publicados por ~658 contas, quase todas portais de notícia, só em maio e junho de 2017 ([arquivo bruto](#)). As notícias foram rotuladas **pelo site de origem**, "sem nenhuma rotulação manual" ([Marten & de Freitas, ENIAC 2020](#)). Qualquer encoder afinado nesse rótulo aprende registro e procedência, seja o Laya sozinho, o Laya multitarefa ou o BERTimbau. O formato de pergunta do Laya não protege contra isso, porque a instrução e os critérios são tokens idênticos em todos os exemplos e o gradiente vem só do texto. Chegar a "100% de exatidão" nesse teste não seria a prova esperada. Pela invariante 10 do projeto, seria o sintoma: ironia é uma tarefa em que anotadores humanos concordam pouco, e um acerto perfeito mostra que o teste mede a fonte. O ganho real virá de três coisas: trocar o negativo por fala coloquial sincera, medir numa régua de domínio com 200 a 400 itens em pares mínimos e rodar três treinos que separem as causas. Pela literatura, a diferença de encoder entre o Laya e o BERTimbau deve ficar em poucos pontos. Contra o multitarefa de 02/10, o Laya só-ironia quase certamente vai "ganhar" no teste interno, mas esse ganho virá da divisão corrigida, e não do fato de treinar só ironia.

6.2 O IDPT ENTREGA O RÓTULO, E NENHUM ENCODER ESCAPA DISSO

O achado de 08/10 confirma o que os próprios arquivos mostram. No `nao-ironico.csv`, o commit `b735dd9` que o Fraus fixa, as hashtags dominantes são **2.519 #economia** e **185 #econom**, e as datas vão só de 2017-05 a 2017-06. As contas mais frequentes são `noticiaminutobr`, `TSFRadio` (rádio portuguesa) e `portalR7`, e há tweets em espanhol e em português europeu no meio. Do outro lado, o `ironia.csv` tem ~12.700 tweets auto-marcados com `#ironia`, de ~9.300 autores, publicados entre 2014 e 2017. Só 78 tweets irônicos falam de economia, contra ~2.564 não irônicos ([dados verificados](#)). O `scripts/preparar_corpus_ironia.py` remove a hashtag `#ironia`, mas o vazamento continua no tópico, na fonte e na época. Nas notícias, `estadao.json` é sempre 0, e `sensacionalista.json` e `the_piaui_herald.json` são sempre 1. Ou seja, a tabela fonte × rótulo tem **células vazias**: nenhum tweet coloquial sincero e nenhuma notícia séria irônica.

O histórico público do IDPT já mostrava isso antes do Fraus. O treino oficial tem 15.212 tweets e 18.494 notícias, enquanto os testes são "300 tweets and 300 news gathered and annotated by the organizers" ([García Subies, CEUR Vol-2943](#)). No teste de tweets, anotado por humanos, **o melhor time chegou a 0,52 de acurácia balanceada** ([Luz et al., BraSNAM 2023](#)), e o BERTimbau afinado ficou em 0,49–0,50, abaixo do acaso ([García Subies](#)). Enquanto isso, o SiDi-NLP relatou **F1 1,000 na validação de tweets**, tanto com BERTimbau quanto com mBERT ([CEUR Vol-2943](#)). O F1 de 0,99 que o BERTimbau retreinado deu no teste interno do Fraus, junto com P(irônico)=1,0 em frases neutras de atendimento, repete esse padrão: um número alto dentro da distribuição que desaba quando o rótulo vem de um humano. O padrão é conhecido na literatura. Quando o rótulo de sarcasmo passa a ser dado pelo próprio autor, o melhor modelo cai de F1 0,874 para 0,364 ([Oprea & Magdy, ACL 2020](#)). Filtrar os vieses do SNLI derruba um modelo de 92% para 62% ([Le Bras et al., 2020](#)).

A esperança de que o Laya "evitaria dados viciados" esbarra em como ele aprende. O fine-tuning do script oficial é completo, com encoder e cabeça juntos, sem LoRA. A sequência sempre começa com a mesma instrução e os mesmos critérios A/B de `PERGUNTA IRONIA` (`[laya/common.py]`, `[build_sequence]`). O que separa A de B é só o `[state]`, isto é, o texto. Se registro, língua ou gênero textual predizem o rótulo, o mmBERT aprende isso igualzinho ao BERTimbau. O próprio guia do Laya avisa: "The loop is only as good as the targets... treat their quality as the ceiling" ([finetune.md](#)). O único ponto a favor do Laya tinha sido o prior zero-shot. Sem treino, ele fez F1 0,554 no teste interno e 0,524 na régua, com 0% de falso positivo, melhor que o BERTimbau. O run de 02/10 mostrou que o fine-tuning completo apaga esse prior (nota interna de 02/10, `[docs/notas/2026-10-02-treino-laya.md]`).

O "100% de exatidão" é inalcançável por um motivo independente do corpus: ironia é ambígua até para humanos. No Tweets8JanSPI, anotado à mão, o Fleiss κ da ironia foi **0,38**, contra 0,59 do sentimento ([BraSNAM 2026](#)). O MultiPICO usa em média cinco anotadores por item justamente porque o julgamento varia ([ACL 2024](#)). Um modelo não pode concordar com um rótulo mais do que os humanos concordam entre si, a menos que esteja lendo outra coisa. A invariante 10 diz isso com outras palavras: "acurácia alta demais é sintoma, não vitória".

■ 6.3 TREINAR SÓ IRONIA EXIGE MEXER EM SETE CÉLULAS E NUM RÓTULO DE PRODUÇÃO

O pacote `[laya==0.3.21]` só faz inferência e não tem API de treino. O fine-tuning roda pelo script de pesquisa `[finetune_single_device.py]`, no commit `[d8a2e597]` ([script](#)). O notebook 07 já o executa em subprocesso e confere o SHA-256. O script aceita tarefa única direto: basta o JSONL ter apenas linhas de `[caso ironia(texto, p)]`, com `[gold.ironia.proBABILITIES = {"A": p, "B": 1-p}]`, em que A = irônico. A ordem das chaves de `[criteria]` define a ordem do alvo. Por isso `PERGUNTA IRONIA` tem de ser importada de `[fraus/sinais/ironia_laya.py]` e nunca redigitada, que é o mesmo cuidado que a invariante 8 pede para a ordem das classes.

As mudanças no notebook 07 ([notebooks/07_treino_laya.ipynb](#)) são estas, pela numeração "#" dos comentários das células:

CÉLULA	HOJE	PARA SÓ IRONIA
#2 (imports)	importa emoção e ironia	pode ficar; os nomes de emoção deixam de ser usados
#4 (corpus de emoção)	GoEmotions-pt + XED-pt	pular inteira ; define <code>dados_emocao</code> , <code>emocao_teste</code> , <code>frases_xed</code> , usados adiante
#5 (divisão de ironia)	<code>dividir_por_autor</code> , <code>conferir_divisao</code> , guarda de impressão contra <code>metricas_ironia.json</code>	não mexer : é o que garante que o BERTimbau do notebook 04 foi treinado na mesma divisão
#6 (JSONL)	laço de emoção com <code>repeticoes_por_classe</code> + ironia suavizada 0,05	só ironia; <code>proibidos</code> = teste de ironia u régua; manter o <code>assert</code> de vazamento
#7 (treino)	<code>MODO_FUMACA</code> , <code>EPOCAS=4</code>	fumaça primeiro (600 casos), depois completo
#8 (comparação)	confere <code>DIR_BERT_EMOCAO</code> , prevê emoção e XED	tirar a emoção; manter <code>prever_*</code> , <code>comparar</code> , <code>laya.load</code>
#9 (ironia)	teste interno, recortes <code>tweets</code> / <code>noticias</code> , régua	sem mudança
#10 (poda + ONNX)	vocabulário com emoção, <code>cobertura_xed</code>	<code>perguntas = {**PERGUNTA_IRONIA}</code> ; a cobertura precisa de outro texto independente
#11 (manifesto)	<code>perguntas: ["emocao", "ironia"]</code>	<code>["ironia"]</code> ; ver a armadilha abaixo
#12 (laudo)	conjuntos de emoção e ironia	só <code>ironia_interno</code> , <code>ironia_tweets</code> , <code>ironia_noticias</code> , <code>ironia_regua</code>

A **armadilha do manifesto** é a parte que não aparece no notebook. Em `fraus/sinais/emocao_laya.py`, `laya_treinado_pelo_fraus(diretorio)` devolve `"emocao" in perguntas_do_artefato(diretorio)`, e um manifesto ausente equivale a `["ironia", ""]`. Assim, um artefato treinado só em ironia é **indistinguível do checkpoint base**, e a dashboard o rotularia como "Laya sem treino", o que é falso. Há um segundo efeito, mais silencioso:

`obter_classificador_emocao_laya_onnx` reaproveita o mesmo agente e responderia emoção com uma cabeça afinada só em ironia. A correção precisa vir antes do treino e com TDD. O manifesto ganha um marcador explícito de procedência, por exemplo `"treinado_pelo_fraus": true` junto com a revisão e `rodada_de_fumaca: false`. `laya_treinado_pelo_fraus` passa a ler esse campo, e a leitura de emoção pelo Laya passa a recusar, com falha alta, um artefato que não declara `emocao`. Os dois testes falham primeiro com o manifesto só-ironia e passam depois.

Os hiperparâmetros estão quase todos fixos no código do script. A linha de comando aceita só `--data`, `--model-dir`, `--output-dir`, `--device`, `--epochs` e `--seed`. Mudar qualquer outra coisa obriga a editar o arquivo, o que quebra o SHA-256 conferido e precisa ser declarado como desvio do caminho oficial.

PARÂMETRO	VALOR NO SCRIPT	RECOMENDAÇÃO PARA SÓ IRONIA
Micro-lote / acumulação	8 / nenhuma (lote efetivo 8)	manter; a receita 2×T4 do autor usa lote efetivo 64, mas isso exige editar o script
LR encoder / cabeça	2,5e-5 / 1e-4, AdamW wd 0,01	manter
Agenda	cosseno até 1e-6, sem warmup; σ do ruído de 0,4 a 0,1	manter
Precisão	fp16 autocast + gradient checkpointing, clip 1,0	manter
<code>max len</code>	1024 (forçado)	controlar pelo <code>MAX CARACTERES</code> do notebook: 280-400 em vez de 1200, para aproximar o tamanho de uma mensagem de chat e baratear o treino
Épocas	padrão 4	2 a 3 : o corpus tem um atalho forte, e mais épocas tendem a decorá-lo
Semente	padrão 0	42 e mais duas sementes se o tempo couber, porque o fine-tuning é instável entre sementes (Dodge et al. 2020 ; Mosbach et al. 2021)
Peso de classe	inexistente	dispensável: a divisão corrigida tem 59% de irônicas dos dois lados ($\approx 1,4:1$)
Suavização do alvo	0,05	considerar 0,1: rótulo por procedência não sustenta certeza de 0,975
Calibração	temperatura por LBFGS em <code>min(400, n//10)</code> itens separados antes do treino	manter; ela não muda o argmax, mas muda qualquer limiar de 0,5

Sobre o tempo na T4: o run de 02/10 processou 142.204 itens (71.102×2 épocas) em 89 minutos, cerca de 26,6 itens/s com a carga incluída (nota interna de 02/10, [docs/notas/2026-10-02-treino-laya.md](#)). Com os 28.365 itens de treino da divisão corrigida, a conta direta dá **~18 minutos por época, ~35 minutos para 2 épocas e ~70-75 minutos para 4**. Essa é uma extrapolação de um único run. As notícias são bem mais longas que as frases do GoEmotions que dominavam aquele treino, então o tempo real pode ser 1,5 a 2× maior. A estimativa que vale é a que a rodada de fumaça imprime. O script não salva checkpoint por época. Chamá-lo várias vezes com `--epochs 1` também não equivale a um treino de N épocas, porque reinicia o cosseno, a σ do ruído e o otimizador e separa de novo a fatia de calibração. Como 2 a 3 épocas cabem numa sessão T4, o caminho mais simples é o atual: copiar para o Drive no fim.

6.4 CONSERTAR O NEGATIVO VALE MAIS QUE TROCAR O ENCODER

Os métodos de debiasing que mexem no modelo, como product-of-experts, autoviés, group DRO e DANN, só redistribuem peso entre exemplos que **contradizem** o atalho (Clark et al. 2019; Sagawa et al. 2020). Com as células "coloquial × não irônico" e "notícia séria × irônico" vazias, não existe exemplo conflitante para repesar. Se a fonte é quase igual ao rótulo, apagar a fonte por treino adversarial apaga o rótulo junto. A correção tem de ser nos dados. O primeiro passo custa minutos: imprimir a tabela de contingência fonte × rótulo e, além dela, treinar um classificador "adivinha a fonte". Se o rótulo sai da fonte, o corpus reprova antes de qualquer GPU, do mesmo jeito que a invariante 10 manda checar a sobreposição de distribuições por rótulo.

Não existe hoje um corpus PT-BR de ironia que seja grande, anotado por humanos e com negativos coloquiais naturais. O melhor candidato é o **MultiPICo-PT**: 1.994 pares post-resposta de Reddit e Twitter, metade brasileira e metade europeia, 49 anotadores nativos, ~38% irônicos, licença CC BY 4.0 e acesso livre no Hugging Face (MultiPICo, ACL 2024; HF). As duas classes saem da mesma amostragem e o rótulo vem do julgamento humano, não da fonte. O formato post-resposta combina com o Fraus, em que a fala do bot vem antes da do cliente. O Tweets8JanSPI (2.935 tweets, 224 irônicos, κ 0,38) tem o mesmo desenho, mas é distribuído desidratado, só com IDs, e trata de um único tópico (BraSNAM 2026). O Kaggle "vinciuscleves" é um clone do mesmo corpus do IDPT (Kaggle). O de manchetes The Onion × HuffPost reproduz o rótulo por procedência (Misra & Arora).

A receita recomendada tem quatro camadas, da mais à menos importante. A primeira é **descartar as notícias**, porque nenhum dos dois lados é fala de cliente, e **substituir o `nao-ironico.csv`** por negativos coloquiais estratificados por polaridade: muito elogio sincero e muita reclamação literal, por exemplo B2W com 5 e com 1 estrela (licença CC BY-NC-SA 4.0, compatível com uso acadêmico (README B2W)) e frases do GoEmotions-PT, que é Apache 2.0. Isso ataca o modo de falha já medido: a cabeça atual marca 74% das resenhas positivas da B2W como irônicas, contra 9% das negativas ([docs/treinamento.md], 04/09/2026). A segunda é **somar o MultiPICo-PT**, como fatia de treino e de validação. A terceira são os **pares mínimos escritos à mão** no domínio de atendimento, no molde da sub tarefa pareada do iSarcasmEval (Abu Farha et al. 2022) e dos negativos por micro-edição do Puntuguese (HF). Um exemplo de par: "ótimo serviço, resolveram rápido" × "ótimo serviço, só esperei 3 horas". Esses pares rendem mais como teste do que como treino, porque dados contrafactuais protegem as features perturbadas, mas podem piorar as que não foram perturbadas (Joshi & He 2022). A quarta é **minerar positivos de domínio** com heurística (elogio + situação negativa, "só que não", "parabéns" numa nota 1) e **confirmar cada um à mão**. Se a heurística virar o rótulo, o projeto volta ao rótulo por marcador. Os tweets irônicos do IDPT podem continuar como positivos, desde que tenham negativos do mesmo meio. Normalizar o estilo (minúsculas, sem URL, @, hashtag nem ponto final) é barato e remove pistas tipográficas, mas não remove o léxico de registro ("PIB", "Meirelles"). Por isso o efeito tem de ser medido com o baseline de estilo, antes e depois.

6.5 TRÊS TREINOS E UMA RÉGUA DE 300 PARES SEPARAM AS CAUSAS

O colapso de 02/10 tem uma explicação demonstrada que não depende da multitarefa. A divisão saiu com treino 86% irônico e teste 43% irônico, porque as 18.373 notícias sem autor caíam todas no teste. O Laya treinado chegou a recall de **0,988 no irônico e 0,087 no não irônico**, com F1-macro de 0,388. A mesma divisão fez o BERTimbau ficar em 0,511, e a rodada de fumaça já mostrava 100% de falso positivo (nota interna de 02/10, [docs/notas/2026-10-02-treino-laya.md]). Ainda assim, a hipótese de interferência entre tarefas tem base. No pre-

finetuning multitarefa, o desempenho cai quando há poucas tarefas (Muppet), e a mistura de 02/10 era 83% emoção para 17% ironia. Mas emoção e sentimento como tarefas auxiliares **ajudam** o sarcasmo em outros trabalhos (Chauhan et al. 2020). Só um desenho controlado decide.

TREINO	O QUE VARIA	COMPARAÇÃO QUE RESPONDE
(i) Laya só ironia	tarefa única, divisão corrigida	(i) × (ii): a multitarefa atrapalha?
(ii) Laya emoção + ironia balanceado	amostragem por temperatura ou uma época de ironia por época de emoção	(i) × (ii): se $\Delta \approx 0$, a hipótese da multitarefa cai
(iii) BERTimbau só ironia (notebook 04)	encoder	(i) × (iii): o encoder importa, com dado e tarefa iguais?

Os três treinos usam a mesma divisão, conferida pela impressão de `metricas_ironia.json`, e a mesma semente. Cada um deve rodar duas vezes: uma no IDPT corrigido e outra no corpus consertado da seção anterior. A troca de corpus é o fator que a literatura indica como dominante, e esse fator 2×3 é o que a banca vai querer ver. A divisão por autor não basta para medir o atalho. Hoje cada notícia sem autor vira um grupo próprio e se espalha pelos dois lados. Por isso é preciso uma função irmã de `dividir_por_autor` que agrupe por **fonte** (leave-one-source-out) e reaproveite `conferir_divisao`. Essa função vai falhar alto se a fonte for quase igual ao rótulo, e essa falha já é o diagnóstico.

O teste interno não decide nada, já que o BERTimbau marca 0,99 nele. Quem decide é uma **régua de domínio independente**, escrita por quem não viu as predições, congelada com `impressao_dos_textos` antes da primeira avaliação e marcada `independente=True` no laudo. A régua tem **200 a 400 itens em pares mínimos** e quatro estratos: elogio sincero, elogio irônico, reclamação literal (que separa ironia de sentimento negativo) e ironia sem marcador. Inclui ainda testes de invariância no estilo do Checklist (Ribeiro et al. 2020): o mesmo texto com e sem ponto final, em caixa alta ou baixa e com um emoji neutro tem de receber a mesma predição. A métrica principal é a **acurácia por par**, em que o modelo acerta os dois membros. O acaso aqui é 25%, e um modelo de estilo não consegue acertar os dois lados de um par com o mesmo registro. O tamanho sai de uma conta binomial própria: com acurácia em torno de 0,8, $n=100$ dá IC de $\pm 7,8$ pontos, $n=200$ dá $\pm 5,5$ e $n=400$ dá $\pm 3,9$. A régua atual tem 20 itens e IC de $-0,28$ a $+0,17$, larga demais para decidir qualquer coisa. Pares mínimos com troca de rótulo derrubam modelos em até 25% em relação ao teste original (Gardner et al. 2020), e é essa queda que o Faus precisa conseguir medir.

Três baselines entram como modelos extras em `preditos_por_modelo`, no espírito do baseline "só a hipótese", que acerta ~67% no SNLI (Gururangan et al. 2018). O primeiro é **só estilo**: regressão logística sobre ~15 features sem léxico (comprimento, proporção de maiúsculas, ponto final, `!?`, dígitos e R\$, URL, @, hashtag, emoji, alongamento, "kkk"). O segundo é **só fonte**. O terceiro é **TF-IDF + LR**, que mostra a parte lexical do registro. Se o baseline de estilo também chega perto de 0,99 no teste interno, o teste mede procedência. O modelo só conta como detector de ironia se vencer o baseline de estilo **na régua**, com diferença demonstrada. `fraus/` `comparacao_modelos.py` já tem `mcnemar_exato`, `bootstrap_da_diferenca` (2.000 reamostras, F1-macro), `avaliar_conjunto` e `montar_laudo`, que já distinguem procedência igual de independente. As quatro peças que faltavam já existem no mesmo módulo: IC de um único modelo, **bootstrap por par** (reamostrar pares, e não itens, para preservar a dependência dentro do par), acurácia por par e **ECE** com diagrama de confiabilidade (Guo et al. 2017). O ECE importa porque o Laya sai superconfiante e a temperatura dele é ajustada numa fatia do próprio corpus viciado.

Para o McNemar, o que conta são os pares discordantes: com ~20 discordantes, só uma divisão de 15:5 ou mais dá $p < 0,05$. Como guarda de regressão, entra a medição que já existe em [docs/treinamento.md](#): P(irônico) médio por estrela da B2W. Se a diferença entre resenhas positivas e negativas continuar grande, o atalho continua lá.

■ 6.6 O GANHO ESPERADO É DE POUCOS PONTOS, E VEM DO DADO

Contra o multitarefa de 02/10, o Laya só-ironia vai melhorar muito no teste interno: de 0,388 para algo bem acima, talvez perto do 0,99 do BERTimbau. Esse salto vem da divisão corrigida e da proporção de 59/59, que eliminam o colapso "tudo é irônico". Ele não deve ser atribuído ao treino em tarefa única nem apresentado como detecção de ironia. Na comparação justa, o treino (i) contra o (ii) balanceado, o tamanho de efeito publicado para multitarefa é de **poucos pontos, em qualquer direção** ([Standley et al. 2020](#); [Chauhan et al. 2020](#)). O que se espera é empate dentro do IC.

Contra o BERTimbau, com tarefa e dado iguais, a literatura não dá motivo para esperar diferença grande. Em PT-BR, o BERTimbau vence o mBERT por margens de 2 a 4 pontos, por exemplo 89,2 contra 86,8 de F1 no ASSIN2 RTE ([neuralmind-ai](#)). O mmBERT-base supera o XLM-R base no XTREME (72,8 contra 70,4), mas português é só ~2,4% da mistura de pré-treino, e o artigo não traz nenhum benchmark de classificação em português ([Marone et al. 2025](#)). Há um precedente em que o mmBERT-base venceu o BERTimbau em NER clínico em português, com micro-F1 de 0,76, num preprint ainda sem revisão ([arXiv 2603.26510](#)). Não existe resultado de mmBERT, XLM-R ou Laya no IDPT, nem de afinação do Laya em ironia em qualquer língua. Na CPU, o Laya é mais lento: 459 ms contra 273 ms por mensagem na ironia, com ressalva, porque o Laya rodou em ONNX FP32 e o BERTimbau em PyTorch (nota interna de 02/10, [docs/notas/2026-10-02-treino-laya.md](#)). A velocidade do ModernBERT só aparece em GPU e em sequências longas.

A única vantagem estrutural do Laya para ironia é o vocabulário multilíngue, que permite misturar o MultiPICO das outras oito línguas. A literatura, porém, é desfavorável à transferência de ironia entre línguas: o viés de tópico dos corpora atrapalha ([Ortega-Bueno et al. 2023](#)), e detectores de sarcasmo não generalizam entre datasets ([Jang & Frassinelli 2024](#)). O ganho que de fato muda o produto aparece na régua de domínio, quando se troca o corpus. Um modelo treinado no IDPT deve ficar perto do acaso por par, como o 0,52 oficial nos tweets. Um modelo treinado com negativos coloquiais e pares mínimos tem espaço para ganhar dezenas de pontos ali, na escala das quedas de 25 a 30 pontos que a literatura de contrast sets e filtragem adversarial atribui ao viés. Essa magnitude é inferência por analogia e não foi medida em português. A régua de 300 pares existe justamente para medi-la.

■ 6.7 RÉGUA DE PARES MÍNIMOS

Estado: **rascunho pronto, anotação em andamento**. Spec em `docs/superpowers/specs/2026-10-08-regua-pares-ironia-design.md` (interno).

- 150 pares (elogio, reclamação, neutra), registro equilibrado conferido por `conferir_registro_equilibrado` e, par a par, mesma vírgula e mesmo número de "?" nos dois lados. O baseline só de estilo acerta 0,18 dos pares fora da amostra no rascunho, abaixo do acaso (0,25). O rascunho é autoral; o rótulo que vale é o da anotação às cegas.
- A anotação mora no Fraus, por link e sem conta: `scripts/criar_link_anotacao.py`
`--quantos N` chama `POST /anotacao/anotadores` (só dev) e imprime `<id> https://fraus-one.vercel.app/anotar/<token>`. O banco guarda só o hash do token e um id opaco (`a` + 8 hex), nunca nome. A página recebe de `GET /anotacao/<token>` só `id` e `texto`, na ordem daquele anotador, e grava por `POST /anotacao/<token>/respostas`; o instante é do servidor. A primeira rodada (João, na página antiga do claude.ai) está versionada em `fraus/dados/anotacao_regua/anotador-claude-ai.json`, anotador `claude-ai-1`.
- Para um grupo de mensagens, `scripts/criar_link_anotacao.py --grupo 5` cria **um** link (`https://fraus-one.vercel.app/anotar/grupo/<token>`) com 5 vagas. Cada pessoa que o abre ganha o próprio anotador (a vaga é ocupada de forma serializada no banco, então 5 não viram 6) e a página guarda o link pessoal só naquele navegador. Em outro aparelho, abrir o link do grupo de novo gasta outra vaga e começa do zero; quem quiser trocar de aparelho guarda o endereço da página pessoal. Repassar o link do grupo também gasta vaga. Grupo cheio responde "Este link já foi usado por todas as pessoas previstas". Revogar o grupo (`POST /anotacao/grupos/<g_id>/revogar`) fecha a entrada; com `?anotadores=1` revoga também quem entrou, e as respostas saem do export.
- O token viaja na URL e fica nos logs de requisição da Vercel e no histórico do navegador, como o link de convite. Link vazado se revoga com `scripts/criar_link_anotacao.py --revogar <id>`; as respostas desse anotador saem do export.
- **Critério de promoção de qualquer cabeça de ironia:** vencer o baseline só de estilo (`fraus/baseline_estilo.py`) em acurácia por par na régua, com IC 95% da diferença excluindo zero. O teste interno do IDPT não entra. No laudo isso é o campo `candidato vence por par` (limite inferior do IC da diferença acima de zero). `por par demonstrada` só diz que o IC exclui zero e também acende quando o candidato é significativamente pior; para promover, vale `candidato vence por par`.
- Depois da anotação: `uv run python scripts/exportar_anotacao.py respostas-fraus.json` baixa `GET /anotacao/respostas` (só dev) e `uv run python scripts/consolidar_regua_ironia.py fraus/dados/anotacao_regua/anotador-claude-ai.json respostas-fraus.json` concatena os arquivos e consolida. Menos de 100 pares confirmados para o script com erro dizendo quantos faltam.
- Cada registro é `{"anotador": "...", "frase_id": "...", "resposta": "ironico|nao ironico|contexto", "instante": "2026-10-08T12:00:00.000000+00:00"}`; o script usa a última resposta de cada anotador por frase.
- A página embaralha a ordem por anotador (semente derivada do id) e nunca põe os dois lados de um par em sequência. A régua congelada guarda duas impressões no meta: uma dos textos e outra de `par_id|rótulo|texto`, de modo que trocar um rótulo ou mover um par também falha alto na carga.

■ 6.8 CONCLUSÃO

A pergunta "Laya ou BERTimbau, só ironia ou multitarefa" foi formulada sobre um corpus que não sustenta nenhuma das quatro respostas, porque nele o rótulo é a fonte. A decisão que de fato move o resultado vem antes do encoder. São três passos: trocar o `nao-ironico.csv` de #economia e as notícias por fala coloquial sincera,

construir a régua de pares mínimos e comparar cada modelo com um baseline de estilo que não sabe nada de ironia. O Laya só-ironia continua valendo como experimento, como a célula (i) de um desenho controlado e não como conserto, e só depois de corrigir `laya_treinado_pelo_fraus` para que o produto não rotule um modelo treinado como "sem treino".

Para a banca, a meta de 100% se transforma num argumento a favor do projeto. Mostrar que o BERTimbau marca 0,99 no teste interno, que um classificador de pontuação e caixa alta chega perto disso, e que ambos caem para perto do acaso numa régua de pares mínimos é uma demonstração empírica da invariante 10 em ironia. Essa demonstração convence mais do que qualquer número alto. Promover o modelo depende de ele vencer o baseline de estilo na régua independente, com IC que exclua zero. O teste interno não serve para essa decisão.

7. Integração com o Tars

Os projetos continuam independentes. O Tars é o **produtor** das conversas e o Fraus é o **consumidor** analítico; nenhum acessa o banco ou importa código do outro.

■ 7.1 CONTRATO

1. No Fraus, crie uma fonte do tipo `webhook` e gere sua chave `frs_...`.
2. No servidor do Tars, configure:

```
FRAUS_API_URL=https://api-fraus.exemplo.com
FRAUS_FONTE_CHAVE=frs_...
```

1. Autenticado no Tars, chame:

```
POST /api/integracoes/fraus/sincronizar
Content-Type: application/json

{"instancia_id":"...", "desde":"2026-09-01T00:00:00Z", "limite":100}
```

O Tars agrupa mensagens por instância, canal e chat, envia timestamps e informa se houve handoff. Quando existe “Resolveu/Não resolveu”, o rótulo segue como `feedback declarado`; ele **não altera a nota**, apenas mede concordância com a inferência em `GET /indicadores`.

■ 7.2 IDEMPOTÊNCIA E PRIVACIDADE

O ID externo usa `instancia:canal:sha256(chat)`; o identificador do visitante não atravessa a fronteira em claro. No Fraus ele recebe ainda o namespace da fonte (`fonte:<id>:...`), portanto fontes diferentes nunca se sobrepõem. Repetir a sincronização atualiza o mesmo atendimento. Antes de persistir, o Fraus aplica sua censura de PII ao texto e ao comentário do feedback.

A sincronização é explícita e fica fora do caminho de resposta do chatbot: se o Fraus estiver indisponível, o atendimento do Tars continua funcionando. A rota retorna `207` quando apenas parte do lote falha, com o resultado por conversa.

■ 7.3 AUTOMAÇÃO OPCIONAL

Um agendador pode chamar a rota com uma janela sobreposta (por exemplo, as últimas 24 horas). Como o envio é idempotente, a sobreposição é preferível a perder conversas na fronteira entre duas execuções.

name: Fraus — Instrumento description: Painel de instrumento de laboratório sobre quase-preto neutro; todo número é um display de sete segmentos e a ausência de dado é um segmento apagado. colors: background: "oklch(0.14 0.004 285)" foreground: "oklch(0.96 0.003 285)" card: "oklch(0.185 0.005 285)" popover: "oklch(0.175 0.005 285)" sidebar: "oklch(0.16 0.004 285)" muted: "oklch(0.245 0.006 285)" muted-foreground: "oklch(0.74 0.01 285)" linha: "oklch(0.96 0.003 285 / 26%)" compasso: "oklch(0.96 0.003 285 / 9%)" border: "oklch(0.96 0.003 285 / 13%)" primary: "oklch(0.83 0.095 84)" primary-foreground: "oklch(0.15 0.02 84)" dito: "oklch(0.78 0.14 72)" dito-texto: "oklch(0.85 0.125 72)" medido: "oklch(0.74 0.13 250)" medido-texto: "oklch(0.82 0.105 250)" detrator: "oklch(0.67 0.24 22)" neutro: "oklch(0.78 0.15 90)" promotor: "oklch(0.79 0.18 190)" sem-sinal: "oklch(0.705 0.015 286.067)" destructive: "oklch(0.79 0.121 22.216)" success: "oklch(0.78 0.16 150)" warning: "oklch(0.8 0.14 80)" atelie-halo: "oklch(0.62 0.2 320)" typography: display-vitrine: fontFamily: "Bricolage Grotesque, ui-sans-serif, system-ui, sans-serif" fontSize: "clamp(2.6rem, min(8vw, 10.5vh), 6.5rem)" fontWeight: 500 lineHeight: 0.95 letterSpacing: "-0.035em" titulo-vitrine: fontFamily: "Bricolage Grotesque, ui-sans-serif, system-ui, sans-serif" fontSize: "clamp(2.1rem, 4.6vw, 3.75rem)" fontWeight: 500 lineHeight: 1.02 letterSpacing: "-0.028em" titulo-instrumento: fontFamily: "Martian Mono, ui-monospace, Cascadia Mono, monospace" fontWeight: 500 letterSpacing: "-0.02em" rotulo-instrumento: fontFamily: "Martian Mono, ui-monospace, Cascadia Mono, monospace" fontSize: "0.6875rem" fontWeight: 400 lineHeight: 1.3 letterSpacing: "0.14em" prosa: fontFamily: "Inter, ui-sans-serif, system-ui, sans-serif" fontSize: "1.0625rem" lineHeight: 1.7 dado: fontFamily: "Martian Mono, ui-monospace, Cascadia Mono, monospace" fontFeature: "tabular-nums" rounded: none: "0" pilula: "999px" cabeca: "50%" spacing: telemetria: "34px" gutter-vitrine: "clamp(1rem, 4vw, 3rem)" secao-vitrine: "clamp(4.5rem, 9vw, 8rem)" pauta: "13px" components: telemetria-topo: backgroundColor: "{colors.background}" textColor: "{colors.muted-foreground}" typography: "{typography.rotulo-instrumento}" height: "34px" painel: backgroundColor: "{colors.card}" textColor: "{colors.foreground}" rounded: "{rounded.none}" padding: "16px 20px" botao-primario-vitrine: backgroundColor: "{colors.background}" textColor: "{colors.foreground}" rounded: "{rounded.none}" padding: "0 1.6rem" height: "3rem" botao-acessar-nav: backgroundColor: "{colors.primary}" textColor: "{colors.primary-foreground}" rounded: "{rounded.pilula}" padding: "0.6rem 1rem" segmento-led: textColor: "{colors.medido}"

8. Design System: Fraus — Instrumento

Registrado em 30/09/2026 a partir do que foi **construído** ([app/globals.css](#), [app/lp.css](#), [components/instrumento/**](#), [components/lp-nova/**](#), [components/shell/**](#)), não do plano. Substitui por inteiro o mundo "Pauta / espaço profundo", que foi descartado; o que dele continua vivo está aqui, com o motivo. Contrato da rodada: [docs/superpowers/specs/2026-09-30-instrumento-design.md](#). Verdade de produto: [PRODUCT.md](#).

8.1 OVERVIEW

Creative North Star: "O Instrumento de Bancada"

Um painel de laboratório sobre quase-preto neutro. Todo número medido é um display de sete segmentos; o segmento **apagado também é desenhado**, então "sem sinal" é um medidor ligado que não leu nada, nunca um

`0`. É a invariante 2 do CLAUDE.md virada objeto. Superfície é retângulo de hairline de 1px, raio zero, rótulo em mono caixa alta. Nada de cartão colorido, nada de cenário: o único cenário que sobra é o orbe da vitrine.

A tese que atravessa a interface inteira não mudou: **o dito fica acima da régua, o medido abaixo**. Âmbar é o que o cliente articulou (fala, texto, emoji); azul é o que o sistema mediu (score, probabilidade, latência, tendência). O produto vende a distância entre os dois, então ela nunca pode ficar invisível. O LED é azul porque número é medido.

Fusão aprovada pelo dono sobre mockups estáticos (seed 197a1d0c): **Régua** (telemetria, hairlines, mono, nav numerado, a régua dito/medido) + **Segmento** (LED, apagado = ausência, lâmpada sempre com palavra) + **Aurora** (só na vitrine: orbe, headline gigante, nav de vidro em pílula, CTA de borda cônica, halo e grão). **Tinta e tom** foi retirado da fusão por decisão do dono: nada de balão, halftone, linhas de velocidade ou sarjeta.

Key Characteristics: - Quase-preto neutro (croma 0,004) com traço de matiz 285; degraus de superfície são passos de claridade, não de cor. - Hairline 1px e raio 0; a linha de 26% (`--linha`) **afirma**, a de 9% (`--compasso`) só agrupa. - Martian Mono em rótulo, dado e título de tela; Inter na prosa; Bricolage só no display da vitrine. - Três cores com dono: âmbar dito, azul medido, dourado só em ação, foco e marca. Magenta só no orbe e no halo. - Um momento vivo na vitrine (o orbe); a ferramenta é estática.

■ 8.2 COLORS

Estratégia: **neutro quase-preto restrito, com duas vozes de dado e um metal de marca**. Tudo em OKLCH; o frontmatter é a fonte normativa e `scripts/contraste.mjs` só lê `oklch()` literal.

8.2.1 Primary

- **Ouro fosco do monograma** (`--primary`, oklch 0.83 0.095 84): cor de **ação e foco**, medida do próprio `public/fraus-logo.svg`. Preenche controle, anel de foco, cursor `■` do nav ativo, o `00` da telemetria. **Nunca** codifica dado. Separa-se do âmbar do dito pelo croma (fosco 0,095 contra saturado 0,14): metálico contra pigmento; e por nunca dividirem superfície.

8.2.2 Secondary (as duas vozes)

- **Âmbar do dito** (`--dito` 0.78 0.14 72; texto `--dito-texto` 0.85 0.125 72): fala, texto, emoji. Acima da régua.
- **Azul do medido** (`--medido` 0.74 0.13 250; texto `--medido-texto` 0.82 0.105 250): score, probabilidade, latência. Abaixo da régua. Cor padrão do `SegmentoLED`.
- Variantes `--fraco` (18% de mistura) para preenchimento; `--texto` para tipo. Cor de marcação e cor de tipo não são a mesma coisa.

8.2.3 Tertiary (categorias, famílias, grafo)

- **Escala de NPS:** `--detrator` (0.67 0.24 22), `--neutro` (0.78 0.15 90), `--promotor` (0.79 0.18 190, teal, longe do dourado de propósito). Sempre com rótulo escrito.
- **Sem sinal** (`--sem-sinal`, 0.705 0.015 286): cinza fora da escala, de propósito.

- **Famílias do fusor:** `--emocao` (10), `--lexico` (140), `--ironia` (230), `--estilo` (170), `--incongruencia` (290, croma 0,15: o gamut estreita nesse matiz), `--tempo` (320, série sempre tracejada). Cada família tem `--fraco` e `--texto`; matiz de nenhuma é reciclado. `--outros-sinal` é cinza para prefixo que nenhuma família reconhece.
- **Sete emoções** (`--emo-medo` 315, `--nojo` 160, `--tristeza` 245, `--raiva` 0, `--surpresa` 280, `--alegria` 125, `--neutro` cinza acromático): cor de **marca, nunca de texto**. Validadas por vizinhos na ordem fixa da pauta (`ORDEM EMOCOES` em `lib/partitura.ts`): medo, nojo, tristeza, raiva, surpresa, alegria, neutro). Nove cores num painel não se separam sozinhas; o que as legitima é o nome escrito em cada linha e a validação por vizinhos (CVD ΔE 12,5, visão normal 16,2, $\geq 3:1$ sobre o card, medidos em 15/09/2026). Trocar um tom ou a ordem sem rodar o validador desfaz a medição. Desprezo (raiva + nojo) é a nota vazada no tom da raiva.
- **Grafo, nove tipos de nó:** `--no-*` com L 0,79 / C 0,19 iguais e só o matiz variando. Quente (30–95) = dito: emoji 30, conversa 60, termo 95. Frio (150–340) = medido/cadastrado: fonte 150, canal 195, feature 265, categoria 305, desfecho 340. `--no-importacao` é cinza (0.7 0 0): nó solto sem origem rastreável, ausência de afirmação. Claridade igual é deliberada: nenhum tipo é mais importante que outro. A tela carrega legenda própria agrupada nas duas famílias.
- **Erro / sucesso / aviso:** `--destructive`, `--success`, `--warning` (este compartilha o matiz do dourado e só vive como texto, nunca como preenchimento) e as trincas `--rich`, `--rich-border`, `--rich-text`.

8.2.4 Neutral

- **Quase-preto** (`--background`), **card** 0.185, **popover** 0.175, **sidebar** 0.16, **muted/secondary/accent** 0.245. Não é preto puro: preto puro faz a régua de meio ponto flutuar no vácuo (argumento herdado do grafite).
- `--muted-foreground` (0.74 0.01 285) é o **piso de cor de texto**: nada mais escuro que isto carrega texto.
- **Halo** (`--atellie-halo`, magenta 0.62 0.2 320): cenografia da vitrine. Sem canal de significado.

8.2.5 Named Rules

A Regra do Dito e do Medido. Âmbar acima da linha, azul abaixo. Onde as duas séries são medidas (NPS × latência) a régua **não** entra, senão vira notação decorativa. `Painel` aceita a prop `regua` só quando o dado realmente se divide. **A Regra do Ouro Fora do Dado.** `--primary` nunca aparece numa série, barra, categoria ou célula. Só ação primária, foco de teclado, cursor do nav, valor de telemetria e a marca. Uma exceção nomeada legada: o cursor de leitura da partitura (não codifica valor). **A Regra do Magenta Cenográfico.** Magenta só no orbe, no halo e na borda cônica do CTA da vitrine. Nunca em dado. **A Regra do Piso.** Todo par de cor que carrega texto cruza AA (4,5:1) por cálculo. O portão manda no vidro; o vidro nunca afrouxa o portão.

■ 8.3 TYPOGRAPHY

Display Font: Bricolage Grotesque (fallback `ui-sans-serif, system-ui`), **só na vitrine.** **Body Font:** Inter (fallback pilha de sistema), com `cv01` e `ss03`. **Label/Mono Font:** Martian Mono (fallback `ui-monospace, Cascadia Mono, Consolas`).

Todas por `next/font/google` no `app/layout.tsx`, baixadas no build e servidas do próprio deploy: zero rede em runtime.

Variáveis: `--fonte-martian`, `--fonte-inter`, `--fonte-bricolage`; tokens de uso `--fonte-mono`, `--fonte-sans`, `--fonte-display`.

Caráter: mono de instrumento para tudo que é rótulo ou número, prosa humana onde mono cansa, e um display de contraste violento apenas onde o visitante chega de passagem.

8.3.1 Hierarchy

- **Display da vitrine** (`.display-vitrine`, 500, `clamp(2.6rem, min(8vw, 10.5vh), 6.5rem)`, 0,95, -0,035em): manchete. O teto depende da altura (`min` com `vh`) para o hero não estourar em monitor largo e baixo. No hero o `h1` (`.ln-hero_titulo`) sobrepõe corpo `clamp(2.8rem, min(6.4vw, 11vh), 6rem)` e peso 700.
- **Título de seção da vitrine** (`.titulo-vitrine`, 500, `clamp(2.1rem, 4.6vw, 3.75rem)`, 1,02, -0,028em).
- **Título de tela** (`.titulo-instrumento`): mono, 500, caixa alta, -0,02em. Nunca display.
- **Rótulo** (`.rotulo-instrumento`, 0,6875rem, 0,14em, caixa alta, `muted-foreground`): campo, painel, telemetria, grupo do menu.
- **Dado** (`.num`): mono + `tabular-nums`. Todo número comparável.
- **Prosa** (Inter): transcrição, explicação, aparato; na vitrine `.ln-prosa`.

8.3.2 Named Rules

A Regra dos 11px. Nenhum texto abaixo de 11px (0,6875rem); a banca lê em projetor. `lib/tipografia.test.ts` varre TSX por `text-[]` e o `globals.css` por `font-size` literal. **A Regra da Display Contida.** Bricolage entra só via `.display-vitrine` e `.titulo-vitrine`, ambas em `globals.css`. `tipografia.test.ts` falha se `--fonte-display` ou `--fonte-bricolage` vazarem para qualquer outra regra, ou se `--fonte-sans`/`--fonte-mono` deixarem de apontar para Inter/Martian. `app/lp.css` sobrepõe só tamanho e peso, nunca família. **A Regra do Tracking Inverso.** Tipo grande fecha (-0,03 a -0,04em), rótulo pequeno abre (0,12 a 0,16em). A escala da vitrine não vale para a dashboard.

8.4 LAYOUT

Dashboard. Pilha de "sistemas" (`Painel`), não grade de cartões: quem agrupa é espaço mais régua. Cada tela abre com a telemetria de 34px, depois a **armadura horizontal** (quatro células lado a lado, largura total, mesma altura curta, separadas por régua), depois o painel dominante. Um dominante por tela (`lib/hierarquia.ts`, `DOMINANTE POR TELA`); `apoio` é o padrão para ninguém virar dominante por omissão. Padding: dominante `p-4 sm:p-6`, apoio `p-4 sm:p-5`. Nav lateral colapsável para 32px (só o número sobrevive).

Vitrine (09/10/2026). `.ln-wrap` máx. 82rem com gutter `clamp(1rem, 4vw, 3rem)`; seções com `padding-block: clamp(5rem, 11vw, 9rem)`. Ordem: hero (manchete à esquerda, orbe à direita), problema, máscara, leituras, analista, limites, fecho. Abaixo de 900px o orbe sobe para o alto e o texto ganha scrim sobre a cena.

A dobra por altura. A manchete do hero usa `clamp(3rem, min(9.2vw, 13.5vh), 8.75rem)`: a 1280×720 o CTA fica acima da dobra. No celular o orbe ocupa o topo e o hero abre com `padding-top: 74vw` para o título começar abaixo dele (orbe atrás de manchete derruba o contraste). O `body` nunca rola na horizontal (`.vt` usa `overflow-x: clip`, que não quebra o `sticky`). Mobile 390 sem corte.

Ritmo. Pauta de 13px, peso 1px, opacidade 0,03 (papel de ensaio virou ruído de instrumento). Grade de 1px da seção Sistema: `gap: 1px` sobre fundo `--linha`.

8.4.1 Named Rules

A Regra da Dobra. Toda mudança no hero se confere a 1280×720 e a 390px de largura: CTA visível, sem rolagem horizontal. **A Regra da Régua Única.** Só a linha `--linha` afirma; filete interno é `--compasso`. Não empilhe bordas.

8.5 ELEVATION & DEPTH

Plano por padrão; profundidade é opacidade e hairline. A superfície é quase opaca (fino/médio ~90–92% de `--card`, denso 97%), com `border: 1px solid var(--linha)` e uma sombra de assentamento `0 1px 0 oklch(0 0 0 / 30%)`. Sem quina iluminada, sem halo, sem chanfro. Blur só onde há sobreposição real: o denso (`blur(16px) saturate(1.2)`: popover/sheet), a faixa sticky do cabeçalho (`blur(12px)`) e o nav de pílula da vitrine (`blur(14px) saturate(1.3)`). Sem `backdrop-filter` o navegador recebe o **piso** opaco (`--vidro-*--piso`), medido por `npm run pisos`.

As classes `.vidro-fino`, `.vidro`, `.vidro-denso` e `.chanfro` **permanecem** (30 componentes as carregam) mas o Instrumento as reduz a hairline e `clip-path: none`; só o tema chuva as reativa por baixo. Pisos medidos em 30/09/2026: 0.192 / 0.191 / 0.188.

O ateliê é **calmo**: planeta, estrelas, grade laser, sol listrado e marca impressa em opacidade 0; sobra a mancha fria a 0,07 e o papel pautado. A mancha dourada foi zerada na revisão final porque esquentava cabeçalho e sidebar; **o dourado é ação, foco e marca, não luz de ambiente.**

8.5.1 Named Rules

A Regra da Espessura como Opacidade. Vidro engrossa por opacidade, nunca por luz baixa. Par que reprova no `contraste` faz a espessura ganhar opacidade.

8.6 SHAPES

Raio zero (`--radius: 0`): retângulo de hairline. O chanfro de 45° do F e do R saiu junto com o vidro. Exceções explícitas, sempre com `rounded-full`: a cabeça da categoria, o ponto de status, o nav em pílula da vitrine (999px), o orbe (50%). Botão da vitrine é retangular. Marcadores de lâmpada são quadrados de 0,5rem com brilho.

8.7 COMPONENTS

8.7.1 SegmentoLED (peça-assinatura)

`components/instrumento/SegmentoLED.tsx` + `lib/segmentos.ts` (parte pura, testável sem React). - **Alfabeto:** `0-9`, `-` (e o menos tipográfico U+2212, mesmo sinal), `+` (segmento do meio + traço vertical), `□`, `◻`, `◼`, espaço. Caractere fora do alfabeto **não some**: entra como texto cru em mono (um display que engole caractere muda o número em silêncio). - **Vírgula com cauda:** a vírgula do pt-BR é um ponto com cauda descendo à esquerda; o ponto (`.`) é célula própria, só círculo. Sem a cauda, `66,1` lia como `66.1` (milhar em pt-BR). - **null** é **apagado**: renderiza N células (padrão 3) com os sete segmentos a 10% de opacidade, nunca `0`. `aria-label` vira ``${rotulo}: sem sinal`. -

rotulo **obrigatório**: SVG de polígonos não diz nada a leitor de tela, e "2,9" de quê não diz nada a quem vê. - **Não calcula, não arredonda, não converte**. Recebe a string já formatada em pt-BR (`formatarNps`, `formatarSegundosLED`,...). Uma fonte só de formatação (invariante 3). - **Cor**: `medido` (azul, padrão), `marca` (dourado, fatos da marca), `tinta` (foreground, observado e não inferido), `erro` (só o easter egg). Brilho: um `drop-shadow(0 0 3px currentColor)` no grupo aceso, não por polígono. Sem estado, serve em Server Component.

8.7.2 TelemetriaTopo

Faixa de 34px no topo de cada tela: mono 11px, caixa alta, 0,14em, `border-b border-linha`, fundo `--background`. Carrega **só fatos verdadeiros do produto** ("NPS ESTIMATIVA", "LLM 00"), nunca métrica inventada. `TelemetriaValor` põe o valor em dourado (marca, não dado). O lado direito some abaixo de `sm`. `TelemetriaDaRota` escolhe pela rota.

8.7.3 Navegação numerada

Lateral: mono 11px, caixa alta, filete inferior, sem raio. Item `01 VISÃO GERAL` com o **cursor** `█` **dourado** na tela ativa; ativa também leva `aria-current="page"` (cor nunca é o único canal). A numeração vem de `lib/navegacao.ts`, a mesma fonte da telemetria; a ordem é a da tela e não a do papel (`usuario` vê 01 e 02, que continuam sendo 01 e 02). Grupos: Seções (5 de olhar), Referência (docs, aparece só com `NEXT_PUBLIC_URL_DOCS`, ícone de saída, aba nova), Ajustes (só `dev`). Rodapé: "Modelos locais / Sem LLM em runtime". Cada item carrega o período na URL.

8.7.4 Painel (hairline)

Sistema de largura total: cabeçalho com `rotulo-instrumento` e `border-b border-linha`; corpo; prosa metodológica sobe para o **aparelho da tela** (`RessalvaDaTela`), recolhível mas nunca removida. Rótulos curtos (`estimativa`, `observado`, `sem sinal`) ficam colados ao dado. Props `erro` e `vazio` substituem o conteúdo no lugar dele; `regra` divide dito/medido. Entrada: sobe 8px uma vez ao entrar em cena (`whileInView` + `once`).

8.7.5 Armadura de indicadores

`FaixaIndicadores`: quatro células em fileira, rótulo + LED, sem barra de progresso decorativa. NPS e CSAT levam "estimativa" e o sublinhado pontilhado (`.estimado`); contenção e latência são observadas e não levam. Cada célula falha sozinha.

8.7.6 Cabeça da categoria (`EtiquetaCategoria`, slot único)

Três formas, nesta ordem de honestidade, num só componente que tabela, piores atendimentos, ficha do grafo e tela do atendimento atravessam: - **cheia**: medido, com evidência, ponto colorido. - **tracejada**: medido, evidência **fraca**, anel tracejado **na cor da classe**; texto por extenso "evidência fraca: motivo". Vem do servidor (`fraus/evidencia.py`), derivada de fala observável (quantas mensagens e palavras), **nunca** da probabilidade do modelo: probabilidade não calibrada não é confiança. - **vazada**: não medido, anel oco cinza (`CabecaVazada`), nunca quarto ponto cheio; escreve "sem sinal" por extenso. Um componente irmão daria três formas em quatro lugares, e o que ninguém lembrasse de trocar continuaria mentindo confiança.

8.7.7 Vitrine — a LP (/, ln-*)

Nasceu em `/leitura` como experimento `vgpu` em 09/10/2026, recebeu a constelação da vitrine antiga no mesmo dia e virou a raiz quando o dono pediu de volta o fósforo e o orbe (09/10/2026). A vitrine anterior (`components/lp/` com WebGL à mão, `lib/vitrine/`, `vitrine.css`, prefixo `vt-`) foi **removida**; `/leitura` redireciona para `/` (`permanent: false`). Dela ficaram `components/lp/fatos.ts` (os números) e as peças abaixo. Mesmo mundo (Instrumento), sem as amarras da ferramenta. - **Fundo e orbe da vitrine antiga, em WGSL** (`lib/lp-nova/shaders/orbe.wgsl`), porte do antigo `FS FUNDO`): fósforo de osciloscópio — retícula, traço âmbar do dito e azul do medido com varredura, curvatura de CRT, vinheta, scanline, grão — e o **orbe de três manchas** (âmbar, azul, magenta, deformadas por fbm, com halo). O orbe é um objeto só que atravessa a página (`lib/lp-nova/orbe.ts`, parte pura e testada): grande no hero, **pequeno no canto de cima enquanto a máscara se forma** (o rosto precisa ler), pequeno no centro nos nós, fusor azul, leitura, grande e âmbar no fecho. **A cor é canal**: âmbar (dito) domina na entrada da conversa e no fecho, azul (medido) no fusor, magenta só oscila. Na leitura **sem sinal**, saturação 0, manchas paradas e o traço do dito vira linha de base tracejada: ausência fora da escala. - **Cartões minimalistas** (pedido do dono): nada de caixa com fundo, chip ou painel com cabeçalho na vitrine. Fala é citação tipográfica com rótulo mono; o Sistema é lista de linhas com hairline. - **LED fino**: `SegmentoLED traco="fino"` — mesma célula e alfabeto, segmento de 4 em vez de 12, sem brilho, apagado a 8%. Só na vitrine; a ferramenta segue no traço cheio. - **Cena** (`components/lp-nova/CenaLeitura.tsx` + `lib/lp-nova/cena.ts`): UM canvas **WebGPU** via `vgpu` 0.5.0. Um compute shader avança as partículas num storage buffer e um `draw` instanciado lê esse buffer; a CPU não toca posição depois da semente. WGSL modular em `lib/lp-nova/shaders/` (loader `@vgpu/wgsl` no Turbopack), validado por `npm run wgsl` (`vgpu check --require-validation`). - **A história é a leitura, com a constelação da vitrine antiga dentro** (mistura aprovada pelo dono em 09/10/2026): a fala `ok, obrigado 😊` é rasterizada e vira as partículas âmbar (o texto do DOM fica transparente nos mesmos pixels), dispersa ao lado do texto, forma a **máscara** da vitrine antiga (oval com olhos e boca vazios, âmbar à esquerda e azul à direita), que se desfaz em **39 nós, um por feature, agrupados na contagem real** (`lib/lp-nova/constelacao.ts`, geometria portada do motor WebGL antigo), e os nós caem no orbe e **saem azuis**. A seção "Toda conversa tem uma máscara" conta a MESMA conversa do hero e mostra a nota GRAVADA dela com a fala que a puxou; o "2,9" digitado à mão da vitrine antiga não veio junto. Os passos do texto acendem pela fase dominante (`data-fase` na raiz). Na seção de leituras o campo toma o humor da **categoria do servidor** (detrator agita, promotor assenta, sem sinal fica cinza e parado). Coreografia pura e testada em `lib/lp-nova/fases.ts`; cada seção marca seu momento com `data-marco`. - **Leituras gravadas, nunca ao vivo**: cinco conversas sintéticas (`dados/lp/*.csv`) lidas pelo motor de produção via `scripts/gravar leituras lp.py` (rota que não grava), com procedência (data, API, `fusor:<sha das importâncias>`). Sem o JSON, a seção mostra o estado vazio que nomeia o script. Marcas por `marcasDaAtribuicao`, a mesma da dashboard. - **Três regimes, sempre escritos** num rótulo mono fixo: `GPU · alto`, `GPU · baixo · quadros lentos` (desce **uma vez**, nunca sobe) e `pôster · <motivo>` (sem WebGPU, GPU recusou, movimento reduzido). O pôster sai do **mesmo WGSL** (`npm run posteres`, Node + Dawn, duas renderizações comparadas byte a byte) e vira fundo de hero, fusor e fecho, com fósforo e orbe dentro. - **Modo de compatibilidade**: workgroup 64 e storage buffer no vértice. Aparelho sem esse limite falha no `draw`, e a página cai no pôster com `falha-gpu`, sem erro de console. - `scripts/validar-lp-playwright.mjs` (`npm run test:lp`) cobra em `/` um canvas, regime vivo e pôster, 390 sem rolagem horizontal, CTA na dobra a 1280×720, saída para `/entrar` sem `pageerror`, device perdido caindo no pôster e o redirect de `/leitura`. - Tamanhos de mono (0,625–0,8125rem) e de prosa curta (0,9375–1,1875rem) fora da escala acima: os mesmos degraus que a vitrine já usava, registrados aqui em vez de canonizados.

8.7.8 Ícone de ação

`components/ÍconeDeAcao.tsx`. **Um verbo, um desenho:** todo botão de ação leva o ícone do VERBO (salvar, cancelar, remover, copiar, criar, atualizar, executar...), tirado de um mapa único, à esquerda do texto; só "próxima" o leva à direita. "Salvar faixas" e "Salvar equipe" têm o mesmo desenho. Controle de **seleção** (perfil, ordem de data, linha clicável de lista) não leva ícone de verbo: ele escolhe, não faz, e o estado já é dito por `aria-pressed`. O ícone é sempre decorativo (`aria-hidden`); o nome do botão é o texto. **Traço:** os ícones vêm do lucide e são redesenhados por CSS (`.lucide` em `globals.css`): 1,5px, ponta reta, junta viva. Ponta redonda a 2px é desenho de app amigável sobre uma superfície de hairline e raio zero. A regra cobre todo ícone da ferramenta; a vitrine não usa lucide.

8.7.9 Estados (coreografia)

Todo componente interativo tem padrão, hover, foco, ativo, desabilitado, carregando, erro e vazio. - **Carregando** = esqueleto com a **forma** do resultado (`app/dashboard/loading.tsx`: armadura de quatro células, gráfico, lista), `aria-busy`. **Nunca desenha LED apagado:** apagado quer dizer "o medidor leu e não há sinal", e aqui ele ainda não leu. Nunca roda girando. - **Vazio** (`EstadoVazio`) nomeia o que falta e o endpoint ou a etapa que resolveria (`Resolvido por ...`, `Falta rodar: ...`). Moldura tracejada. Nunca preenche com número simulado. Com API fora, fala menos: a ação está na faixa do topo e a prosa recolhe para aparato. - **Erro isolado:** o sistema que falha mostra o próprio erro no lugar dele, com o peso dele; os vizinhos continuam de pé. - **Sem sinal:** LED apagado ou cabeça vazada. Nunca zero, nunca cinza dentro da escala. - **Foco:** anel de 2px `--ring` (dourado) em `:focus-visible`, também para `summary` e links.

8.7.10 Easter egg da mentira (`lib/mentira.tsx`)

Com o painel do easter egg aberto, os indicadores da visão geral exibem números **falsos**: violação deliberada e **encenada** da invariante 2, sustentada por quatro travas: (1) nada sai da memória (zero escrita, zero API; estado de render); (2) leitor de tela recebe a verdade (o falso é `aria-hidden` e um `sr-only` diz que foi falsificado e qual o real); (3) selo `--destructive` inescapável no mesmo quadro; (4) volta sozinha em 8s, `Esc` ou clique fora. Gráficos e tabela **não** mentem. O falso é determinístico (`(d*7+3)%10` só nos Algarismos, sem `Math.random`) e o LED vai a `cor="erro"`. Qualquer variação que quebre uma trava não é o easter egg, é defeito.

8.7.11 Motion

Vocabulário em `lib/movimento.ts` (150–250 ms, curva `--ease-fluid` `cubic-bezier(0.32, 0.72, 0, 1)`). Na **ferramenta**, dois momentos autorados: o cursor de leitura na linha do tempo e a entrada dos sistemas (8px, uma vez, escalonada em 40 ms na armadura). Resposta a gesto (hover, a marca `MarcaFraus` que responde a cinco cliques) é outra categoria e não conta.

A exceção da vitrine (cena de GPU). Desde 01/10/2026 a vitrine tem um canvas com laço de quadros — decisão do dono, que inverte a de 30/09 (orbe em CSS puro, sem canvas); desde 09/10/2026 ele é WebGPU (vgpu). A regra que sobrevive é onde ele mora: **só na vitrine** (`app/page.tsx`), nunca no layout raiz, porque a ferramenta é lida por horas e não pode ter GPU girando atrás de tabela e gráfico. O device é liberado ao desmontar; sem WebGPU, a página mostra o pôster e segue legível. `scripts/validar-lp-playwright.mjs` cobra exatamente um canvas e o regime certo. `prefers-reduced-motion` **congela, não desacelera:** na vitrine a cena entra no regime `pôster` — nenhum laço, imagem fixa renderizada do mesmo WGSL. Na ferramenta, o bloco global zera transições e o `<Movimento>` passa `reducedMotion="user"` ao Motion.

■ 8.8 DO'S AND DON'TS

8.8.1 Do:

- **Do** passar ao `SegmentoLED` a string já formatada em pt-BR e sempre um `rotulo`; `valor={null}` para ausência.
- **Do** manter âmbar acima e azul abaixo; use `-texto` quando a cor carrega tipo.
- **Do** manter o dourado em ação, foco, cursor e marca, e o magenta no orbe e no halo.
- **Do** rodar `npm run contraste` e `npm run pisos` ao tocar token de cor, vidro ou ateliê, e `npm run build` (ele compila o CSS e processa assets; os outros gates não).
- **Do** conferir trabalho visual olhando, a 1280×720 e a 390px, nos dois temas.
- **Do** rotular amostras sintéticas como tal em todo lugar em que aparecem.
- **Do** nomear o que falta em estado vazio, com a etapa ou o endpoint.
- **Do** derivar score, nota e categoria no servidor; o LED apresenta, não calcula.

8.8.2 Don't:

- **Don't** renderizar `0` para ausência; `null` é segmento apagado e "sem sinal" por extenso.
- **Don't** usar kickers/eyebrows acima de título na vitrine (removidos na revisão final).
- **Don't** usar dourado em série, barra, categoria ou célula de dado, nem como luz de ambiente.
- **Don't** usar Bricolage fora de `.display-vitrine`/`.titulo-vitrine`, nem display em rótulo ou dado.
- **Don't** pôr orbe, halo, grade ou coreografia atrás de tabela, gráfico ou qualquer rota do Operate.
- **Don't** escrever texto abaixo de 11px.
- **Don't** empilhar bordas, reintroduzir chanfro, raio ou quina iluminada no tema Instrumento.
- **Don't** reciclar matiz de família nem reordenar/retocar as sete emoções sem rodar o validador.
- **Don't** pontuar atendentes: nenhum elemento de interface pode transformar reconhecimento de emoção em score de performance por pessoa (CLAUDE.md, EU AI Act).
- **Don't** pôr comentário ou regra que cite `.tema-chuva` antes do bloco real dele: o gate recorta pela primeira ocorrência da string e mede o bloco errado (ver Temas e gates).

■ 8.9 TEMAS E GATES

Dois temas, ambos escuros. O padrão é o Instrumento (`:root, .dark`), o app renderiza sempre com `.dark`); a segunda opção é **chuva de neon** (`.tema-chuva`), que sobrepõe só o chassi: asfalto índigo, dourado de sódio (`[0.81 0.1 82]`), quina de neon ciano/magenta, atelier aceso e vidro mais grosso. Ela **não toca a camada de dado**: `--dito` continua âmbar, `--medido` azul, famílias intactas; o dourado segue fora de dado; todo par de texto cruza AA nos dois temas.

Gate ciente de tema. `scripts/contraste.mjs` recorta o bloco base (`:root, .dark { }`) e cada bloco cujo seletor começa com `.tema-`, sobrepõe à base como a cascata (um tema só declara o que muda) e roda a matriz inteira por tema. Um mapa único deixaria o segundo tema sobrescrever o primeiro e o relatório sairia verde medindo um tema só.

`scripts/pisos.mjs` mede o pior caso de vidro sobre o ateliê (só lê `oklch()` literal, por isso os pisos são pré-calculados, não `color-mix`). **O cuidado com `.tema-chuva`**. O laço que descobre temas usa `^(\\tema-...)\s*\{` (ancorado no início de linha), mas `recortarBloco` localiza o bloco por `indexOf` do seletor **em qualquer ponto do arquivo**, e depois abre na primeira `{` seguinte. Logo, qualquer comentário ou regra anterior ao bloco real que escreva a string `.tema-chuva` (por exemplo `.tema-chuva .vidro-fino`) faz o gate recortar o trecho errado e medir o bloco errado, com relatório verde. Em `globals.css` o bloco real vem antes de toda outra menção e o comentário que o antecede fala do tema sem escrever o seletor. Mantenha assim.

Ciclo de verificação: `pisos`, `contraste`, `tsc --noEmit`, `lint` e `build` não se substituem, e nenhum vê a tela. Quatro já devolveram verde com o build quebrado.

■ 8.10 ADIÇÕES E DESVIOS CONSCIENTES DESTA RODADA

- **Revisão final (30/09/2026):** kickers removidos da LP; dourado só em ação e marca (mancha dourada do ateliê zerada, LED da faixa de fatos em `tinta`); hero a 1280×720 com CTA na dobra (orbe por altura); mobile 390 sem corte nem rolagem horizontal; armadura da visão geral em faixa curta com leitura do NPS em faixa própria; grafo com respiro e reenquadramento ao redimensionar.
- **`.lp-*` no `globals.css`** é resíduo de duas vitrines atrás; a LP de hoje vive em `app/lp.css` com prefixo `In-` justamente para não herdar hover de peça extinta.
- **Nove tipos de nó do grafo não foram alterados nesta rodada, mesmo com o revisor pedindo.** Os matizes estão congelados: a família (quente/fria) já diz dito/medido de longe, o matiz diz o tipo de perto, a legenda agrupa os nove, e claridade/croma iguais impedem hierarquia accidental. Mexer nas cores em rodada de chassi, sem validar de novo legenda, contraste e leitura no canvas, trocaria uma decisão medida por gosto. Reabra só como decisão própria, com validação.
- **Vidro que não é mais vidro:** `--vidro-*-fundo`, `--vidro-quina`, `.chanfro`, `.especular` ficam como vocabulário porque o tema chuva os usa. Em `lib/hierarquia.ts`, a diferença entre `vidro` e `vidro-fino` é hoje só o padding e o tipo; não leia como duas opacidades.
- **Não canonizado (defeitos carregados, não regras):** comentários do `globals.css` ainda citam croma "0.085"/"0.15" para dourado/âmbar (o token vigente é 0.095/0.14) e "DESIGN.md seção 4" numerada como no arquivo antigo. Os ícones `lucide-react` deixaram de ser defeito carregado em 02/10/2026: ganharam traço próprio e regra de uso (ver "Ícone de ação"); o glifo  do cursor continua sendo texto, e continua sendo a peça.

9. Conformidade regulatória — o Fraus e o EU AI Act

Escrito em 10/09/2026 e corrigido em 28/09/2026. Este documento existe porque o Fraus **classifica emoção a partir de texto**. Ele descreve o escopo, o que o sistema infere, o que ele nunca decide sozinho e — a parte que virou regra de código — **o que ele está proibido de fazer**.

9.1 1. POR QUE ESTE DOCUMENTO EXISTE

`fraus/sinais/emocao.py` classifica **sete emoções** (as de Ekman) e deriva **desprezo** da díade raiva + nojo. O Regulamento (UE) 2024/1689 define “sistema de reconhecimento de emoções” como inferência baseada em **dados biométricos**. O Fraus recebe texto de chat, não imagem, voz ou outro dado biométrico; portanto não deve ser apresentado como enquadrado automaticamente nessa definição. Ainda há obrigações de proteção de dados, transparência e limitação de finalidade, que dependem do contexto concreto de implantação.

Esta documentação é orientação técnica, não parecer jurídico. O enquadramento de uma implantação comercial deve ser revisto por profissional qualificado.

9.2 2. ESCOPO DO SISTEMA

O que analisa	transcrições de atendimento por chatbot, já encerradas
Sobre quem infere	o cliente — e apenas o cliente
O que infere	satisfação (score 0-100 → nota 0-10 → categoria NPS), emoção por mensagem, ironia por mensagem, indicadores agregados
Quando roda	em lote, sobre conversa terminada. Nunca em tempo real durante o atendimento
O que decide sozinho	nada
Base do modelo	três BERTimbau fine-tunados + regressão logística, inferência local em CPU

9.3 3. AS TRÊS OBRIGAÇÕES QUE JÁ VALEM, E A QUE VEM

9.3.1 3.1 Transparência como compromisso do produto

Mesmo sem afirmar o enquadramento no Art. 50(3), inferência sobre emoção não deve ser escondida de quem é afetado nem de quem interpreta o resultado.

Como o Fraus cumpre: toda tela onde a leitura de emoção aparece (`components/CabecasDeLeitura.tsx`), compartilhado pelo simulador e pela análise) carrega a etiqueta de exposição, em linguagem de gente, junto da nota metodológica que já qualificava o NPS como inferido. A obrigação de *informar o cliente final* é de quem opera o chatbot — o Fraus fornece o texto e diz que ela existe; ele não fala com o cliente.

9.3.2 3.2 Limite de finalidade: não pontuar trabalhadores

O Art. 5(1)(f) proíbe determinados sistemas biométricos de inferência de emoção no trabalho. O Fraus textual não deve reivindicar que essa proibição específica se aplica automaticamente a ele. Ainda assim, pontuar atendentes seria uma expansão de finalidade de alto impacto e permanece proibida por política do produto.

9.3.3 Regra: o Fraus não pontua atendentes.

O atendente humano aparece no transcript — a escalação é parte do dado — e o classificador de texto **recusa pontuar mensagem que não seja do cliente** (`sinais/texto.py`; `qtd humano` e `latencia_mediana_humanos` são contagem e relógio, nunca julgamento). "Score de performance por atendente" é a sugestão mais recorrente em analytics de atendimento, e no Fraus ela está **fechada por desenho**, não esquecida no backlog.

Vale com o mesmo peso das invariantes do `CLAUDE.md`: implementar isso não é feature nova, é violação.

9.3.4 3.3 Controles recomendados para uma implantação comercial

Não se afirma aqui que o Fraus textual seja automaticamente “alto risco” pelo Anexo III. Os controles abaixo continuam recomendados porque tratam dados de atendimento e inferências sobre pessoas.

Onde o Fraus já está, hoje, em relação a cada uma:

OBRIGAÇÃO	ESTADO NO FRAUS
Supervisão humana	o sistema não decide nada. O analista curou o léxico (<code>[sinais/curadoria.py]</code>), move as faixas de NPS (<code>[PUT /configuracoes]</code>) e lê os transcritos. Nenhuma saída aciona ação automática sobre ninguém
Logging	<code>[entregas_webhook]</code> registra toda entrega e toda recusa, com veredito e motivo. É o embrião da trilha de auditoria
Transparência	etiqueta de estimativa em todo número inferido; limitações declaradas em tela e em <code>[docs/treinamento.md]</code>
Monitoramento pós-mercado	<code>[GET /saude/deriva]</code> reporta quais features estão fora da faixa de distribuição em que o modelo foi treinado
Avaliação de conformidade	não feita. É trabalho de quem for colocar isto em produção comercial, e está fora do escopo do TCC
Avaliação de impacto em direitos fundamentais	não feita, pelo mesmo motivo

Isto não é uma declaração de conformidade. É um mapeamento honesto do que já está de pé e do que não está.

■ 9.4 4. LIMITAÇÕES QUE IMPORTAM PARA QUEM LÊ UM NÚMERO DO FRAUS

- **O NPS é inferido do texto, não perguntado.** O intervalo de confiança que a interface mostra cobre a **incerteza amostral** — não a incerteza do modelo.
- **A cabeça de ironia é pouco confiável** e por isso **não pontua** desde 04/09/2026: no corpus de treino ela funciona como detector de sentimento positivo. Continua sendo exibida, com a ressalva colada.
- **O sinal de tempo é treinado em dados sintéticos**, calibrados por literatura, porque nenhum corpus público em português tem timestamps de diálogo. Fora da faixa de latência do treino, o relógio domina a nota — é limitação medida e declarada, e `[GET /saude/deriva]` a torna visível.
- **Desprezo não é classe treinada;** é derivado da díade raiva + nojo.
- **Ausência de dado nunca vira zero.** Conversa sem fala do cliente sai "sem sinal", em toda superfície.

■ 9.5 5. FONTES

- [Regulamento \(UE\) 2024/1689 — texto oficial](#)
- [Art. 50 — obrigações de transparência](#)
- [Diretrizes de transparência da Comissão Europeia](#)
- [Obrigações de transparência em vigor desde 02/08/2026 — Cooley](#)

- [Emotion AI de cliente vira alto risco — CX Today](#)

10. Encolher os modelos — o que foi medido, e por que a int8 foi recusada

Medido em 10/09/2026, na máquina de desenvolvimento, contra os checkpoints reais. Nada aqui exigiu retreino: quantização dinâmica é transformação **pós-treino** sobre os pesos já aprendidos.

10.1 O PROBLEMA

3 BERTimbau (fp32)	$3 \times 416 \text{ MB} = \mathbf{1.249 \text{ MB}}$
<code>torch</code> que os executa	497 MB
imagem de inferência	~1,75 GB

Isso limita hospedagem, encarece o container e — desde que o teto de função Python da Vercel subiu para 500 MB (24/02/2026) — é a única coisa entre o Fraus e um deploy serverless.

10.2 O QUE FOI TESTADO

Quantização dinâmica int8, sobre `bertimbau-satisfacao`, com as mesmas dez sondas em todas as variantes:

ESTRATÉGIA	TAMANHO	MAX ΔP	CLASSE MUDOU
qint8 em tudo	105,5 MB	0,4535	1 sonda
qint8 só em MatMul	172,0 MB	0,4953	1 sonda
qint8 MatMul + <code>per_channel</code>	172,4 MB	0,1385	nenhuma
qint8 só em MatMul	172,0 MB	0,2620	1 sonda
qint8 em tudo + <code>per_channel</code>	105,9 MB	0,1015	nenhuma
qint8 tudo + <code>per_channel</code> + <code>reduce_range</code>	105,9 MB	0,1798	nenhuma

Duas coisas valem ser registradas porque contrariam a intuição comum:

- 1. `per_channel` **é o que decide**, não a escolha de quais operadores quantizar. Ele dá a cada coluna de peso a própria faixa, em vez de espremer a matriz inteira numa só — e numa camada de atenção as colunas têm magnitudes muito diferentes.
- 2. **"não quantize o embedding, só MatMul" saiu pior nas duas pontas**: 63% maior e menos fiel. Foi medido, não herdado de tutorial.

10.3 O RESULTADO DA MELHOR VARIANTE

Aplicada aos três modelos:

modelo	antes	depois	redução	max Δp	classe mudou
satisfacao	416.2M	105.9M	3.93x	0.1015	nenhuma
emocao	416.2M	105.9M	3.93x	0.1129	nenhuma
ironia	416.2M	105.8M	3.93x	0.1486	nenhuma
TOTAL	1248.6M	317.6M	3.93x		

Velocidade: **2,10x mais rápido** em CPU (309 ms → 147 ms por lote de 10 frases, `bertimbau-satisfacao`).

Pela medida de classificador — nenhuma sonda mudou de classe vencedora — isso passaria.

10.4 E É AQUI QUE ELA É RECUSADA

Diferença de probabilidade é a medida errada para este sistema. O score do Fraus é uma *projeção* das três probabilidades num eixo 0–100; a nota é o score arredondado; a categoria é a nota caindo numa faixa. Cada degrau absorve ou **amplifica** o erro do anterior.

Medido ponta a ponta, com o motor inteiro montado dos dois jeitos, sobre 180 conversas determinísticas do simulador:

desvio médio de score	0,83 pontos
desvio máximo	25,65 pontos
notas diferentes	16 de 180 (8,9%)
categorias diferentes	3 de 180 (1,7%)

E o caso que encerra a discussão:

sim-0-635276501	59,95	->	85,61	detrator	->	promotor
-----------------	-------	----	-------	----------	----	----------

Uma conversa atravessa **duas faixas** e troca de veredito. Categoria é o que a tela mostra e o que o NPS agrega: trocar o backend não pode trocar o veredito.

Decisão: os artefatos int8 NÃO são promovidos. Eles ficam em `modelos-onnx/` (fora do git) para quem quiser reproduzir a medição, e o portão de `scripts/comparar_backends.py` continua vermelho de propósito.

■ 10.5 A VARIANTE MISTA, MEDIDA EM 11/09/2026

A tabela de caminhos abaixo listava "fp32 na satisfação, int8 no resto" como **estimativa**, nunca medida. A hipótese era que o flip vinha de quantizar o modelo que *governa* o score, e que poupá-lo resolveria. Foi medida, com o mesmo portão e as mesmas 180 conversas:

	INT8 NOS TRÊS	MISTA
desvio médio de score	0,83	0,356
desvio máximo	25,65	18,99
notas diferentes	16 de 180	6 de 180
categorias diferentes	3 de 180	2 de 180

A hipótese estava metade certa, e metade não basta. Poupar a satisfação corta o dano quase pela metade — e ainda troca o veredito de duas conversas. A mesma `sim-0-635276501` do caso original segue atravessando faixa (59,95 → 78,94).

10.5.1 De quem é a culpa, então

A ironia **saiu do vetor do Fusor** em 04/09/2026 (ver `NOMES_FEATURES`), então quantizá-la não pode mover score nenhum. Isso deixa apenas a emoção como suspeita, e a medição confirma — satisfação fp32 + emoção fp32 + **ironia int8**:

desvio medio de score : 0.000 pontos desvio MAXIMO : 0.000 pontos notas diferentes : 0 de 180 CATEGORIAS diferentes : 0 de 180 ACEITO

Zero exato, nas 180. Duas coisas ficam provadas de uma vez:

1. **ONNX fp32 é troca de executor, não de modelo.** Δp máximo `0,0000` por modelo, desvio `0,000` ponta a ponta. O que a conversão custa é nada.
2. **A única cabeça quantizável de graça é a ironia**, e exatamente porque ela não pontua. As duas que pontuam não toleram int8.

10.5.2 E o que isso faz com a máquina de 1 GB

Nada de bom. O `VM.Standard.E2.1.Micro` (o único shape com capacidade em sa-saopaulo-1 em 11/09/2026) tem 1 GB de RAM, e o sistema operacional come ~200:

VARIANTE	PESOS	VEREDITO	CABE EM ~800 MB
ONNX fp32 nos três	1.250 MB	✓ idêntico	✗
fp32 + fp32 + ironia int8	939 MB	✓ desvio 0,000	✗
fp32 satisfação + int8 nas outras	628 MB	✗ 2/180 trocam	⚠ no limiar
int8 nos três	318 MB	✗ 3/180 trocam	✓

A ordem é cruelmente monótona: **tudo que cabe troca veredito, e tudo que preserva o veredito não cabe**. Não há ajuste no meio — a emoção precisa ficar em fp32, e aí são dois codificadores de 416 MB, que já estouram o orçamento sozinhos.

⚠ MEDIÇÃO DE RAM PENDENTE, E POR QUE O NÚMERO PRELIMINAR NÃO SERVE

Medindo o processo com o motor montado, o ONNX consumiu **mais** residente que o torch (pico 1.366 MB contra 1.075 MB): o `onnxruntime` carrega o grafo inteiro de imediato, enquanto o torch mapeia os `safetensors` e só paga as páginas que toca.

Esse número **não está limpo**: a sonda importava `fraus.motor` e `fraus.fusor` antes de escolher o backend, e são eles que arrastam torch e sklearn — 354 MB de base nos dois casos. Medir o caminho ONNX de verdade exige uma venv **sem** torch, e hoje `torch>=2.3` é dependência **principal** no `pyproject.toml`: a imagem sem torch que o extra `onnx` promete não existe sem mover essa dependência para um extra.

A conclusão acima não depende disso — 939 MB de peso já passam de 800 —, mas a promessa de "ONNX corta a imagem em 2,6x" continua **não verificada em RAM**, só em disco.

10.5.3 O que sobra de ganho real

Independente da máquina pequena, a variante aceita **é um ganho para o deploy na A1**: desvio `0,000`, e a imagem perde os 497 MB do `torch` em disco (depois de mover a dependência). Na sessão de 11/09/2026 o envio do código e dos pesos para a VM era de 1,3 GB — cortar isso quase pela metade é menos tempo de deploy e menos coisa para dar errado no meio.

E se 1 GB algum dia virar requisito de verdade, o caminho não é quantização: **é um codificador com três cabeças**. Os três checkpoints são ajustes finos do mesmo BERTimbau, então os 110M de parâmetros estão duplicados três vezes. Um encoder compartilhado com três cabeças de classificação daria ~416 MB em precisão cheia — cabe em 1 GB sem quantizar nada e sem risco de veredito. O preço é retreino, e portanto a invariante 10 de novo; não é ajuste de deploy.

10.6 O BACKEND ONNX EM PRODUÇÃO, MEDIDO EM 15/09/2026

A variante aceita acima (satisfação e emoção fp32, ironia int8) virou um executor de verdade:

`FRAUS_BACKEND=onnx`. O padrão continua `torch`, e não há detecção nem fallback — backend desconhecido ou grafo ausente derruba o boot (invariante 7). O `torch` saiu das dependências principais para o extra `torch`, e o import dele ficou preguiçoso: `fraus.motor` não o arrasta mais (há teste para isso).

Isso destravou a **medição de RAM que estava pendente**, agora numa venv sem torch instalado (`uv sync --extra dev --extra onnx`), com o app montado por `criar_app_padrao` e 30 conversas atribuídas:

EXECUTOR	PICO DE RSS	180 CONVERSAS (<code>ATRIBUIR</code>)	CATEGORIAS TROCADAS
torch	1.848 MB	53,7 s	referência
ONNX, 1 thread (config antiga)	—	45,2 s	0
ONNX, todos os núcleos	1.445 MB	35,0 s	0
ONNX, todos os núcleos, sem arena	1.347 MB	33,2 s	0

Desvio máximo de score contra o torch: `6,5e-5`. A venv sem torch ocupa 430 MB contra 5,0 GB, e a imagem Docker `--build-arg BACKEND=onnx` mede **995 MB**, já com o motor real respondendo `/analisar`.

Duas coisas contrariam a intuição e foram medidas, não supostas:

1. **A fusão offline `-O2` do otimizador de transformers saiu mais lenta** (41,2 s contra 35,0 s), embora tenha fundido tudo (12 `Attention`, 24 `SkipLayerNormalization`, 12 `BiasGelu`, GELU exata). A sessão já roda com `ORT_ENABLE_ALL`, que funde na carga; o grafo pré-fundido só perde otimizações que o runtime faria por conta própria. **Recusada.**
2. **`intra_op_num_threads = 1` era o gargalo.** Fazia sentido quando várias requisições podiam entrar no modelo juntas. Agora o `Motor` deixa uma passada por vez (`FRAUS_INFERENCIAS_SIMULTANEAS`), e a passada usa os núcleos (`FRAUS_ONNX_THREADS`, padrão 0 = o runtime escolhe).

10.6.1 Emoção com int8 só nos pesos — aceita pelo portão, não promovida por padrão

A int8 recusada acima é **dinâmica**: quantiza pesos e computa as ativações em int8. A variante `--precisao int8-pesos` (`MatMulNBits`, 8 bits, blocos de 32, simétrica) guarda os pesos em int8 e mantém a ativação em fp32. Aplicada só à emoção, com satisfação fp32 e ironia int8, nas mesmas 180 conversas:

	ONNX ACEITO (EMOÇÃO FP32)	EMOÇÃO INT8-PESOS
peso da emoção	417 MB	184 MB
pico de RSS da API	1.347 MB	1.056 MB
180 conversas (<code>atribuir</code>)	33,2 s	49,6 s
desvio máximo de score	0,0001	0,51 ponto
notas / categorias trocadas	0 / 0	0 / 0

Ela passa no portão (zero categorias) e no do script ($\Delta p 0,0148 < 0,02$). Mas, ao contrário do fp32, **não é o mesmo número**: meio ponto de score é margem que uma conversa encostada na fronteira 6/7 ou 8/9 pode atravessar num corpus maior que as 180. Por isso ela fica disponível e documentada, **não** como o padrão de `modelos-onnx/`. E 1.056 MB **ainda não cabe** numa máquina de 1 GB, então ela não resolve o caso que justificaria o risco.

A conclusão sobre o encoder compartilhado abaixo não mudou.

10.7 O QUE SOBRA COMO CAMINHO

CAMINHO	TAMANHO ESTIMADO	FIDELIDADE	CABE EM 500 MB?
ONNX fp16	3 × 208 = 624 MB + ~50 MB de runtime	quase idêntica	× — mas corta a imagem de 1,75 GB para ~0,67 GB
fp32 na satisfação, int8 no resto	416 + 106 + 106 = 628 MB	boa no que pontua	×
int8 nos três	318 MB + ~50 MB	1,7% troca de veredito	✓

Ou seja: **o orçamento de 500 MB só é alcançável pagando o preço que foi recusado acima**. Isso não é um impasse técnico a resolver com mais tuning — é a Vercel serverless não sendo o alvo certo para este sistema, e vale mais dizer isso do que espremer o número até ele caber.

O ganho continua real fora da Vercel: ONNX fp16 num container corta a imagem em ~2,6× e remove o `torch`, sem custo de fidelidade digno de nota. Esse é o próximo experimento, e ele não precisa de retreino nenhum também.

■ 10.8 REPRODUZIR

```
uv sync --extra conversao
uv run python scripts/encolher_modelos.py          # converte e afere
uv run python scripts/comparar_backends.py --n 180 # o portão que recusou
```

■ 10.9 VARIANTE EXPERIMENTAL: UM ENCODER, TRÊS CABEÇAS

O runtime agora aceita `FRAUS_BACKEND=onnx-multitarefa`, mas ele **não é o padrão** e nenhum artefato foi promovido. Diferente da quantização, compartilhar o encoder exige treino: os três checkpoints atuais aprenderam representações diferentes durante o fine-tuning.

O caminho implementado usa destilação. Os modelos atuais são os professores e um BERTimbau único aprende simultaneamente as distribuições de satisfação, emoção e ironia. A saída ONNX possui três tensores nomeados e o `Motor` executa o encoder uma vez por lote — não três vezes com pesos apenas compartilhados.

```
uv sync --extra treino --extra onnx
uv run python scripts/treinar_multitarefa.py corpus.csv --epocas 3
uv run python scripts/comparar_backends.py --multitarefa --n 180
```

O corpus aceita CSV/JSONL com campo `texto` ou TXT com uma frase por linha. Use um corpus público representativo; não exporte conversas de clientes sem base legal e anonimização. O candidato é gravado separadamente e só deve virar `modelos-onnx/bertimbau-multitarefa` depois de cumprir, em conjunto:

- zero troca de categoria nas 180 conversas de regressão;
- métricas por cabeça não inferiores às publicadas nos cartões dos modelos;
- pico de RSS abaixo de 800 MB no container completo;
- inspeção das fatias críticas antes da troca da régua.

O tamanho esperado do grafo fp32 é aproximadamente o de um BERTimbau (cerca de 416 MB) mais três camadas lineares pequenas. O número de RAM continua sendo uma hipótese até o artefato treinado ser medido; o código não o apresenta como ganho concluído.

11. Limitações conhecidas

Esta página existe porque um sistema que estima satisfação **precisa** declarar onde erra. Nada aqui é segredo, e nada aqui está escondido na tela: cada item tem contrapartida visível na interface.

■ 11.1 LIMITAÇÕES OPERACIONAIS DO DEPLOY GRATUITO

A hospedagem da API deixou de ser pendência em 17/09/2026, mas “publicada” não significa “sem limites”. A função ONNX tem cerca de 1,46 GB e usa Large Functions na Vercel. Depois de esfriar, o primeiro `/saude` observado levou 10,8 s; requisições seguintes foram rápidas, mas não há garantia de latência.

Cold start e "motor aquecendo". A API é serverless: cada instância tem o próprio motor, e ele só ficava pronto depois da primeira predição. Desde 30/09/2026 o motor aquece em segundo plano no boot da instância e a cada sonda de saúde, e um workflow sonda a API a cada 5 minutos. Isso **reduz** o cold start, mas não o elimina — a Vercel não garante que a sonda e o usuário caiam na mesma instância. Eliminá-lo exige um processo sempre ligado com pelo menos 1,5–2 GB de RAM; o plano gratuito do Render (512 MB, dorme em 15 min) não serve, porque a API mede cerca de 1,06 GB de RSS com os três modelos.

Vercel Hobby, Supabase Free e Oracle Object Storage gratuito oferecem cotas, não SLA. Ao ultrapassá-las, a aplicação pode ser limitada ou pausada. O estado fica no Supabase; o Oracle participa do build, não da inferência. Uma restauração do projeto pode alterar a URI do pooler, e o transaction pooler precisa usar a porta indicada no painel (6543 no deploy atual).

Essas limitações são operacionais. Elas não alteram as limitações científicas abaixo e não autorizam apresentar a estimativa como NPS declarado.

■ 11.2 O NPS É INFERIDO, NÃO PERGUNTADO

Toda exibição carrega a etiqueta de estimativa e o sublinhado pontilhado de proveniência. Apresentar como NPS declarado seria falso.

O intervalo de confiança cobre a incerteza AMOSTRAL, não a do modelo. Ele responde "quantas conversas sustentam este número"; ele **não** responde "o quanto o modelo pode estar errado" — isso exigiria calibração, que ainda não existe. Um IC apresentado como se cobrisse o erro do modelo é pior que não ter IC nenhum, e por isso a ressalva está impressa junto do número, não só aqui.

O intervalo usa **Wald ajustado triangular AW(3,T)**: pseudocontagens de 0,75 para cada extremo e 1,5 para neutros, com variância dividida por $n + 2$, conforme [Turk, Cinderich e McNeill \(2026\)](#). Isso evita intervalo de largura zero quando todas as conversas pertencem à mesma categoria. O NPS pontual e a contagem continuam sendo os observados; o ajuste só participa dos limites do intervalo.

Abaixo de **30 atendimentos com sinal** o cartão principal e a leitura do intervalo ocultam o ponto estimado e mostram o $\lfloor n \rfloor$. Esse corte é uma regra de exibição, não uma garantia de precisão a partir de 30. A série diária e o

campo `nps` da API preservam o agregado observado; `nps intervalo.nps` é o campo que autoriza a exibição do ponto no cartão. Sem intervalo (API antiga ou falha do endpoint), a dashboard preserva o agregado disponível.

■ 11.3 O SINAL DE TEMPO É TREINADO EM DADOS SINTÉTICOS

Nenhum corpus público de review em português tem timestamps de diálogo. O simulador (`fraus/ingest/simulador.py`) gera conversas determinísticas com latências calibradas por literatura de live chat.

E isso tem uma consequência medida, não hipotética. As features de latência não têm teto, e o corpus nunca passou de minutos (medianas de 5/30/200 s). Numa conversa com **3 horas** de espera, medida em 08/09/2026 contra a API real:

```
latencia_primeira_resposta_s -85,405
duracao_total_s             -35,759
latencia_mediana_s          -20,569
latencia_p90_s              -12,326
texto_prob_satisfeito_media +4,574 ← o texto inteiro
```

O tempo pesa **20x o texto**, e a conversa sai em `4,4e-24`. O peso *por unidade* continua pequeno (0,12-0,16); o que explode é o **z-score** que entra no `StandardScaler`.

O conserto de verdade — `log1p` ou winsorização antes do scaler — exige retreino. Enquanto ele não vem, `GET /saude/deriva` torna o caso **visível**: contra o fusor real, essa conversa dá `|z| = 124,9` em `latencia_mediana_s`, contra um limiar de 4.

■ 11.4 A CABEÇA DE IRONIA É POUCO CONFIÁVEL

Ela reporta acurácia `1.0` no corpus gerado e erra **6 em 10** falas sinceras de atendimento, com 0,999 de confiança — vazamento de marcador de discurso no gerador, já corrigido no gerador e pendente de retreino.

O vazamento não chega mais à nota, porque a ironia saiu do vetor em 04/09/2026 por um motivo diferente e mais grave (ver [Arquitetura](#)). Ela continua sendo exibida por mensagem, com a ressalva colada ao número.

■ 11.5 DESPREZO NÃO É UMA CLASSE TREINADA

Nenhum corpus em português a anota. Ela é derivada da díade raiva + nojo (Plutchik, 1980) pela **média geométrica** — que exige as *duas* emoções juntas, enquanto a média aritmética daria meio ponto para raiva pura sem nojo nenhum, o que é raiva, não desprezo.

■ 11.6 NENHUMA FEATURE AGREGADA REVERTE UMA PROBABILIDADE SATURADA

A frase canônica — *"que atendimento maravilhoso, só esperei 3 horas"* — sai promotora sempre que o relógio não a contradiz.

`incongruencia_situacao_negativa` foi criada para alcançá-la e tem peso **−0,193**; ela não vence os **+2,78** de `texto_prob_satisfeito_media` quando o BERTimbau lê a frase como elogio sincero.

Por isso a **contestação** (08/09/2026) **marca em vez de corrigir**: quando `score > 95` e `latencia_mediana_s > 180`, a tela avisa — e o atendimento **continua contando no NPS**, com teste provando isso. Tirá-lo do agregado esvaziaria o indicador em silêncio se o limiar estivesse mal calibrado.

O limiar de score em 95 é o único número do projeto **sem procedência**, e está declarado assim no código e na spec. O de tempo vem de IJHCI 2025.

■ 11.7 O QUE NÃO DÁ PARA CONSERTAR SEM CORPUS DE ATENDIMENTO REAL

Duas coisas ficam condicionadas ao mesmo recurso que não existe publicamente em português — um corpus de **diálogo de atendimento** rotulado:

1. **retreinar a cabeça de ironia** em domínio certo;
2. **a interação tempo × texto** como feature. Ela nasceria com o peso invertido: o texto vem do B2W, a latência sai de distribuição por rótulo, e satisfeito-e-lento está rotulado *satisfeito* por construção. Isso é leitura do corpus, não experimento.

12. Referência de código

■ 12.1 INDICADORES

NPS, CSAT, contenção, **falso containment** e a série diária. Todo agregado deste módulo devolve `None` na ausência — nunca `0`, que seria apresentar como medido um número que ninguém mediu.

| 12.1.1 fraus.indicadores

Indicadores agregados.

NPS aqui é INFERIDO do texto, não declarado pelo cliente. A dashboard rotula como estimativa -- apresentar como NPS declarado seria falso.

Faixas canônicas: 0-6 detrator, 7-8 neutro, 9-10 promotor. Categoria SEMPRE derivada no servidor.

| categoria_nps

```
categoria_nps(score_0_100: float, faixas: dict[Categoria, tuple[int, int]] | None = None) -> Categoria
```

Categoria da nota segundo `faixas` -- por PARAMETRO, nunca global mutável.

Quando a configuração muda as faixas, quem lê passa as faixas vigentes; o default é o padrão de fábrica. Assim a faixa continua morando num lugar só e não existe estado global que mude sob os pés de quem já calculou.

| validar_faixas_nps

```
validar_faixas_nps(faixas: dict[str, tuple[int, int]]) -> None
```

Recusa faixa que não cubra 0..10 de forma contígua, nomeando o problema.

Faixa de NPS com buraco ou sobreposição não é preferência de gosto: seria nota sem categoria (ou com duas), e o erro apareceria muito depois, na leitura de um atendimento qualquer. Levanta `ValueError` -- a borda HTTP traduz para 400 com a mesma frase.

| calcular_nps

```
calcular_nps(scores: list[float], faixas: dict[Categoria, tuple[int, int]] | None = None) -> float | None
```

Percentual de promotores menos percentual de detratores, em `[-100, 100]`.

Sem score algum devolve `None` -- ausência de dado não é insatisfação, e "NPS +0" seria apresentar como medido um número que ninguém mediu.

| nps_com_intervalo

```
nps_com_intervalo(scores: list[float], faixas: dict[Categoria, tuple[int, int]] | None = None) -> dict | None
```

NPS inferido com intervalo de confiança de 95% (AW(3,T)).

Hoje "NPS -12" aparece igual com 8 conversas e com 8.000, e a primeira pergunta de quem avalia e "quantas conversas sustentam esse numero?". Mostrar ponto estimado sem incerteza e inconsistente com um sistema que já recusa transformar ausencia em zero.

O ponto observado continua sendo percentual de promotores menos percentual de detratores, sem suavizacao, igual a `calcular_nps`. Para o INTERVALO usamos Wald ajustado triangular AW(3,T): acrescentar 0,75 as contagens de cada extremo e 1,5 aos neutros. São pseudocontagens estatísticas, nunca atendimentos reais nem mudança no score do fusor.

```
n_ajustado = n + 3
p_prom = (promotores + 0,75) / n_ajustado
p_det = (detratores + 0,75) / n_ajustado
media = p_prom - p_det
variancia = (p_prom + p_det) - media**2
erro = sqrt(variancia / (n_ajustado - 1))
ic = media +- 1,96 * erro (tudo x100 na escala do NPS)
```

Referencia e implementacao dos autores (variancia com $n_{ajustado} - 1$): <https://doi.org/10.3390/stats9020045>
https://github.com/philturk/Net_Promoter_Score_Confidence_Intervals O ajuste evita largura zero em categoria unanime. Esconder o ponto em amostra pequena e uma regra de exibicao SEPARADA do metodo estatistico: abaixo de `N MINIMO NPS` ele nao e devolvido. O `n` permanece observado.

None quando nao ha score nenhum -- nao existe intervalo de coisa nenhuma.

HONESTIDADE OBRIGATORIA, e ela precisa aparecer na tela junto do numero: este intervalo captura so a incerteza AMOSTRAL. Ele NAO captura a incerteza do MODELO, que exigiria calibracao. Um IC apresentado como se cobrisse o erro do modelo e pior que nao ter IC nenhum.

| calcular_csat

```
calcular_csat(scores: list[float]) -> float | None
```

Percentual de atendimentos com nota ≥ 7 . Sem score algum devolve None.

| mediana

```
mediana(valores: list[float]) -> float | None
```

Mediana, ou None na lista vazia -- nunca 0.0.

Zero seria "respondeu instantaneamente", que e o oposto de "nao da para saber". A serie temporal precisa poder dizer que nao mediu.

| serie_diaria

```
serie_diaria(registros: list[tuple[Conversa, float | None]], faixas: dict[Categoria, tuple[int, int]] | None = None) -> list[dict]
```

NPS inferido e latencia mediana por dia, ordenados do mais antigo.

O dia sai de `iniciada_em` da conversa, lido no fuso do produto (`fraus.fuso`) -- nao no offset em que a fonte o mandou. A latencia do dia e a mediana das MEDIANAS de cada atendimento, nao a mediana de todas as esperas juntas: um unico atendimento com trinta idas e vindas dominaria o dia inteiro se as esperas fossem jogadas num balde so.

`nps` e `latencia_mediana_s` sao None no dia sem dado, e os dois motivos sao diferentes -- por isso `atendimentos` e `com_score` vem separados. Um dia pode ter atendimento e nenhum score (conversa sem fala do cliente), e apresentar isso como NPS 0 seria inventar medicao.

| containment_rate

```
containment_rate(conversas: list[Conversa]) -> float | None
```

Percentual de conversas resolvidas sem intervencao humana.

None no conjunto VAZIO -- nunca 0.0, que se leria como "nenhum atendimento foi contido", o pior numero da escala, onde nao houve atendimento nenhum.

A distincao que importa e que esta funcao NAO depende de score: conversa sem fala do cliente nao tem NPS nem CSAT e mesmo assim conta como contida (ha teste exigindo 100% nesse caso). O que a ausencia de score nao mede, ela continua medindo. So o conjunto vazio nao tem resposta.

Ate 10/09/2026 devolvia 0.0, e o front nao acreditava: `page.tsx` escrevia `total conversas ? containment_rate : null`. A guarda no cliente era a prova de que alguem ja tinha sentido o problema -- e ela so protegia a dashboard. Export, webhook e qualquer terceiro lendo /indicadores recebiam o zero. Uma ponta so decide agora.

| contidos_com_score

```
contidos_com_score(registros: list[tuple[Conversa, float | None]]) -> int
```

Quantos atendimentos NAO escalaram e tem score -- o denominador honesto.

A tela precisa dele para dizer "3 de 12" em vez de so um percentual: sem o denominador, 33% sobre tres atendimentos parece a mesma coisa que 33% sobre trezentos.

| falso_containment

```
falso_containment(registros: list[tuple[Conversa, float | None]], faixas: dict[Categoria, tuple[int, int]] | None = None) -> float | None
```

Percentual de atendimentos contidos que saíram DETRATORES.

Conteve e o cliente saiu insatisfeito e sucesso falso: a metrica de contencao sobe enquanto a experiencia piora. So o atendimento COM score entra na conta -- conversa sem fala do cliente nao e nem sucesso nem fracasso, e o denominador precisa dizer sobre quantos se esta falando.

O limiar de insatisfacao e a categoria `detrator` das faixas VIGENTES, recebidas por parametro (invariante 4) -- nunca um 6 digitado aqui.

None quando nenhum atendimento contido tem score -- nunca 0.0, que se leria como "nenhum contido saiu insatisfeito", afirmacao que ninguem mediu.

| tempo_mediano_resposta

```
tempo_mediano_resposta(registros: list[tuple[Conversa, float | None]]) -> float | None
```

Mediana de TODAS as esperas cliente -> resposta do conjunto.

`None` quando nao ha nenhum par -- nunca zero: zero numa medida de tempo de resposta se le como "respondeu na hora", e conjunto sem par nao mediu espera nenhuma.

| lexico_por_classe

```
lexico_por_classe(registros: list[tuple[Conversa, float | None]], faixas: dict[Categoria, tuple[int, int]] | None = None, limite: int = 6) -> list[dict]
```

Palavras e emojis caracteristicos de cada categoria do conjunto.

Ordenado por DISTINCAO, nao por frequencia: `distincao` e a diferenca entre a fracao do termo nesta classe e a fracao dele nas outras, em [-1, 1]. O termo que aparece em toda parte nao explica classe nenhuma.

Mesma regra da derivacao que a dashboard fazia no cliente (e mantem como plano B): so a fala do CLIENTE conta -- o texto do bot e roteiro --, e conversa sem score fica fora, porque sem categoria nao ha classe onde contar. Emoji e recortado por cluster de grafema, pela mesma extracao do sinal de emoji, para a contagem nao divergir do motor.

■ 12.2 MODELO CANÔNICO

`Conversa` e `Mensagem`. Toda fonte de entrada desemboca aqui, e o resto do sistema não sabe de onde o dado veio.

12.2.1 fraus.modelos

Modelo canonico de conversa. Toda fonte de dado e normalizada para ca.

Conversa

Bases: `BaseModel`

`tem_fala_cliente` property

tem_fala_cliente: <code>bool</code>

O cliente escreveu alguma coisa -- e o que importa para a OPERACAO.

`tem_sinal_cliente` property

tem_sinal_cliente: <code>bool</code>

Ha fala do cliente que diga algo sobre satisfacao -- e o que a NOTA exige.

Desde 15/09/2026, conversa em que toda fala do cliente e formula de cortesia ("ok, obrigado", "valeu") nao tem sinal: a mesma formula fecha atendimento bom e ruim. Ver [fraus/cortesia.py](#).

`motivo_sem_sinal` property

motivo_sem_sinal: Literal['sem_fala_do_cliente', 'so_cortesia'] None
--

POR QUE nao ha nota -- `None` quando ha sinal.

Existe para a tela nao dizer "o cliente nao falou" a quem escreveu "ok, obrigado", e para ela nao precisar reimplementar a regra da cortesia em TypeScript (invariante 3: regra derivada mora no servidor).

■ 12.3 O FUSOR E AS 39 FEATURES

`NOMES_FEATURES` é o **contrato**. O comentário dele conta a história inteira do vetor — por que subiu para 40, por que a ironia saiu, e por que 39 é o número de hoje.

12.3.1 fraus.fusor

Fusor das sete famílias de sinal que entram no vetor (de oito no sistema -- a ironia continua existindo, so nao pontua mais; ver `NOMES_FEATURES`).

LogisticRegression com padronizacao: interpretavel de proposito -- o trabalho precisa defender POR QUE um atendimento recebeu a nota, e coeficiente de regressao logistica responde isso; floresta densa nao.

O peso da latencia e APRENDIDO aqui, nunca arbitrado: a relacao com satisfacao e nao-linear e moderada por contexto.

FusorIncompativeError

Bases: `RuntimeError`

Artefato treinado (`[.joblib]`) nao bate com `NOMES_FEATURES` vigente.

Distinta de `ModeloAusenteError` (fraus/sinais/texto.py) de proposito: as duas impedem servir predicao (invariante 7), mas o remedio e diferente. Um modelo AUSENTE pede treinar do zero; um modelo INCOMPATIVEL ja existe e foi treinado com um contrato antigo -- o remedio e RETREINAR com o contrato atual, nao gerar um artefato novo do nada. Misturar as duas mensagens levaria quem le o erro a rodar o notebook errado.

Fusor

`escala_do_tempo` `property`

```
escala_do_tempo: str
```

Escala em que as features de espera foram aprendidas ("segundos" = antiga).

pontuar

```
pontuar(features: dict[str, float]) -> float
```

Score 0-100: P(satisfeito) mais P(neutro) pesado por `PESO_NEUTRO_NO_SCORE`.

O score e uma PROJECAO das tres probabilidades num eixo, nao uma quarta predicao: o modelo continua o mesmo, treinado do mesmo jeito. O peso do neutro decide onde a classe do meio pousa nesse eixo -- e, por composicao com a faixa de NPS, em qual categoria ela cai. A tabela e o raciocinio estao na constante.

O que este metodo NAO e: um lugar para consertar predicao. Se o modelo confunde as classes, o conserto e treino -- as medianas por classe do `scripts/medir_faixas.py` sao justamente o que separa os dois casos.

| prever

```
prever(features: dict[str, float]) -> int
```

Classe predita: 0 insatisfeito, 1 neutro ou 2 satisfeito.

| contribuicoes

```
contribuicoes(features: dict[str, float]) -> dict[str, float]
```

Quanto cada feature empurrou a nota DESTA conversa, com sinal.

Eixo: coeficiente da classe satisfeito menos coeficiente da classe insatisfeito, multiplicado pelo valor JA PADRONIZADO da feature (o `StandardScaler` do pipeline aplicado, nunca o valor bruto). Positivo empurrou a nota para cima (rumo a satisfeito); negativo puxou para baixo (rumo a insatisfeito). E o numero da CONVERSA, diferente de `importancias`, que e o peso medio GLOBAL aprendido pelo modelo.

Se o modelo aprendeu menos de tres classes e nao tem as duas pontas (insatisfeito e satisfeito) para formar a diferenca, nao ha eixo interpretavel: a contribuicao volta zerada para todas as features, em vez de estourar.

| eixo_global

```
eixo_global() -> dict[str, float]
```

Quanto cada feature empurra a nota NO MODELO INTEIRO, com sinal.

Diferente de `importancias`, que e o peso ABSOLUTO (diz que a feature pesa, nao para que lado), e de `contribuicoes`, que e o numero de UMA conversa. O grafo da memoria consome este: ele responde "o que o modelo aprendeu que caracteriza um detrator".

Nao ha padronizacao aqui porque nao ha conversa: e o coeficiente cru.

Fusor nao treinado devolve `{}` -- e a ausencia de eixo, que o grafo distingue de "todas as features pesam zero".

| distribuicao_de_treino

```
distribuicao_de_treino() -> dict[str, dict[str, float]]
```

Media e desvio de cada feature NO CORPUS DE TREINO, por feature.

O `StandardScaler` treinado guarda `mean` e `scale` -- e isso e o retrato da distribuicao em que o modelo aprendeu, ja gravado dentro do artefato. LER NAO E TREINAR: nao precisa do corpus, nem de notebook, nem de GPU.

Existe para responder uma pergunta que a invariante 10 so sabia fazer no notebook: "esta conversa esta dentro da faixa em que o modelo foi treinado?". A guarda de vazamento olha a distribuicao no CORPUS; em runtime aparece latencia de tres horas que o corpus nunca teve, o z-score explode e o relógio assume a nota.

Fusor nao treinado devolve `{}` -- ausencia do retrato, que e diferente de "todas as medias sao zero".

| `z_das_features`

```
z_das_features(features: dict[str, float]) -> dict[str, float]
```

Quantos desvios de treino cada feature desta conversa esta da media.

E exatamente o que o `StandardScaler` entrega ao modelo -- o mesmo numero que multiplica o coeficiente. Por isso a sonda usa `transform` em vez de refazer a conta: uma segunda implementacao da padronizacao divergiria da real no dia em que o pipeline mudasse, e a rota de diagnostico passaria a diagnosticar a si mesma.

Feature constante no treino tem `scale` = 1 por decisao do proprio scikit-learn, entao nao ha divisao por zero aqui.

Fusor nao treinado devolve `{}`.

| `assinatura`

```
assinatura() -> str | None
```

Impressao digital dos pesos aprendidos -- muda a cada retreino.

Hash do que DECIDE a nota: media e escala do scaler, coeficientes, interceptos, ordem das classes e a escala do tempo. Existe para o banco saber com qual modelo cada conversa foi pontuada (`Motor.regua`). Fusor nao treinado nao tem assinatura.

| `importancias`

```
importancias() -> dict[str, float]
```

Peso absoluto medio de cada feature -- alimenta a explicacao na dashboard.

| `carregar` classmethod

```
carregar(caminho: Path) -> Fusor
```

Carrega o artefato e VALIDA a forma contra `NOMES_FEATURES` antes de devolver.

`joblib.load` puro (o comportamento anterior) nao checava nada: um artefato de 40 features carregava sem erro contra um contrato de 38, a API subia normalmente, `/modelo/similar` funcionava (nao passa pelo fusor) e enganava quem testava a mao -- o `ValueError` do `StandardScaler` so estourava como HTTP 500 na primeira pontuacao real, em `/ingestao` ou `/conversas/importar`. Isso confundiu duas vezes na mesma semana. A invariante 7 pede falha alta e EXPLICITA para modelo incompativel, e "explicita" nao basta se so acontece na primeira predicao -- precisa ser na CARGA, que e quando a API sobe.

| montar_features

```
montar_features(conversa: Conversa, classificador, classificador_emocao, curadoria=None) -> dict[str, float]
```

Junta as SETE familias de sinal do vetor numa linha unica de features.

Sao sete familias NO VETOR -- oito sinais existem no sistema, porque a ironia continua sendo lida por mensagem (ver `Motor.ironia de`), so nao entra aqui. Ver o comentario acima de `NOMES FEATURES` para o porque.

Os dois classificadores (texto e emocao) sao OBRIGATORIOS desde que o contrato subiu para 35: emocao deixou de ser leitura decorativa e passou a mover a nota. Aceitar `None` aqui produziria vetor incompleto, e vetor incompleto vira `KeyError` la em `vetorizar` -- com a diferenca de que o erro apontaria para o lugar errado. Note que NAO ha `classificador ironia` aqui: a cabeca de ironia continua obrigatoria para o Motor (invariante 7), mas este montador de vetor nao precisa mais dela.

`curadoria` e opcional e chega POR PARAMETRO, nunca por estado global: e o que o analista ensinou ao lexico, lido a cada requisicao. Ela alcanca TRES familias -- `lexico *`, `emoji *` e `incongruencia *`, que le os dois lexicos por dentro e portanto herda a curadoria deles -- e MAIS NENHUMA: o contrato continua de 39 chaves (invariante 9), e o que ela muda e o VALOR dessas familias, jamais o conjunto de features.

| vetorizar

```
vetorizar(features: dict[str, float]) -> list[float]
```

Ordem canonica. Feature faltando e `KeyError`; feature sobrando e `ValueError`.

Nenhum dos dois casos pode virar zero silencioso (invariante 9): falta ja estourava; sobra nao estourava -- um dict com as 39 chaves certas MAIS uma extra passava batido, gerando um vetor do tamanho certo por acaso. E justamente o cenario que aconteceria se uma familia de sinal saísse do contrato (como `ironia *` saiu em 04/09/2026) mas continuasse sendo produzida por engano em algum chamador: o vetor ficaria certo e a regressao passaria despercebida.

■ 12.4 OS SINAIS

As oito famílias que leem a conversa. Todas leem **apenas a fala do cliente** (`mensagens_cliente`) — e isso não é detalhe de implementação, é a fronteira que mantém o sistema fora da proibição de reconhecimento de emoção no local de trabalho. Ver [Conformidade](#).

12.4.1 Texto — o único que pontua sozinho

12.4.2 `fraus.sinais.texto`

Sinal de texto: BERTimbau fine-tunado, rodando em CPU.

Predicao POR MENSAGEM, nunca por conversa inteira -- e isso que permite a atribuicao por sentenca na dashboard ("quais trechos puxaram a nota").

BERTimbau supera as variantes multilingues em classificacao de sentimento PT-BR (Souza, Nogueira, Lotufo -- arXiv:2201.03382).

ModeloAusenteError

Bases: `RuntimeError`

Modelo nao encontrado ou ilegivel. Falha alta: sem modelo nao ha predicao.

ClassificadorTexto

`prever_mensagens`

```
prever_mensagens(textos: list[str]) -> list[list[float]]
```

Probabilidades [insatisfeito, neutro, satisfeito] para cada texto.

`carregar_torch`

```
carregar_torch(caminho_modelo: Path, rotulo: str)
```

Tokenizador e modelo do checkpoint, com o torch importado SO AQUI.

O torch (497 MB) deixou de ser dependencia principal: com `FRAUS_BACKEND=onnx` a API pontua sem ele. Importar no topo do modulo faria `fraus.motor` -- que todo mundo importa -- exigir torch mesmo quando nenhum checkpoint torch vai ser carregado. Faltar o torch quando o backend E torch continua falha alta (invariante 7), com o remedio escrito.

| prever_torch

```
prever_torch(tokenizador, modelo, textos: list[str]) -> list[list[float]]
```

Softmax por texto, num lote com padding pela mensagem mais longa.

12.4.3 Emoção

| 12.4.4 fraus.sinais.emocao

Sinal de emocao: segundo BERTimbau fine-tunado, rodando em CPU.

Requisito de banca: distinguir as SETE emocoes humanas. A cabeca treinada tem sete classes -- as seis basicas de Ekman (1992) mais o neutro, que nao e emocao e sim a ausencia dela. A setima emocao, o DESPREZO, nao e treinada: nenhum corpus rotulado em portugues a anota, e treinar so ela em corpus de outra procedencia ensinaria o modelo a reconhecer o estilo do texto em vez do desprezo. Ela e derivada da diade primaria raiva + nojo, que e como o Plutchik (1980) a define -- ver [desprezo derivado](#).

Corpus: [go emotions ptbr](#), traducao PT-BR do GoEmotions (Demszky et al., 2020: 58k comentarios do Reddit, 27 emocoes + neutro), reduzido as seis de Ekman pelo mapeamento oficial que o proprio GoEmotions publica.

LIMITACAO METODOLOGICA, a declarar no relatorio: o corpus foi traduzido por maquina, sem revisao humana. Por isso a avaliacao reporta tambem o XED-pt, que nao passou por traducao automatica. Ver docs/treinamento.md.

| ClassificadorEmocao

| prever_mensagens

```
prever_mensagens(textos: list[str]) -> list[list[float]]
```

Probabilidades das sete classes, na ordem de NOMES_EMOCOES.

| desprezo_derivado

```
desprezo_derivado(prob_raiva: float, prob_nojo: float) -> float
```

Desprezo como diade primaria raiva + nojo (Plutchik, 1980).

Media GEOMETRICA, nao aritmetica: desprezo exige as DUAS emocoes juntas, e a media aritmetica daria 0.5 para raiva pura sem nojo nenhum -- o que e raiva, nao desprezo. O produto puro subestimaria (0.7 e 0.7 viram 0.49); a geometrica devolve 0.7, que e a leitura certa de "as duas altas".

features_emocao

```
features_emocao(conversa: Conversa, classificador) -> dict[str, float]
```

Media de cada emocao nas falas do cliente, mais o desprezo derivado.

Sao OITO features: as sete classes treinadas e o desprezo da diade. A media e sobre as mensagens do cliente, coerente com o sinal de texto -- fala do bot nao carrega emocao do cliente.

12.4.5 Tempo

12.4.6 fraus.sinais.tempo

Sinal de tempo.

Latencia entra como FEATURE aprendida, nunca como penalidade linear: o efeito sobre satisfacao e nao-linear (CSAT pico entre 5-10s, queda de 2-3 pontos por minuto extra, 57% de abandono acima de 3min) e moderado por contexto -- indicador de digitacao e suporte emocional atenuam o dano (From Seconds to Sentiments, IJHCI 2025).

Nada aqui e persistido: tudo deriva dos timestamps do modelo canonico.

latencias_da_conversa

```
latencias_da_conversa(conversa: Conversa) -> list[float]
```

Intervalo de cada mensagem do cliente ate a proxima resposta.

PUBLICA de proposito: a serie temporal ([fraus.indicadores.serie_diaria](#)) e a transcricao da dashboard precisam da MESMA regra que vira feature do modelo. Duas definicoes de latencia -- uma para treinar, outra para exibir -- fariam o grafico contar uma historia que o fusor nunca viu.

12.4.7 Léxico

12.4.8 fraus.sinais.lexico

Sinal lexico: SentiLex-PT02 com escopo de negacao.

Fonte: SentiLex-PT02 -- Silva, Carvalho e Sarmento, "Building a Sentiment Lexicon for Social Judgement Mining" (PROPOR 2012), CC-BY. 79.189 formas flexionadas depois da conversao, em [fraus/dados/sentilex_pt02.csv](#). O [scripts/preparar_sentilex.py](#) documenta o que foi normalizado.

POR QUE um lexicon se o BERTimbau ja le o texto: porque o lexicon nao foi treinado nos mesmos dados nem carrega os mesmos vieses. Dois sinais de procedencia independente que concordam sustentam a nota melhor que um sozinho, e discordancia entre eles e informacao -- e o formato tipico da IRONIA, texto lexicamente positivo com contexto negativo.

A NEGACAO e tratada por escopo, nao por saco de palavras: "nao foi otimo" inverte a polaridade de "otimo". Sem isso o lexicon leria a frase como positiva, que e o erro classico de analise lexical de sentimento.

LIMITACAO DO RECURSO, a declarar no relatorio: o SentiLex e um lexicon de JULGAMENTO SOCIAL -- anota polaridade dirigida a entidades humanas. Ele e forte no adjetivo que julga ("pessimo", "otimo", "incompetente") e NEUTRO em verbo de afeto do proprio falante: `gostar`, `adorar` e `odiar` valem 0 nele. Por isso o sinal lexico COMPLEMENTA o BERTimbau e nao o substitui -- o transformer e quem le "adorei o produto".

| sem_acento

```
sem_acento(texto: str) -> str
```

Remove diacriticos, preservando o resto.

O cliente de chat nem sempre acentua, entao `otimo` precisa achar `otimo`. Quem consome constroi um INDICE com isto -- nunca compara termo a termo em laco, que seria varrer o lexicon inteiro a cada token.

| polaridade_do_termo

```
polaridade_do_termo(termo: str, curadoria: Curadoria | None = None) -> int
```

Polaridade em -1/0/1. Termo fora do lexicon e nao curado vale 0.

| anotar_texto

```
anotar_texto(texto: str, curadoria: Curadoria | None = None) -> list[tuple[str, int, bool]]
```

Termos do lexicon achados em `texto`: (termo, polaridade final, negado).

A polaridade final ja vem com a negacao aplicada. `negado` diz se ela foi invertida -- e o que alimenta a feature de negacao, e o que permite mostrar na dashboard POR QUE um termo positivo contou como negativo.

| features_lexico

```
features_lexico(conversa: Conversa, curadoria: Curadoria | None = None) -> dict[str, float]
```

Tres features do lexicon sobre as falas do cliente.

`lexico_cobertura` existe para o fusor saber QUANTA evidencia lexical sustenta a polaridade media: media de -1 apoiada em um termo vale menos que a mesma media apoiada em dez.

■ 12.5 EVIDÊNCIA E DERIVA

Os dois módulos que respondem "**dá para confiar nesta medida?**" — um por conversa, outro pelo modelo inteiro. Nenhum dos dois altera predição: são diagnóstico.

12.5.1 Evidência fraca — a cabeça tracejada

| 12.5.2 `fraus.evidencia`

Evidencia fraca: tem dado, e não há material suficiente para afirmar.

A interface tinha DUAS formas -- cabeça cheia (medido) e cabeça vazada (não medido, `CabecaVazada.tsx`). Faltava a terceira, e o buraco era visível: uma conversa cujo cliente escreveu só "ok" saia CHEIA, com a mesma confiança visual de uma conversa de 40 turnos.

cheia	medido, com evidencia
tracejada	medido, evidencia fraca
vazada	não medido

O QUE ESTE MÓDULO NÃO FAZ, e é a decisão mais importante dele: não usa a probabilidade do modelo. Probabilidade não calibrada NÃO É CONFIANÇA, e apresenta-la como se fosse seria a desonestidade exata que este projeto recusa em todo lugar -- calibrar exige retreino, que está fora de escopo. A medida aqui é EVIDÊNCIA OBSERVÁVEL: quantas vezes o cliente falou e quanto ele escreveu. Auditável por quem abrir o transcript, e defensável sem apelar para propriedade nenhuma do modelo.

Pelo mesmo motivo o campo se chama `evidencia_fraca`, e não `confianca`: o segundo nome prometeria uma calibração que não existe.

| `motivos_de_evidencia_fraca`

<code>motivos_de_evidencia_fraca</code> (conversa: Conversa, palavras_minimas: <code>int</code> = PALAVRAS_MINIMAS) -> <code>list[str]</code>

Por que a evidência desta conversa é fraca -- em texto, não em código.

"Evidência fraca" sozinho não aciona ninguém; "uma única mensagem do cliente" aciona. A tela imprime estes motivos ao lado da forma, porque forma sozinha obriga o leitor a conhecer a convenção -- a mesma regra que faz `CabecaVazada` exigir rótulo textual.

Lista vazia = evidência suficiente.

| evidencia_fraca

```
evidencia_fraca(conversa: Conversa, palavras_minimas: int = PALAVRAS_MINIMAS) -> bool | None
```

A conversa tem score, mas pouca fala para sustenta-lo?

`None` sem fala nenhuma do cliente -- e o estado "sem sinal", que ja tem forma propria. "Sem sinal" e "sinal fraco" sao coisas DIFERENTES, e confundi-las na interface perderia a distincao que o produto inteiro defende: ausencia nao e um valor baixo.

12.5.3 Deriva de distribuição — a invariante 10 em runtime

| 12.5.4 fraus.deriva

Deriva de distribuicao: a invariante 10 como alarme de runtime.

A invariante 10 diz que o corpus de treino nao pode entregar o rotulo, e o criterio de `docs/treinamento.md` e explicito: "se as features que lideram forem as circunstanciais (tempo, contagem de turnos), procure o vazamento". Essa guarda roda no NOTEBOOK, sobre o corpus.

O que ela nao alcanca: o corpus do sinal de tempo nunca passou de minutos (medianas de 5/30/200 s no simulador), e em producao aparece conversa com tres horas de espera. Medido em 08/09/2026 contra a API real, uma conversa assim recebe `latencia_primeira_resposta_s -85,4` contra `texto_prob_satisfeito_media +4,6` -- o tempo pesa 20x o texto e o score sai em 4,4e-24. O peso POR UNIDADE continua pequeno (0,12-0,16); o que explode e o z-score que entra no scaler.

Este modulo nao corrige nada disso. O conserto de verdade -- `log1p` ou winsorizacao antes do scaler -- exige RETREINO, e retreino esta fora de escopo. O que ele faz e tornar o caso VISIVEL: dizer quais features da amostra recente estao fora da faixa em que o modelo aprendeu, com o nome de cada uma.

Por que da para fazer sem retreinar: o `StandardScaler` treinado ja carrega `mean` e `scale` por feature dentro de `modelos/fusor.joblib`. Ler nao e treinar.

A parte PURA mora aqui, separada da rota, porque ela nao precisa de BERTimbau nenhum -- e o que permite a regra falhar em milissegundos de pytest.

| resumo_de_deriva

```
resumo_de_deriva(zs_por_conversa: list[dict[str, float]], distribuicao: dict[str, dict[str, float]], limiar: float = LIMIAZ_Z) -> dict | None
```

Quais features da amostra recente sairam da faixa de treino, e quanto.

`zs_por_conversa` e uma lista de `{feature: z}` -- o valor JA padronizado pelo scaler do fusor, que e literalmente o numero que multiplica o coeficiente na predicao.

O z reportado por feature e o **pior modulo da amostra**, nunca a media: uma conversa fora da faixa entre cem dentro ainda e o caso a investigar, e a media a diluiria ate virar ruido de arredondamento -- apagando exatamente o evento que esta rota existe para achar.

Amostra vazia devolve `z_absoluto_observado: None` e `dentro_da_faixa: None`, nunca `0.0/True`: zero ali se leria como "exatamente na media do treino", que e a afirmacao mais tranquilizadora possivel a respeito de uma amostra que nao existe.

`None` quando o fusor nao tem retrato de treino (nao treinado): ausencia de diagnostico, diferente de um relatorio vazio com cara de saudavel.

NAO ALTERA PREDICAO NENHUMA. E diagnostico.